Marcelo Fernandes de Oliveira Junior
Gustavo Oliveira dos Santos
Christian Lima Santana

Análise Comparativa de Desempenho e Eficiência Energética entre Processamento de OCR em Borda e em Nuvem aplicado à Smart Metering.

São Paulo
2026

Marcelo Fernandes de Oliveira Junior
Gustavo Oliveira dos Santos
Christian Lima Santana

Análise Comparativa de Desempenho e Eficiência Energética entre Processamento de OCR em Borda e em Nuvem aplicado à Smart Metering.

Trabalho de Conclusão de Curso II apresentado ao SENAC Santo Amaro como requisito parcial para conclusão do curso de Bacharelado em Ciência da Computação.

SENAC – Serviço Nacional de Aprendizagem Comercial
Unidade Santo Amaro
Curso de Bacharelado em Ciência da Computação

Orientador: Afonso César Lelis Brandão

São Paulo
2026

Resumo

Este trabalho busca propor uma análise comparativa entre dois métodos de processamento para sistemas de Smart Metering (medição inteligente): Edge Computing (Computação em Borda), com inferência de Optical Character Recognition (OCR) realizada localmente no microcontrolador ESP32-CAM por meio de modelo de aprendizado de máquina treinado de forma supervisionada e quantizado em inteiros de 8 bits, e Cloud Computing (Computação em Nuvem), com inferência realizada em ambiente remoto através de modelo treinado de forma supervisionada e quantizado em ponto flutuante de 32 bits. A motivação reside no elevado custo de substituição dos mais diversos medidores convencionais, amplamente utilizados em ambientes residenciais e privados, por modelos inteligentes nativos, o que favorece o *retrofitting* como alternativa mais economicamente viável. A transmissão dos dados entre os nós de borda e o gateway central é realizada via rede Long Range Wide Area Network (LoRaWan), utilizando o Spreading Factor SF8, configuração que assegura conformidade com o limite regulatório de *dwell time* de 400 ms estabelecido pelo Ato nº 14.448 da ANATEL, ao mesmo tempo em que maximiza o alcance dentro desse limite. A viabilidade do envio de imagens fragmentadas via LoRaWan foi demonstrada analiticamente, com uma imagem de 160×120 pixels em escala de cinza, comprimida utilizando o algoritmo JPEG com aproximadamente 3,10 KB, podendo ser transmitida em 38 pacotes em cerca de 13,3 s de rádio ativo. Os dois fluxos de processamento serão avaliados segundo três métricas quantitativas: acurácia de leitura OCR (%), latência de ponta a ponta (ms) e consumo energético por leitura (mAh). O objetivo deste trabalho é apresentar resultados que orientem a escolha arquitetural mais adequada para implantações de *Smart Metering* em larga escala, considerando o *trade-off* entre eficiência energética, precisão de reconhecimento e complexidade de manutenção do sistema.

Palavras-chave: Smart Metering. ESP32. LoRaWan. OCR.

Abstract

This work aims to propose a comparative analysis between two processing methods for Smart Metering systems: Edge Computing, with OCR inference performed locally on the ESP32-CAM microcontroller using a machine learning model trained in a supervised manner and quantized to 8-bit integers, and Cloud Computing, with inference performed in a remote environment through a model trained in a supervised manner and quantized to 32-bit floating point. The motivation lies in the high replacement cost of the most diverse conventional meters, widely used in residential and private environments, by native smart models. This favors *retrofitting* as a more economically viable alternative. Data transmission between the edge nodes and the central gateway is carried out via a LoRaWan network, using Spreading Factor SF8, a configuration that ensures compliance with the dwell time regulatory limit of 400 ms established by ANATEL Act No. 14.448, while maximizing range within that limit. The feasibility of sending fragmented images via LoRaWan was demonstrated analytically, with a 160×120 pixel grayscale image, compressed using the JPEG algorithm to approximately 3.10 KB, which can be transmitted in 38 packets in about 13.3 s of active radio time. The two processing flows will be evaluated according to three quantitative metrics: OCR reading accuracy (%), end-to-end latency (ms), energy consumption per reading (mAh). The objective of this work is to present results that guide the most appropriate architectural choice for large-scale *Smart Metering* deployments, considering the *trade-off* between energy efficiency, recognition precision, and system maintenance complexity. **Keywords**: Smart Metering. ESP32. LoRaWan. OCR.

Lista de ilustrações

Figura 1 – Campos contidos em um pacote de Uplink 16
Figura 2 – Campos contidos em Frame Header em Downlink 17
Figura 3 – Pilha de elementos das camadas LoRa/LoRaWan 18
Figura 4 – Implementação Típica de Rede LoRaWan 19
Figura 5 – Módulo ESP32-CAM integrado com LoRa 20
Figura 6 – Gateway LoRaWan compatível com placas Raspberry Pi 21
Figura 7 – Distribuição de Trabalhos Relacionados à Computação em IoT 23
Figura 8 – Representação Visual em Grafo da estrutura básica de uma Rede Neural 31
Figura 9 – Definição de ROI de uma Imagem 34
Figura 10 – Arquitetura do sistema e os dois fluxos de processamento 42
Figura 11 – Comparação do desempenho do modelo U-Net Principal (384 × 384) e Compacto (160 × 160) na segmentação de visores. 52
Figura 12 – Evolução do treino do modelo de localização da janela 55
Figura 13 – Evolução do treino do classificador de dígitos (fase A) 57
Figura 14 – Leitura com a janela de dígitos marcada manualmente em cinco capturas do ESP32-CAM 58
Figura 15 – Montagem da medição de corrente do nó sensor 69

Lista de tabelas

Tabela 1 – Strings de busca utilizadas no levantamento bibliográfico. 15
Tabela 2 – Classes de Configuração LoRa 16
Tabela 3 – Parâmetros de Configuração LoRa 18
Tabela 4 – Vantagens e Desvantagens na Adoção de Métodos de Processamento . 24
Tabela 5 – Uplink: Canais de Frequência 26
Tabela 6 – Downlink: Canais de Frequência 26
Tabela 7 – Taxas de transmissão Uplink e configurações LoRaWAN. 27
Tabela 8 – Parâmetros de Downlink Data Rate 27
Tabela 9 – Downlink RX1 Data Rate Mapping 28
Tabela 10 – Componentes de hardware do protótipo 43
Tabela 11 – Principais componentes de software 44
Tabela 12 – Parâmetros do enlace LoRaWAN 46
Tabela 13 – Formato do *payload* de aplicação em cada fluxo 47
Tabela 14 – Tamanho médio do JPEG de três imagens de 160×120 pixels 47
Tabela 15 – Pacotes, tempo de rádio e tempo total de envio por leitura 49
Tabela 16 – Pipelines de OCR desenvolvidos 51
Tabela 17 – Resumo da avaliação do modelo CRNN com CTC no conjunto de teste 53
Tabela 18 – Arquitetura do classificador de dígitos 56
Tabela 19 – Etapas de um ciclo de leitura em cada fluxo 59
Tabela 20 – Uso da PSRAM pelo fluxo em borda 60
Tabela 21 – Tempo de inferência por leitura no nó e no servidor (mediana) 62
Tabela 22 – Plano de experimentação 63
Tabela 23 – Comparação de métricas reportadas 70

Lista de Abreviaturas e Siglas

ABP Activation by Personalization 19, 42, 44, 46

BW Bandwidth 18, 46

CNN Convolutional Neural Network 31, 32, 35

CR Coding Rate 18, 46

CRNN Convolutional Recurrent Neural Network 35

CSS Chirp Spread Spectrum 26

CTC Connectionist Temporal Classification 35

DR Data Rate 26, 27

DSR Design Science Research 38

EIRP Effective Isotropic Radiated Power 28

IA Inteligência Artificial 28, 29

IaaS Infrastructure as a Service 23

IoT Internet of Things 11, 12, 14, 19, 22, 23

ISM Industrial, Scientific and Medical 24, 25

LoRa Long Range 12, 14, 15, 25, 26, 28, 38, 45, 48

LoRaWan Long Range Wide Area Network 2, 3, 13–19, 37, 38, 40–44, 46, 49, 60

LPWAN Low Power Wide Area Network 18

LR-FHSS Long Range Frequency Hopping Spread Spectrum 26

LSTM Long-Short Term Memory 29, 30

OCR Optical Character Recognition 2, 3, 5, 13, 28, 32, 37, 38, 40–44, 46, 49–51, 61, 62, 67

OTAA Over the Air Activation 19

PDR Packet Delivery Ratio 40, 63–65, 68

QAT Quantization-Aware Training 36

RGB Red, Green, Blue 32

RNA Redes Neurais Artificiais 36

RNN Recurrent Neural Network 35

ROI Region of Interest 34

SaaS Software as a Service 23

SF Spreading Factor 18, 26, 45, 46

Sumário

1 INTRODUÇÃO . . . 11
1.1 Contextualização do problema . . . 11
1.2 Formulação do problema . . . 12
1.3 Objetivos e hipótese . . . 13
1.3.1 Objetivo geral . . . 13
1.3.2 Objetivos específicos . . . 13
1.3.3 Hipótese . . . 13
1.4 Justificativa . . . 14

2 REVISÃO BIBLIOGRÁFICA / TRABALHOS RELACIONADOS . . 15
2.1 Metodologia da pesquisa bibliográfica . . . 15
2.2 Fundamentos teóricos . . . 15
2.2.1 LoRa . . . 15
2.2.2 LoRaWan . . . 18
2.2.2.1 Dispositivos Finais (End Nodes) . . . 19
2.2.2.2 Gateways . . . 20
2.2.2.3 Servidores de Rede e Aplicação . . . 22
2.2.3 Edge Computing e Cloud Computing . . . 23
2.2.4 Restrições Regulatórias . . . 24
2.2.5 Inteligência Artificial . . . 28
2.2.5.1 Machine Learning . . . 28
2.2.5.1.1 Aprendizado Supervisionado . . . 29
2.2.5.1.2 Deep Learning . . . 29
2.2.5.1.3 O que são LSTMs. . . . 29
2.2.5.1.4 Limitações em *Edge AI* e alternativas convolucionais. . . . 30
2.2.5.2 Redes Neurais . . . 30
2.2.5.2.1 Redes Neurais Convolucionais . . . 31
2.2.5.2.2 Camadas que Constituem uma CNN . . . 31
2.2.5.2.3 Camada Convolucional . . . 32
2.2.5.2.4 Camada de Pooling . . . 32
2.2.5.2.5 Camada Totalmente Conectada . . . 32
2.2.5.3 OCR - Fundamentos e Escopo . . . 32
2.2.5.3.1 Pipeline Tradicional . . . 33
2.2.5.3.2 Evolução Tecnológica: dos Métodos Heurísticos ao Deep Learning . . . 33
2.2.5.4 Arquitetura U-Net - Segmentação de ROI . . . 34

2.2.5.5 Arquitetura CRNN e CTC . . . 35
2.2.5.6 Quantização e QAT . . . 36
2.3 Trabalhos correlatos . . . 36
2.3.1 Trabalho 1: Solução IoT para Digitalização de Medidores de Consumo por Visão Computacional e MicroML . . . 37
2.3.2 Trabalho 2: Sistema de Gerenciamento Inteligente de Água Utilizando Sensores Baseados em IoT . . . 37
2.3.3 Trabalho 3: Rede de Sensores Sem Fio Habilitada para LoRa para Leitura e Faturamento de Consumo de Água Usando Reconhecimento Óptico de Caracteres . . . 38

3 DESENVOLVIMENTO / IMPLEMENTAÇÃO / EXPERIMENTOS 40
3.1 Metodologia da pesquisa . . . 40
3.1.1 Desempenho da Rede de Comunicação . . . 40
3.1.2 Acurácia da Visão Computacional (OCR) . . . 40
3.1.3 Autonomia Energética . . . 41
3.2 Detalhes da implementação . . . 41
3.2.1 Visão geral da solução . . . 41
3.2.2 Componentes de hardware . . . 42
3.2.3 Componentes de software . . . 43
3.2.4 Simplificações em relação a uma implantação real . . . 44
3.2.5 Enlace LoRaWAN . . . 45
3.2.5.1 Estudo preliminar e escolha da configuração de rádio . . . 45
3.2.5.2 Formato das mensagens . . . 46
3.2.5.3 Preparo da imagem e fragmentação . . . 47
3.2.5.4 Tempo no ar . . . 48
3.2.5.5 Recepção, descriptografia e remontagem . . . 49
3.2.6 Reconhecimento dos dígitos . . . 50
3.2.6.1 Evolução das abordagens . . . 50
3.2.6.1.1 Teste de viabilidade com OCR genérico. . . . 50
3.2.6.1.2 Pipelines avaliados. . . . 50
3.2.6.2 Detalhamento dos modelos explorados (U-Net e CTC) . . . 51
3.2.6.2.1 O modelo de localização U-Net (Pipelines A e C). . . . 51
3.2.6.2.2 O reconhecedor CRNN com CTC (Pipeline A). . . . 53
3.2.6.2.3 Inviabilidade do Pipeline A no hardware alvo. . . . 53
3.2.6.3 Detalhes do Pipeline B (Adotado) . . . 54
3.2.6.4 Conjuntos de dados . . . 54
3.2.6.5 Modelo de localização da janela . . . 55
3.2.6.6 Classificador de dígitos . . . 56
3.2.6.7 Quantização e exportação . . . 57

3.2.6.8 Leitura com janela calibrada na instalação . . . 57
3.2.7 Firmware do nó sensor . . . 59
3.2.7.1 Ciclo de operação . . . 59
3.2.7.2 Execução dos modelos no microcontrolador . . . 60
3.2.7.3 Firmware de depuração e calibração . . . 61
3.2.8 Servidor da aplicação . . . 61
3.2.8.1 Tempo de inferência no servidor . . . 62
3.2.8.2 Recursos disponíveis apenas no servidor . . . 62
3.3 Experimentos e cenário de testes . . . 63
3.3.1 Métricas . . . 64
3.3.1.1 Reconhecimento . . . 64
3.3.1.2 Latência e entrega . . . 64
3.3.1.3 Energia . . . 65
3.3.2 Cenário de bancada . . . 65
3.3.3 Avaliação do reconhecimento . . . 66
3.3.3.1 Efeito da quantização (H1) . . . 66
3.3.3.2 Leitura de ponta a ponta . . . 66
3.3.4 Latência e entrega (H2) . . . 67
3.3.5 Consumo de energia (H3) . . . 68
3.4 Resultados e análises . . . 70

4 CONCLUSÃO . . . 71
4.1 Retomada do problema e dos objetivos . . . 71
4.2 Contribuições para a Computação . . . 71
4.3 Limitações e trabalhos futuros . . . 71

Observações importantes . . . 72

Apresentação oral (banca) . . . 73

REFERÊNCIAS . . . 75

1 Introdução

Atualmente, a implantação de conceitos de Internet of Things (IoT) nos mais diferentes dispositivos presentes no dia a dia de milhões de pessoas ao redor do mundo está numa crescente jamais antes vista. Como explicado por Tedjojuwono e Jahja (2024), a adoção de dispositivos IoT é um dos principais pilares na ascensão da indústria 4.0, principalmente na pauta da construção de cidades inteligentes. E ainda, por característica, tais tecnologias viabilizam a coleta de dados de forma constante, promovendo insumos para realização de análises, manutenções e aprimoramentos de sistemas através de planos de ação mais eficientes e eficazes.

Em contrapartida, como explicado por Fortune Business Insights (2025) e Uggeri e Simeon (2020), o alto custo inicial exigido para realizar a transição de dispositivos analógicos para soluções inteligentes nativas constitui o principal entrave para a adoção de tais tecnologias em larga escala. A complexidade de infraestrutura necessária para a substituição física dos medidores limita a viabilidade econômica de projetos em áreas privadas de pequeno e médio porte.

Abordando a pauta de automação residencial e comercial, o tema de smart metering surge como ponto focal na discussão. Como evidenciado pelo contraste em abordagens de Vilela (2021) e Adam (2022), o desenvolvimento de sistemas focados em smart metering enfrenta um impasse arquitetônico, onde o método de processamento divergia entre os projetos.

A implementação proposta por Vilela (2021) é a denominada Edge Computing, onde o processamento é realizado localmente no dispositivo final, assim otimizando recursos energéticos e latência. Em oposição, a proposta de Adam (2022), realizando processamento em cloud (Cloud Computing), surge como uma solução focada em facilidade de manutenção, utilização de softwares mais robustos e menor complexidade de implementação.

Neste contexto, a finalidade deste trabalho é realizar a comparação entre os métodos de processamento citados, definindo limites, vantagens e desvantagens sob determinado ambiente.

1.1 Contextualização do problema

O mundo, neste momento, está passando pelo que pode ser chamado de era da digitalização, havendo severas transformações no modo de transmissão de dados e informações (TEDJOJUWONO; JAHJA, 2024). Atualmente, a humanidade encontra-se sob influência da expansão da denominada indústria 4.0, que tem como foco a digitalização dos processos de manufatura e produção, mas que engloba inclusive o modo de vida e acesso a informações da sociedade sob o braço tecnológico denominado *Cidades Inteligentes*.

Neste contexto, a utilização de hardware inteligente deixou de ser mero luxo de grandes corporações para se tornar o paradigma central na mudança citada anteriormente. Em relação a equipamentos de medição inteligentes, também classificados como Smart Metering, hoje já há uma série de avanços que possibilitam o processamento de dados de maneira mais veloz e eficiente. Mas essa evolução também acompanhou a necessidade de adaptação de infraestrutura, assim agregando altos custos para a transição das tecnologias analógicas para o digital (ALVISI et al., 2019).

Como apresentado por Alvisi et al. (2019), o retrofitting é uma das soluções mais difundidas para contornar o alto custo de digitalização de sistemas nativamente analógicos. Tal conceito tem como objetivo realizar a digitalização de sistemas analógicos reaproveitando o hardware já existente. Essa concepção é amplamente abordada na academia, havendo grandes teses, trabalhos e dissertações sobre o tema.

Diante de tal abordagem para redução de custos, a aplicabilidade da técnica de retrofitting enfrenta um impasse. Atualmente, na academia encontram-se mais difundidos trabalhos que abordam o uso de microcontroladores aliados à tecnologia Long Range (LoRa) utilizando o método de processamento em borda. Como explicado por Vilela (2021), a utilização deste método traz grandes vantagens em relação à latência e otimização de consumo energético. Em oposição, como Andriulo et al. (2024) apresenta, o método de processamento em cloud destaca-se por proporcionar maior granularidade de dados, simplificação no desenvolvimento de firmware e acurácia ligeiramente melhor em relação ao conceito de Edge Computing. Todavia, permanece a dúvida de qual método de processamento utilizar no desenvolvimento de uma solução IoT baseada na técnica de retrofitting. A resolução do problema criará um alicerce, auxiliando o desenvolvimento de futuras ferramentas que, por concepção, apresentarão a mesma natureza de problemática.

1.2 Formulação do problema

Com base no contexto apresentado, revela-se a dúvida sobre os impactos arquitetônicos causados pela escolha de métodos de processamento em determinado projeto. Este trabalho busca responder o questionamento: De que maneira a adoção de determinado método de processamento, em borda ou em nuvem, impacta a eficiência energética, acurácia, complexidade de desenvolvimento de firmware e desempenho computacional de um sistema IOT voltado para coleta de dados de medidores analógicos?

1.3 Objetivos e hipótese

1.3.1 Objetivo geral

Comparar a viabilidade técnica, a acurácia e o consumo energético entre o processamento de OCR realizado localmente no ESP32-CAM (abordagem Edge Computing) e a inferência OCR realizada remotamente em servidor na nuvem (Cloud), utilizando a rede LoRaWan como meio de transmissão.

1.3.2 Objetivos específicos

1. Implementar os dois fluxos de processamento que compõem o experimento comparativo: o fluxo Edge, em que o OCR ocorre localmente no ESP32-CAM; e o fluxo Cloud, em que a imagem é transmitida para processamento remoto;

2. Configurar a rede LoRaWan conectando os nós da rede com o objetivo de reduzir ao máximo o consumo energético;

3. Avaliar e comparar os dois fluxos segundo as métricas: precisão de leitura (%), latência de processamento (ms), consumo energético por leitura (mAh) e volume de dados transmitidos (bytes/leitura), identificando o *trade-off* entre processamento local e transmissão.

1.3.3 Hipótese

Considerando o trade-off apresentado, foram estabelecidas hipóteses com o objetivo de verificar se a utilização de determinada abordagem apresenta vantagens quando observadas sob o mesmo cenário. As hipóteses orientam a definição dos experimentos e a análise dos resultados obtidos.

- **H1**: A quantização em inteiros de 8 bits, aplicada ao modelo implementado no fluxo de processamento em borda, causa redução de acurácia em comparação ao modelo quantizado em ponto flutuante de 32 bits, implementado no fluxo de processamento em nuvem.

- **H2**: O método de processamento local pior desempenho computacional no dispositivo final, em comparação com o método de processamento remoto.

- **H3**: A abordagem de processamento em nuvem apresenta autonomia energética inferior durante a transmissão de mensagens quando comparada ao método de processamento local.

1.4 Justificativa

Este trabalho propõe uma abordagem analítica sobre métodos de processamento implementados em sistemas baseados em tecnologias IoT com o objetivo de aplicação da técnica de retrofitting em instrumentos de medição analógicos. Tal temática tem como alicerce a lacuna literária identificada por Andriulo et al. (2024), na qual carecem trabalhos que apresentam informações úteis para a etapa de desenvolvimento arquitetural de tais ferramentas.

Diante de tal contexto, revela-se a importância acadêmica deste trabalho, apresentando de forma científica comparações entre os métodos de processamento, definindo limites e estipulando vantagens na adoção de determinado método. E ainda, este trabalho tem como pauta discorrer sobre os temas: desenvolvimento de sistemas baseados em tecnologia LoRa, aplicação de algoritmos de packet loss (perda de pacotes) e uplink de dados segmentados em pacotes via LoRaWan.

O desenvolvimento deste trabalho tem por foco a disponibilização de insumos para tomada de decisões no desenvolvimento de sistemas IoT que empregam protocolos de rede de baixa largura de banda.

2 Revisão bibliográfica

2.1 Metodologia da pesquisa bibliográfica

A pesquisa bibliográfica foi conduzida de forma sistemática, estruturando-se em três temas principais: Hardware, Software e Redes. Para a seleção dos trabalhos, utilizaram-se as bases de dados Google Scholar, MDPI e Intechopen.

O processo de seleção seguiu um fluxo de filtragem (funil). Inicialmente, foram identificados 120 artigos por meio da análise de títulos, utilizando strings de busca majoritariamente em inglês, tais como “iot AND energy efficiency” e “ESP32 AND camera monitoring”. Como ferramentas auxiliares na descoberta de conexões entre autores e na triagem inicial de relevância, foram utilizadas as plataformas ResearchRabbit e Gemini Deep Research.

Na etapa de refinamento, aplicou-se a leitura dos resumos (abstracts) e a verificação de disponibilidade integral dos textos. Foram adotados como critérios de inclusão: artigos publicados entre 2020 e 2026, relevância direta para sistemas de telemetria e foco em implementações práticas. Como critério de exclusão, descartaram-se trabalhos com acesso restrito ou que não apresentassem detalhes técnicos sobre a integração dos eixos propostos.

Tabela 1 – Strings de busca utilizadas no levantamento bibliográfico.

ID	**Eixo Temático**	**String de Busca**
ST01	Redes	"IoT"AND "LoRaWan"AND "water waste"
ST02	Contexto	smart water meter market
ST03	Contexto	smart water meter costs and benefits
ST04	Hardware	"IoT"AND "energy efficiency"
ST05	Hardware	"ESP32"AND "camera monitoring"
ST06	Hardware	"ESP32-CAM"AND "analog meter"AND "OCR"
ST08	Software	"automatic meter reading"OCR accuracy "error rate"evaluation
ST09	Redes	"packet delivery ratio"LoRaWan "latency"monitoring system
ST010	Contexto	"water consumption predictionmean absolute error"forecasting metrics

2.2 Fundamentos teóricos

2.2.1 LoRa

Como explicado por Pastório et al. (2022), LoRa é uma tecnologia, patenteada e desenvolvida pela Semtech, que tem como foco viabilizar a transmissão de dados sem fio em longo alcance através de radiofrequência.

A rede LoRaWan é constituída por dispositivos finais, gateways e servidores de rede/aplicação. Com o objetivo de realizar um gerenciamento mais fino de consumo

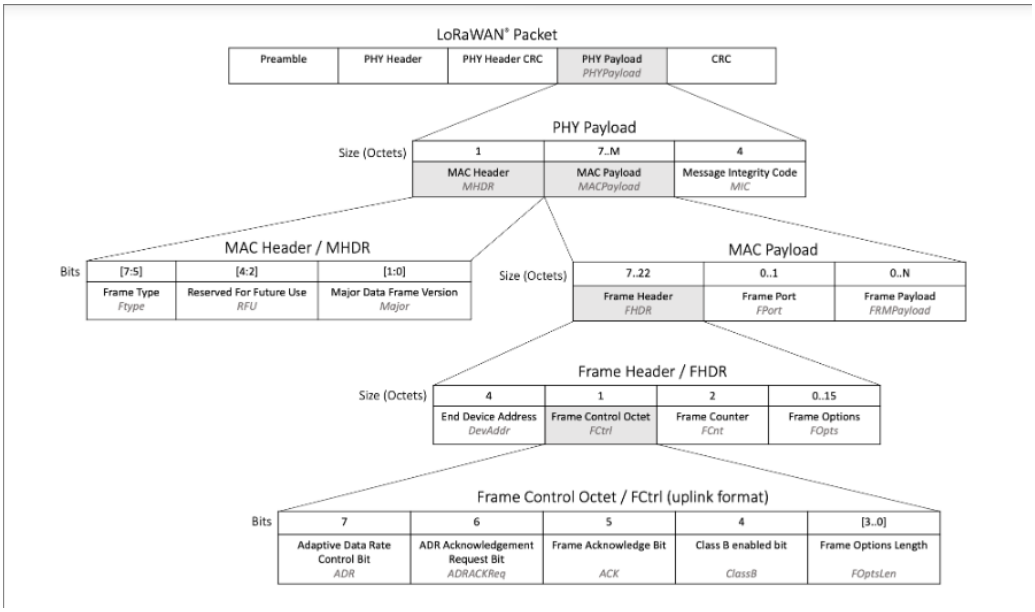
energético, a comunicação entre os dispositivos finais e servidores de rede é definida por três classes como demonstrado na tabela 2.

Tabela 2 – Classes de Configuração LoRa

Classe	**Descrição**	**Característica**
Classe A	O dispositivo final é habilitado a receber downlink imediatamente após ser realizado um envio de mensagem (uplink).	Menor consumo energético e menor tráfego de mensagens.
Classe B	O dispositivo final é habilitado a receber downlink em períodos agendados e conhecidos.	Consumo energético significativamente maior que a classe A, mas viabiliza comunicação mais frequente entre servidor e dispositivo final.
Classe C	O dispositivo final é continuamente habilitado a receber downlink.	Consumo energético muito alto, mas viabiliza a comunicação entre as partes de forma quase ininterrupta.

Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024b).

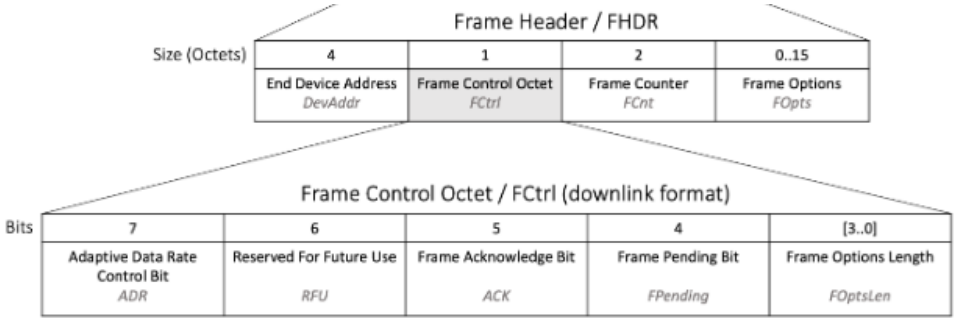
Figura 1 – Campos contidos em um pacote de Uplink



Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024c).

A figura 6 apresenta a estrutura de pacote de rede que é utilizada na tecnologia LoRa. Tal estrutura é utilizada tanto no envio de dados pelo dispositivo final (uplink) quanto no recebimento de dados (downlink).

Figura 2 – Campos contidos em Frame Header em Downlink



Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024c).

A estrutura do pacote de rede de uplink e downlink é a mesma, mas há diferenças em flags específicas entre os tipos de transmissão de dados. A figura 2 apresenta a estrutura do "Frame Header / FHDR" utilizada em downlink. As estruturas de pacote divergem na flag de classe B, no uplink, por meio da qual o dispositivo final informa ao servidor estar no modo de recebimento de mensagens em tempo programado, e, no pacote downlink, na flag FPending, utilizada para informar a existência de mais mensagens a serem enviadas pelo servidor/gateway.

Conforme apresentado pela LoRa Alliance Regional Parameters Workgroup (2025) o tamanho máximo de MACPayload, respeitando o padrão regulatório AU915, que será mais bem explicado e abordado posteriormente, em bytes, é variável e dependente do tamanho ocupado pelo atributo Fopts, que é utilizado para solicitar alterações de parâmetros de rede ao dispositivo final. Não utilizando repetidores de sinal que interligam dispositivos finais com gateways, o tamanho máximo de MACPayload é de 250 bytes, restando em torno de 227 bytes utilizáveis em payload.

Para viabilizar a modulação LoRa em dispositivo final, é necessário realizar a configuração de três parâmetros fundamentais. A tabela 3 apresenta as definições dos principais parâmetros que ajustam o funcionamento da rede LoRaWan.

Tabela 3 – Parâmetros de Configuração LoRa

Parâmetro	**Definição**
Bandwidth (BW)	Determina a capacidade máxima de transmissão em determinado período de tempo (medido em kHz).
Spreading Factor (SF)	Determina o alcance do sinal de rádio; o SF é determinado por classes, nas quais, quanto maior a classe, maior a área de alcance do sinal.
Coding Rate (CR)	Número de bits destinados à redundância de dados, utilizado para possibilitar recuperação da mensagem em caso de erros na transmissão.

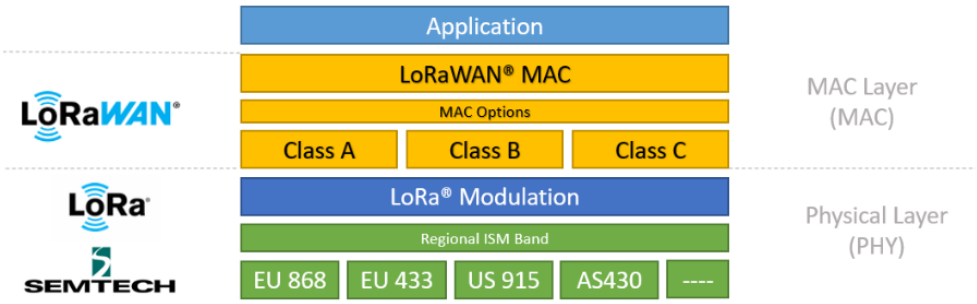
Fonte: Adaptado de Pastório et al. (2022).

2.2.2 LoRaWan

O LoRaWan é um protocolo de comunicação baseado na tecnologia Low Power Wide Area Network (LPWAN), que possui o objetivo de viabilizar a comunicação em grandes distâncias, conciliada ao baixo consumo energético. O protocolo é open-source e mantido pela LoRa Alliance. Como citado anteriormente, a rede LoRaWan é formada por dispositivos finais, gateways, servidores de rede e servidores de aplicação (PASTóRIO et al., 2022).

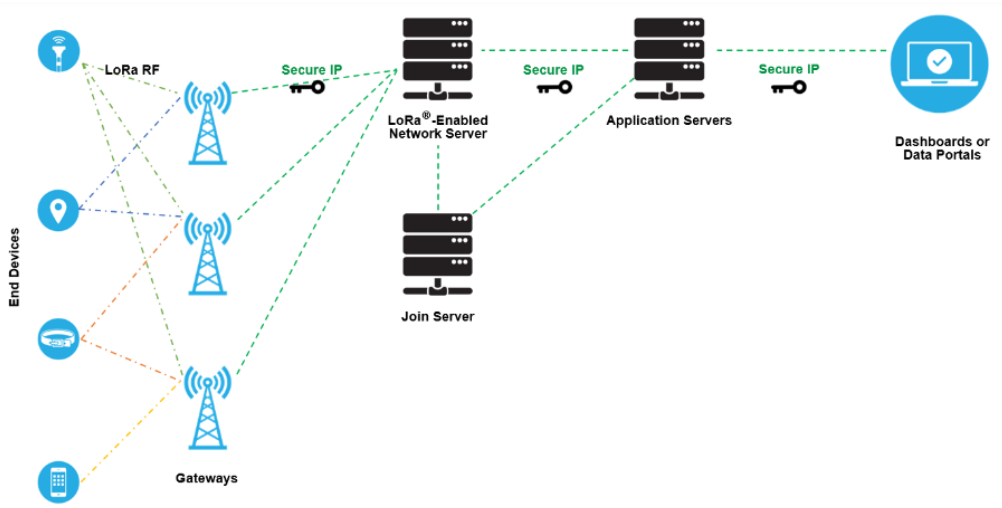
A figura 3 apresenta a pilha de elementos das camadas MAC e Física que compõem a arquitetura do protocolo LoRaWan.

Figura 3 – Pilha de elementos das camadas LoRa/LoRaWan



Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024a).

Figura 4 – Implementação Típica de Rede LoRaWan



Fonte: Adaptado de Semtech Corporation (Semtech Corporation, 2024a).

A figura 4 apresenta a arquitetura típica de um sistema IoT que utiliza como base o protocolo LoRaWan para viabilizar a comunicação entre os dispositivos. Cada elemento que compõe uma rede LoRaWan possui uma responsabilidade; neste momento, é extremamente válido analisá-las individualmente:

2.2.2.1 Dispositivos Finais (End Nodes)

Semtech Corporation (2024a) define que dispositivos finais habilitados para LoRaWan são sensores ou atuadores que são conectados a gateways via modulação de radiofrequência LoRa.

Na maioria das aplicações, esses dispositivos são autônomos, dependentes de baterias como fonte energética primária e possuem como responsabilidade digitalizar condições físicas como temperatura, captura de imagens, níveis de tremulação, entre outros.

Em relação às possíveis configurações que podem ser aplicadas nos dispositivos finais, como apresentado por Pastório et al. (2022), cada nó pode assumir uma das configurações que a tabela 2 apresenta. Outro parâmetro configurável é o tipo de autenticação dos dispositivos em rede, havendo o padrão Over the Air Activation (OTAA), no qual o próprio dispositivo gerencia a conexão à rede, e a Activation by Personalization (ABP), na qual os dispositivos são vinculados de forma manual a uma rede em específico, não passando pelo processo de entrada implementado no OTAA.

O LoRaWan utiliza o padrão AES de criptografia simétrica, aplicado tanto na camada de aplicação quanto na camada de rede.

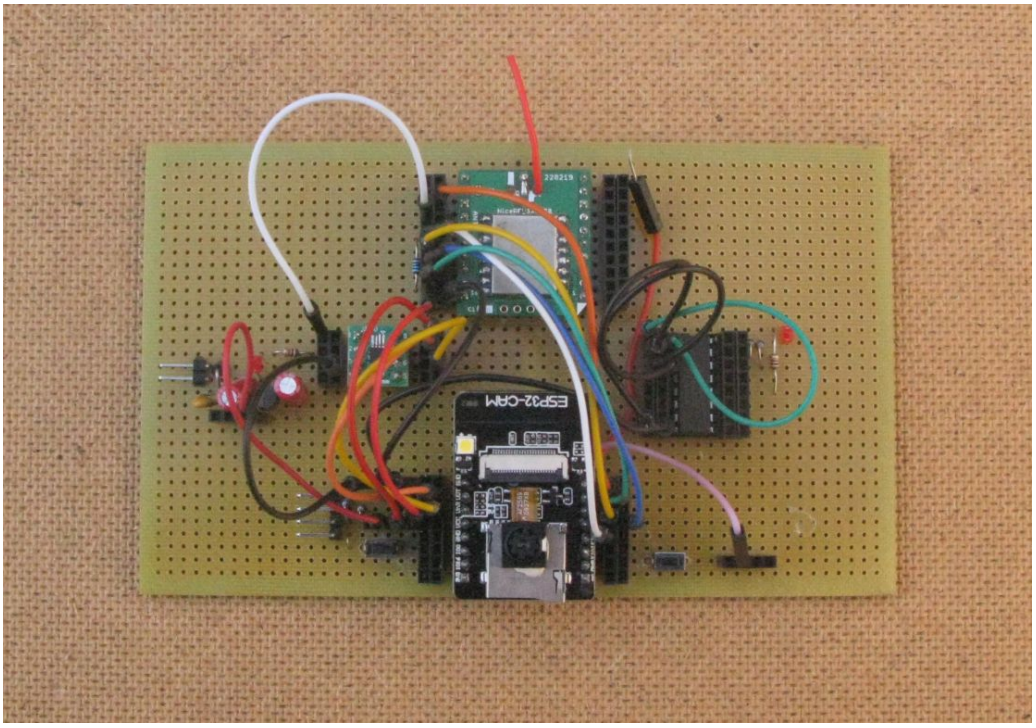
Os dispositivos finais, corriqueiramente, são desenvolvidos sobre microprocessadores, podendo tanto utilizar placas que já incluem módulos LoRaWan nativos quanto serem desenvolvidos sobre plataformas como ESP32, Arduino, Raspberry Pi, entre outras. A

partir da construção do dispositivo, ou acoplamento de módulos, possibilita-se a captura de dados em ambiente (PASTóRIO et al., 2022).

Como destacado por Sari et al. (2022) e Ahmad et al. (2021), o ESP32 destaca-se nesse cenário como uma plataforma robusta de baixo custo, integrando conectividade sem fio e múltiplos modos de operação. Diferentemente de arquiteturas mais simples, o ESP32 permite a execução de tarefas complexas e segurança criptográfica sem comprometer severamente o ciclo de vida da bateria, desde que devidamente configurado.

Segundo Ferreira e Cavalcante (2020), a vida útil da bateria é influenciada criticamente não apenas pelo hardware, mas pela eficiência das rotinas de escrita em periféricos e pela otimização do firmware. Diante de tal afirmação, destaca-se a função *deep sleep* presente no ESP32 que possui como foco a minimização do consumo energético quando em estado ocioso.

Figura 5 – Módulo ESP32-CAM integrado com LoRa



Fonte: (ROBINSON, 2021).

2.2.2.2 Gateways

O gateway, como demonstrado na figura 4, é um elemento da rede LoRaWan que tem como responsabilidade agrupar dispositivos finais e viabilizar a comunicação entre o dispositivo final e o servidor de rede (PASTóRIO et al., 2022).

A comunicação entre o dispositivo final e o gateway é realizada via protocolo LoRaWan, na qual o gateway pode realizar a tarefa de registrar tráfego de mensagens. Como explicado por Semtech Corporation (2024a), os dispositivos finais, ao realizar o envio de uplink,

distribuem as suas mensagens a todos os gateways alcançáveis dentro da mesma rede; tal função tem como objetivo garantir que ao menos um dos gateways da rede receba e transmita a mensagem ao servidor de rede.

Os gateways operam na camada física e, convencionalmente, assumem a responsabilidade apenas de viabilizar o tráfego de requisições uplink e downlink e realizar a verificação de integridade do pacote, não realizando processamento intermediário da mensagem para enviá-la ao servidor de rede (Semtech Corporation, 2024a).

A comunicação entre o gateway e o servidor de rede, corriqueiramente, é estabelecida utilizando protocolo TCP/IP. Toda comunicação dentro do protocolo LoRaWan é bidirecional (PASTóRIO et al., 2022).

É possível encontrar gateways prontos de múltiplos canais sendo comercializados, mas é plenamente viável desenvolver gateways a partir de módulos e placas de prototipagem como a apresentada na figura 5.

Figura 6 – Gateway LoRaWan compatível com placas Raspberry Pi



Fonte: Catálogo de Produtos (Radioenge, 2026).

2.2.2.3 Servidores de Rede e Aplicação

Como apresentado por Pastório et al. (2022), servidores de rede e servidores de aplicação, dada a estrutura em que foram desenvolvidos, tanto podem ser implementados sob o mesmo hardware, como podem estar fisicamente distantes, realizando comunicação através de protocolo TCP/IP.

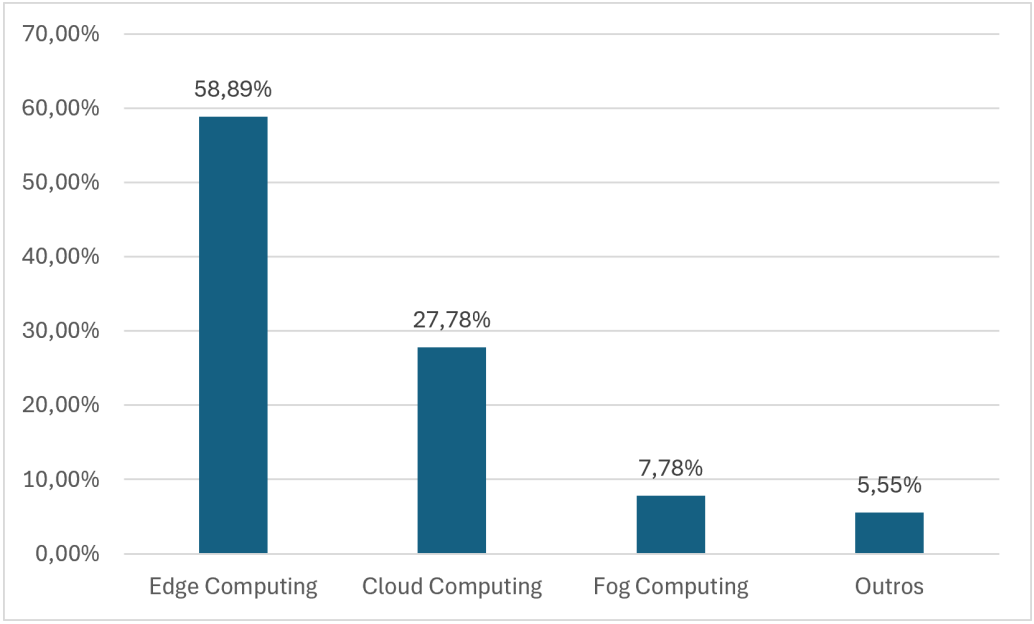
Como explicado pela Semtech Corporation (2024a), servidores de rede são responsáveis por gerenciar dinamicamente a rede de dispositivos, adaptando o sistema para se ajustar a um cenário de constante mudança de parâmetros. Entre as responsabilidades dos servidores de rede estão: receber, descartar duplicatas e descriptografar mensagens enviadas por uplink, realizar autenticação, garantir segurança com o protocolo AES, confirmar recebimento de mensagens, responder a requisições de camada MAC, realizar configuração de dispositivos finais através de requisições downlink, entre outras.

Servidores de Aplicação, por sua vez, são responsáveis por receber pacotes enviados pelo servidor de rede, tratar e disponibilizar em formato de informação ao usuário os dados extraídos pelos dispositivos finais. Como a Semtech Corporation (2024a) explica, é uma de suas responsabilidades também gerenciar os payloads de downlink a serem enviados aos dispositivos finais.

Ambos os tipos de servidores possuem diversas soluções prontas para implementação tanto em ambiente local quanto em nuvem. Para servidores de rede, destaca-se o Chirp Stack, que tem por foco o gerenciamento de dispositivos de IoT em rede privada. Já para servidores de aplicação, o Tago IO mostra-se como uma ferramenta robusta, com foco corporativo e ampla gama de ferramentas voltadas para gerenciamento de dispositivos (PASTóRIO et al., 2022).

2.2.3 Edge Computing e Cloud Computing

Figura 7 – Distribuição de Trabalhos Relacionados à Computação em IoT



Fonte: Adaptado de Andriulo et al. (2024).

Edge Computing (computação em borda), em IoT, é uma arquitetura baseada em processamento descentralizado de dados, na qual ocorre o processamento com o objetivo de construção de informações a partir de dispositivos que têm por responsabilidade inicial a extração de dados do ambiente real. Como explicado por Andriulo et al. (2024), o gráfico apresentado na figura 7 demonstra que Edge Computing é a abordagem mais amplamente difundida na literatura, sendo uma solução voltada à segurança de informações e à otimização de redes de baixa largura de banda, promovendo possível redução de custos operacionais.

Cloud Computing (computação em nuvem), ainda no ambiente IoT, é um tipo de arquitetura pautado pela alta escalabilidade de armazenamento de dados e poder computacional centralizado ou não. Como explicado também por Andriulo et al. (2024), este tipo de arquitetura tem como prós a possível adoção de provedores de nuvem, acoplando serviços como Infrastructure as a Service (IaaS), Software as a Service (SaaS), entre outros, assim reduzindo investimentos iniciais, apresentando maior facilidade de manutenção sistêmica e menor complexidade no desenvolvimento de firmware.

Tabela 4 – Vantagens e Desvantagens na Adoção de Métodos de Processamento

Método	**Vantagens**	**Desvantagens**
Edge Computing	• Menor latência na transmissão de mensagens; • Menor tráfego de mensagens em rede; • Foco em segurança de dados;	• Possíveis altos custos iniciais; • Capacidade computacional limitada; • Dificuldades de execução de modelos de machine learning; • Dificuldades em escalabilidade;
Cloud Computing	• Maior poder computacional disponível; • Escalabilidade; • Possível menor custo inicial;	• Alta latência; • Maior risco em segurança de dados; • Maior quantidade de pacotes enviados à rede;

Fonte: Adaptado de Andriulo et al. (2024).

A tabela 4 explicita que tanto Cloud Computing quanto Edge Computing trazem prós e contras em relação à sua implementação em projetos. Cada projeto tem uma necessidade a ser suprida, e não se atentar a nuances do processo de modelagem sistêmica pode acarretar a absorção de desvantagens com as quais o projeto não está preparado para lidar.

2.2.4 Restrições Regulatórias

Ao redor do mundo, protocolos de transmissão que utilizam ondas de rádio como meio de propagação de sinal têm seu funcionamento regulado por órgãos competentes pela homologação de dispositivos de comunicação. No Brasil, a ANATEL (Agência Nacional de Telecomunicações) é a responsável por regular, fiscalizar e homologar dispositivos com o foco de estabelecer redes de comunicação.

O AU915 é a restrição imposta pela ANATEL no Ato nº 14.448, estabelecida em 4 de dezembro de 2017 (ANATEL, 2017), em que se adota o padrão australiano de radiocomunicação. Visto que o Brasil adotou o padrão americano de distribuição de canais, o Industrial, Scientific and Medical (ISM), o AU915 foi escolhido por conta da já existente implantação de infraestrutura de rádio que ocupava parcialmente as faixas abaixo de 915 MHz; portanto, o padrão americano (US915), que funciona em uma faixa de frequência entre 902 e 928 MHz, tornou-se inadequado para implantação em território brasileiro.

O ISM foi desenvolvido em Atlanta em 1947, elaborado pela FCC (Federal Communications Commission), com o objetivo de delimitar internacionalmente os espectros de frequências de rádio para as mais diversas finalidades. Em 1985, a FCC autorizou o uso livre de espectros de rádio em bandas ISM sem a necessidade de licenciamento para utilização em determinados espectros de frequência.

Entre as regulações implementadas pelos mais diversos países ao redor do mundo, dois termos ganham grande relevância. O *duty cycle* é a relação entre o tempo em que um dispositivo ocupa um canal de rádio e o tempo total de observação. Já o *dwell time* é o tempo máximo de ocupação contínua de uma frequência por transmissão (ADAM, 2022).

Tendo o AU915 operando entre as faixas de frequência de 915 e 928 MHz, a norma brasileira não impõe restrição de duty cycle; em seu lugar, o Ato nº 14.448 estabelece que, para sistemas de salto em frequência operando nas faixas de 902–907,5 MHz e 915–928 MHz com largura de faixa de canal inferior a 250 kHz, caso do LoRa com 125 kHz, o tempo médio de ocupação de qualquer radiofrequência não deve ser superior a 0,4 segundos em um intervalo de 14 segundos, com uso mínimo de 35 canais de salto.

O mesmo Ato prevê, no item 10.3, um segundo enquadramento para equipamentos que operam nessas faixas: o de sistemas que empregam modulação digital. Nesse caso, não há exigência de salto em frequência nem limite de ocupação por canal, mas o equipamento deve atender a três requisitos: largura de faixa a 6 dB de, no mínimo, 500 kHz; potência de pico máxima de 1 W na saída do transmissor; e densidade espectral de potência de, no máximo, 8 dBm em qualquer faixa de 3 kHz. Antenas com ganho superior a 6 dBi exigem a redução da potência de transmissão (item 10.5) (ANATEL, 2017). Como os canais de 125 kHz não atingem a largura de faixa mínima, o enquadramento no item 10.3 só é possível com os canais de 500 kHz do LoRa, o que permite a operação em um canal fixo.

O AU915 utiliza 72 canais de uplink, sendo 64 de 125 kHz e 8 de 500 kHz, e 8 canais de downlink que se distribuem entre os espectros de frequência delimitados pela restrição regulatória. No caso de uplink, como mostrado pela tabela 5, os canais de 125 kHz são numerados entre 0 e 63 e distribuídos com espaçamentos de 200 kHz, iniciando em 915,2 MHz até 927,8 MHz, enquanto os de 500 kHz são numerados entre 64 e 71 e distribuídos com espaçamentos de 1,6 MHz, de 915,9 MHz até 927,1 MHz. Já no caso de downlink, como mostrado na tabela 6, os canais são numerados de 0 a 7 e distribuídos entre 923,3 e 927,5 MHz com espaçamentos de 500 kHz (LoRa Alliance Regional Parameters Workgroup, 2025).

Tabela 5 – Uplink: Canais de Frequência

Canal	**Frequência Inicial (MHz)**
0	915,2
1	915,4
2	915,6
...	...
63	927,8

Tabela 6 – Downlink: Canais de Frequência

Canal	**Frequência Inicial (MHz)**
0	923,3
1	923,8
2	924,3
...	...
7	927,5

O termo salto de frequência especificado pelo AU915 trata-se de um padrão de envio de pacotes em canais de frequência. Na tecnologia LoRa, adotam-se dois padrões de envio: o Chirp Spread Spectrum (CSS), no qual o uplink é transmitido em um canal escolhido de forma pseudoaleatória dentro dos limites regulatórios especificados, em que cada pacote é enviado de forma atomizada, e, em downlink, a mensagem é enviada no mesmo canal em que o uplink foi recebido (Semtech Corporation, 2024a); e o Long Range Frequency Hopping Spread Spectrum (LR-FHSS), que tem por objetivo ampliar a capacidade de escalonamento do sistema, sendo aplicável apenas em uplink, funcionando de forma a segmentar a mensagem em diversos pacotes e distribuí-los em diversos canais, assim prevenindo a sobrecarga de canal (LoRa Alliance Regional Parameters Workgroup, 2025).

Outro importante ponto para adequação aos limites regulatórios brasileiros é a configuração de Data Rate (DR) e, por consequência, do SF. Como explicado pela LoRa Alliance Regional Parameters Workgroup (2025), DR é um indicador de velocidade de transmissão de dados em pacote, assim como o SF, organizado em classes que vão de 0 a 15, em que, quanto menor a classe, menor é a taxa de transmissão de dados em bit por requisição. Este indicador está extremamente vinculado ao SF, de forma que quanto menor a classe de SF, maior é a classe de DR, evidenciando, assim, menor tempo de pacote no ar, e, ao contrário, quanto maior a classe SF, menor é a classe DR. As tabelas 7 e 8 demonstram como cada classe de DR impacta no funcionamento de um sistema baseado em tecnologia LoRa.

Tabela 7 – Taxas de transmissão Uplink e configurações LoRaWAN.

Uplink Data Rate	**Configuration**	**Indicative Physical Bit Rate [bit/sec]**
0	LoRa: SF12 / 125 kHz	250
1	LoRa: SF11 / 125 kHz	440
2	LoRa: SF10 / 125 kHz	980
3	LoRa: SF9 / 125 kHz	1760
4	LoRa: SF8 / 125 kHz	3125
5	LoRa: SF7 / 125 kHz	5470
6	LoRa: SF8 / 500 kHz	12500
7	LR-FHSS CR1/3: 1.523 MHz BW	162
8	RFU	
9	LoRa: SF6 / 125 kHz	9375
10	LoRa: SF5 / 125 kHz	15625
11-14	RFU	
15	Defined in [TS001][3]	

Fonte: Adaptado de LoRa Alliance, (LoRa Alliance Regional Parameters Workgroup, 2025).

Tabela 8 – Parâmetros de Downlink Data Rate

Downlink Data Rate	**Configuration**	**Indicative Physical Bit Rate [bit/sec]**
0	LoRa: SF5 / 500 kHz	62500
1-7	RFU	
8	LoRa: SF12 / 500 kHz	980
9	LoRa: SF11 / 500 kHz	1760
10	LoRa: SF10 / 500 kHz	3900
11	LoRa: SF9 / 500 kHz	7000
12	LoRa: SF8 / 500 kHz	12500
13	LoRa: SF7 / 500 kHz	21900
14	LoRa: SF6 / 500 kHz	37500
15	Defined in [TS001][3]	

Fonte: Adaptado de LoRa Alliance, (LoRa Alliance Regional Parameters Workgroup, 2025).

O DR impacta também na configuração de recebimento de mensagens. Como explicado anteriormente, quando o dispositivo final estiver em classe A e enviar uma requisição uplink, o servidor de rede, ao receber a mensagem, devolverá uma requisição downlink. Essa última requisição será lida em uma janela de tempo específica, e sob um data rate

especificado em uma tabela. A tabela 9 especifica as possíveis configurações da janela de recebimento (chamada de RX1), que entram em conformidade com o AU915.

Tabela 9 – Downlink RX1 Data Rate Mapping

Uplink Data Rate	**Downlink Data Rate**					
`RX1DROffset`	**0**	**1**	**2**	**3**	**4**	**5**
DR0	DR8	DR8	DR8	DR8	DR8	DR8
DR1	DR9	DR8	DR8	DR8	DR8	DR8
DR2	DR10	DR9	DR8	DR8	DR8	DR8
DR3	DR11	DR10	DR9	DR8	DR8	DR8
DR4	DR12	DR11	DR10	DR9	DR8	DR8
DR5	DR13	DR12	DR11	DR10	DR9	DR8
DR6	DR13	DR13	DR12	DR11	DR10	DR9
DR7	DR9	DR8	DR8	DR8	DR8	DR8
DR9	DR14	DR13	DR12	DR11	DR10	DR9
DR10	DR0	DR14	DR13	DR12	DR11	DR10

Fonte: Adaptado de LoRa Alliance, (LoRa Alliance Regional Parameters Workgroup, 2025).

Outro fator importante a determinar dentro de um sistema LoRa é o Effective Isotropic Radiated Power (EIRP), que é o valor em dBm que representa a potência do sinal de rádio emitido pelo hardware. O valor máximo permitido pela regulação AU915 é de +30 dBm. O EIRP é determinado pela fórmula 2.1, em que o parâmetro *TXPower* é configurável via software no dispositivo final.

$$\text{EIRP} = \text{MAX EIRP} - 2 \times \text{TXPower} \tag{2.1}$$

2.2.5 Inteligência Artificial

2.2.5.1 Machine Learning

Segundo Vilela (2021), Machine Learning pode ser definida como um ramo da Inteligência Artificial (IA) responsável por capacitar computadores a simular aprendizado com base em processamento de dados. Essa técnica está em ascensão atualmente, sendo amplamente aplicada em softwares que têm por objetivo a identificação de padrões, como é o caso do OCR, detecção de phishing em e-mails, detecção de uso fraudulento de cartões de crédito, entre outras aplicações.

Tal subconjunto tem por característica a distinção de métodos de aprendizagem, tendo como os mais difundidos os métodos supervisionado, não supervisionado e semissupervisionado.

2.2.5.1.1 Aprendizado Supervisionado

Neste método de aprendizagem, o objetivo é ensinar ao modelo de IA, através de objetos previamente rotulados, a identificar características de modo a classificar novos objetos (VILELA, 2021). Entre os algoritmos mais importantes para a aplicação deste tipo de método de treinamento estão:

1. k-Nearest Neighbours;
2. Regressão Linear;
3. Máquina de Vetores de Suporte (SVM);
4. Árvore de Decisão e Floresta Aleatória;
5. Redes Neurais;

Observação: estes algoritmos não necessariamente são concorrentes; portanto, em um único modelo, pode haver implementação de um ou mais dos algoritmos citados (VILELA, 2021).

2.2.5.1.2 Deep Learning

Deep Learning, ou aprendizado profundo, é uma técnica que se encontra em um subcampo do Machine Learning, cujo desenvolvimento foi pautado na simulação do processo de aprendizagem humana. Nesse caso, utiliza-se como base o algoritmo de redes neurais, construindo estruturas conectadas de forma a emular o comportamento do cérebro humano (JESUS; COQUEIRO, 2023).

Para o processamento de dados sequenciais ou cronológicos, como linhas de texto em sistemas de reconhecimento de caracteres, as arquiteturas profundas evoluem para estruturas capazes de reter dependências temporais.

2.2.5.1.3 O que são LSTMs.

A Long-Short Term Memory (LSTM) é uma arquitetura de rede neural recorrente proposta por Hochreiter e Schmidhuber (1997) para superar a limitação das RNNs tradicionais em aprender dependências de longo prazo, problema causado pelo desaparecimento do gradiente durante a retropropagação (HOCHREITER; SCHMIDHUBER, 1997). A unidade básica da LSTM é a célula de memória, cujo estado interno é controlado por três portas multiplicativas: a **porta de esquecimento** (*forget gate*), que decide quais informações do estado anterior devem ser descartadas; a **porta de entrada** (*input gate*), que seleciona quais novas informações serão armazenadas no estado da célula; e a **porta de saída** (*output gate*), que define o que será exposto como saída da célula naquele passo

de tempo (HOCHREITER; SCHMIDHUBER, 1997; HANNUN, 2017). Esse mecanismo de portas permite que a LSTM mantenha informações relevantes por muitos passos de tempo e descarte informações irrelevantes, o que é essencial para o reconhecimento de sequências textuais, nas quais o contexto de caracteres anteriores e posteriores influencia a interpretação de cada símbolo (SHI; BAI; YAO, 2017).

2.2.5.1.4 Limitações em *Edge AI* e alternativas convolucionais.

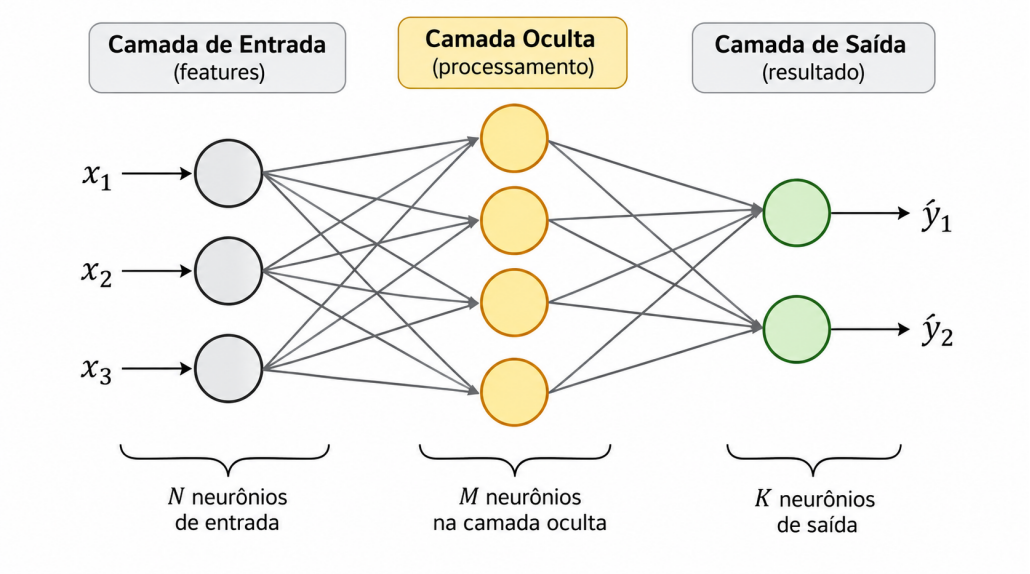
Apesar de sua eficácia na modelagem de contexto, as LSTMs impõem desafios significativos em dispositivos de borda com restrições severas de memória e poder computacional. A natureza sequencial da recorrência impede o paralelismo ao longo do eixo temporal, e o estado interno da célula precisa ser mantido entre passos de tempo, o que eleva o consumo de memória e a latência de inferência em microcontroladores (NGUYEN et al., 2026). Além disso, a implementação eficiente de operadores recorrentes em *runtimes* C++ para Edge Computing ainda é limitada, com suporte incompleto em algumas bibliotecas de inferência para microcontroladores (NGUYEN et al., 2026). Como alternativa, empregam-se **convoluções temporais 1D com núcleos largos** (ou convoluções dilatadas), que emulam o campo receptivo contextual da recorrência sem o custo computacional associado ao estado recorrente, permitindo processamento paralelo e maior eficiência em hardware embarcado (NAZRE et al., 2024; SHI; BAI; YAO, 2017).

2.2.5.2 Redes Neurais

Como apresentado por Jesus e Coqueiro (2023), redes neurais são estruturas baseadas em grafos, representadas em camadas; cada camada é composta por nós, denominados neurônios, e arestas, denominadas sinapses. A estrutura básica de uma rede neural possui três camadas: entrada, camada intermediária e saída, que são, respectivamente, responsáveis por executar as funções de receber os dados externos, processar os dados e entregar o resultado.

A técnica de redes neurais apresenta derivações amplamente utilizadas como alicerce no desenvolvimento de sistemas baseados em Deep Learning.

Figura 8 – Representação Visual em Grafo da estrutura básica de uma Rede Neural



2.2.5.2.1 Redes Neurais Convolucionais

Atualmente, as redes neurais convolucionais são amplamente aplicadas em processamento de dados não estruturados, como imagens, vídeos e áudios. Tal técnica tem como característica o processamento de pequenos trechos de dados por vez, ao contrário de outras técnicas que tentam realizar o processamento de forma atômica (JESUS; COQUEIRO, 2023).

Como apresentado por Atika et al. (2022), o desenvolvimento de uma Convolutional Neural Network (CNN) inicia-se pela definição da arquitetura em implementação. Tal técnica abrange uma vasta quantidade de arquiteturas que foram desenvolvidas para auxiliar e alcançar melhores resultados para determinados requisitos do software.

2.2.5.2.2 Camadas que Constituem uma CNN

Uma CNN é constituída por três principais camadas: camada de entrada de dados, camada de saída, também denominada camada densa, na qual cada um de seus neurônios está conectado à camada anterior, e diversas outras camadas intermediárias, denominadas por alguns autores camadas ocultas (AYENI, 2022).

As camadas intermediárias de uma CNN apresentam comportamento de aprofundamento em reconhecimento de abstrações a cada camada sobreposta, de tal forma que as camadas iniciais são responsáveis por identificar traços, contornos, linhas etc., enquanto as camadas finais possuem a responsabilidade de identificar objetos, rostos e valores, por exemplo. Tais camadas são desenvolvidas com a intercalação de camadas de convolução e pooling (agrupamento), de forma que cada entrada de uma camada é a saída da camada anterior.

2.2.5.2.3 Camada Convolucional

Como Ayeni (2022) explica, a camada convolucional é a camada responsável por processar imagens de forma a encontrar padrões ao aplicar um filtro (kernel/neurônio) a cada trecho da imagem de entrada, denominado campo receptivo. O neurônio é responsável por aplicar operações de multiplicação sobre o campo receptivo, gerando um único valor para cada trecho de imagem processado, e, posteriormente, os valores calculados pelos neurônios são armazenados em uma estrutura de três dimensões (AYENI, 2022).

A conversão da imagem para escala de cinza antes do processamento por uma CNN é justificada por reduzir a complexidade cromática do padrão Red, Green, Blue (RGB) (três canais) para um canal único, o que impacta diretamente a profundidade dos kernels e o volume de parâmetros da primeira camada da rede (AYENI, 2022).

2.2.5.2.4 Camada de Pooling

Segundo Ayeni (2022), a camada de pooling possui a responsabilidade de filtrar características predominantes encontradas na camada de convolução. Tal camada possui a importantíssima função de otimizar processamento, descartando dados menos significativos e mitigando o fenômeno de overfitting. As camadas de pooling possuem diversas derivações, que são escolhidas e implementadas a fim de otimizar resultados alcançados pelo modelo, mas a derivação mais aplicada sobre CNNs é a Max Pooling.

A operação de pooling reduz a resolução espacial dos mapas de características gerados pela camada convolucional, utilizando tipicamente uma janela de varredura (filtro) de 2×2.

2.2.5.2.5 Camada Totalmente Conectada

Como Ayeni (2022) explica, a camada denominada Fully-Connected (Totalmente Conectada) possui a função de interpretar abstrações mais complexas da imagem e classificá-las. Essa camada possui todos os seus parâmetros conectados com os parâmetros da camada anterior, corriqueiramente resultando em um número de nós de saída igual à quantidade de classes de classificação.

2.2.5.3 OCR - Fundamentos e Escopo

O OCR é um subcampo da Visão Computacional e da Inteligência Artificial dedicado à conversão de imagens contendo texto (impresso, manuscrito ou gravado) em dados legíveis por máquina (CHEN et al., 2021). Originalmente voltado à digitalização de documentos, o escopo foi ampliado pelo *Deep Learning* para aplicações industriais e de IoT (JAWAS; INDRIANTO, 2018).

2.2.5.3.1 Pipeline Tradicional

Um sistema clássico de OCR opera em etapas interdependentes (CHEN et al., 2021):

1. **Aquisição e pré-processamento**: melhoria de contraste, correção de inclinação (*deskewing*), remoção de ruído e binarização; em dispositivos embarcados, etapas de segmentação semântica podem isolar a Região de Interesse (ROI) antes do reconhecimento.
2. **Localização e segmentação de texto**: identificação das regiões de texto.
3. **Reconhecimento de caracteres**: classificação dos segmentos ou da sequência completa.
4. **Pós-processamento**: dicionários, correção ortográfica ou verificação sintática para mitigar erros residuais.

Em OCR, a leitura de uma palavra ou sequência numérica se beneficia do contexto em ambas as direções: o caractere atual é mais bem interpretado quando se conhece tanto o que veio antes quanto o que veio depois. Por isso, a CRNN emprega tipicamente camadas **bidirecionais** de LSTM (BiLSTM), que processam a sequência da esquerda para a direita e da direita para a esquerda e concatenam as representações resultantes (SHI; BAI; YAO, 2017; HANNUN, 2017).

2.2.5.3.2 Evolução Tecnológica: dos Métodos Heurísticos ao Deep Learning

A literatura descreve três gerações principais (CHEN et al., 2021):

Primeira geração (Template Matching). Sobreposição pixel a pixel com gabaritos estáticos; alta vulnerabilidade a variações de escala, rotação e fontes.

Segunda geração (Características manuais + classificadores clássicos). Uso de descritores como HOG e SIFT, combinados com SVM ou MLP; dependência de *feature engineering*.

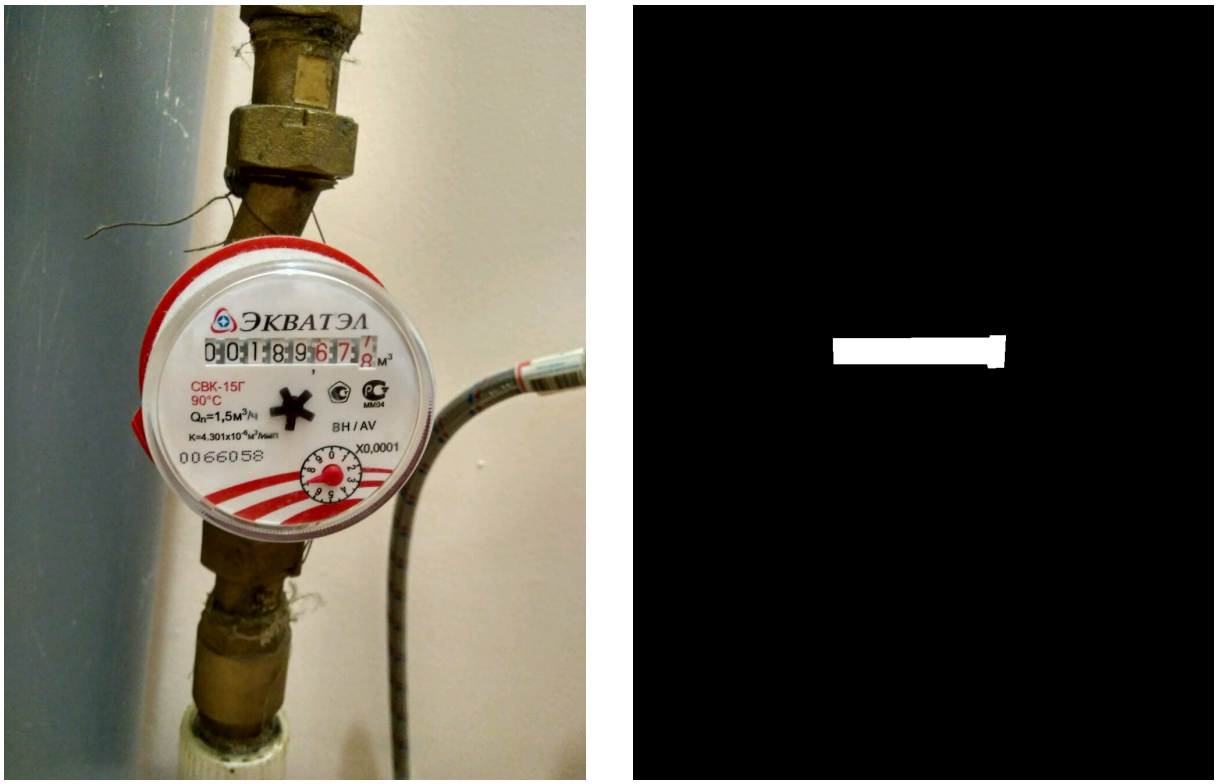
Terceira geração (End-to-End Deep Learning). Elimina a segmentação explícita de caracteres; modelos *end-to-end* aprendem simultaneamente representações visuais e dependência contextual sequencial (SHI; BAI; YAO, 2017).

2.2.5.4 Arquitetura U-Net - Segmentação de ROI

Dado o objetivo da implementação de CNN neste trabalho, destaca-se a arquitetura U-Net.

Como Atika et al. (2022) explicam, a arquitetura U-Net tem por objetivo a definição de bordas dentro de imagens, sendo amplamente utilizada na biomedicina com o foco de delimitar estruturas presentes em imagens; tal característica é explorada a fim de definir Region of Interest (ROI) dentro de uma imagem.

Figura 9 – Definição de ROI de uma Imagem



(a) Imagem Original (b) ROI Extraída da Imagem Original

Fonte: Autores (2026).

A arquitetura U-Net é caracterizada pela presença de três etapas de processamento: Contração, Gargalo e Expansão. A camada de contração é composta por vários blocos, nos quais cada bloco recebe uma entrada e aplica uma camada de convolução seguida por uma camada de operação de max-pooling; essas camadas são matrizes que, respectivamente, apresentam a estrutura 3x3 e 2x2.

Com adendo, as camadas que constroem a arquitetura U-Net utilizam a função de ativação ReLU apresentada na equação 2.2.

$$f(x) = \text{ReLU}(x) = \max(0, x) = \begin{cases} x, & \text{se } x > 0 \\ 0, & \text{se } x \leq 0 \end{cases} \tag{2.2}$$

2.2.5.5 Arquitetura CRNN e CTC

A arquitetura Convolutional Recurrent Neural Network (CRNN) é amplamente utilizada em aplicações que exigem inferência de classes em sequência, pois combina um extrator convolucional (CNN) a fim de obter representações visuais, uma camada recorrente (Recurrent Neural Network (RNN)), com o objetivo de modelar dependências contextuais ao longo do eixo horizontal, e a função de perda Connectionist Temporal Classification (CTC), com a finalidade de alinhar as previsões da rede à sequência de rótulos sem a necessidade de segmentação explícita dos caracteres (SHI; BAI; YAO, 2017).

A técnica CTC resolve o problema central de sequências de comprimento variável: em geral, não se sabe onde cada caractere começa e termina na imagem, e o número de *frames* produzidos pela CNN é maior do que o número de caracteres da palavra ou número a ser reconhecido (GRAVES et al., 2006). Para lidar com isso, a técnica CTC introduz dois mecanismos fundamentais: um *token* oculto de espaçamento (*blank*) e uma função de colapso que mapeia a sequência de previsões da rede para o rótulo final (GRAVES et al., 2006; HANNUN, 2017).

O *token blank* e o colapso de repetições. A cada passo de tempo, a rede produz uma distribuição de probabilidade sobre o alfabeto acrescido do símbolo especial *blank* (ϵ). O *blank* representa a ausência de emissão de caractere naquele instante, funcionando como um separador entre caracteres idênticos adjacentes (GRAVES et al., 2006). Após a rede produzir a sequência completa de previsões, aplica-se uma função de colapso $\mathcal{B}$ que (i) remove caracteres repetidos consecutivos e (ii) remove os *blanks* (HANNUN, 2017). Por exemplo, a sequência de caminho $--h--e--l--l--o--$ colapsa para "hello"; sem o *blank*, as repetições de "l" seriam indistinguíveis de um único "l" (HANNUN, 2017).

Marginalização de alinhamentos. O treinamento da CTC não exige um alinhamento predefinido entre os *frames* e os caracteres. Em vez disso, a função de perda calcula a probabilidade da sequência de rótulos correta como a soma das probabilidades de todos os caminhos de alinhamento válidos que, após o colapso, produzem aquele rótulo (GRAVES et al., 2006). Essa marginalização é feita de forma eficiente pelo algoritmo *forward-backward*, que explora a estrutura do problema para evitar a enumeração explícita de todos os caminhos, que seria computacionalmente inviável (GRAVES et al., 2006; HANNUN, 2017). Assim, a rede pode ser treinada diretamente com pares (imagem, transcrição), sem anotações de posição caractere a caractere.

Inferência. No momento da inferência, a rede produz, para cada passo de tempo, a classe mais provável (ou uma distribuição sobre as classes). A sequência final é obtida pelo colapso das previsões, opcionalmente combinado com busca em feixe (*beam search*) para explorar múltiplos alinhamentos e melhorar a acurácia (HANNUN, 2017). O artigo

de Hannun (2017) apresenta uma explicação visual detalhada desse processo, incluindo exemplos passo a passo de como o *blank* e o colapso atuam em sequências reais (HANNUN, 2017).

Por que a CTC é adequada ao OCR. Em OCR de palavras, números de medidores ou placas, o número de caracteres varia e não há garantia de que cada caractere ocupe exatamente um número fixo de *frames* na saída da CNN. A CTC elimina a necessidade de segmentação manual e permite que a CRNN seja treinada de ponta a ponta com sequências de comprimentos diferentes, mantendo a coerência entre as previsões vizinhas e evitando o colapso de repetições idênticas (SHI; BAI; YAO, 2017; GRAVES et al., 2006).

2.2.5.6 Quantização e QAT

Quantização é um método de simplificação dos parâmetros de um modelo de Redes Neurais Artificiais (RNA). Como Ferreira (2022) explica, a aplicação de tal técnica traz como benefício a redução do tamanho de modelos, viabilizando a utilização em equipamentos de baixo poder computacional, além do aumento de velocidade de inferência e simplificação de cálculos realizados pelo processador, frente a um modelo que, na maioria dos casos, possui seus parâmetros representados em pontos flutuantes de 32 bits.

A técnica Quantization-Aware Training (QAT), com foco em transformação de parâmetros para inteiros de 8 bits, simula a perda de precisão da conversão de FP32 para INT8 durante o treinamento, permitindo que os pesos da rede se adaptem e mitiguem a degradação de acurácia. Em sistemas de OCR para medidores, modelos quantizados alcançam tamanhos da ordem de centenas de kB e inferência em subsegundo em hardware embarcado (ADEMOLA; PETLENKOV; LEIER, 2023; JAWAS; INDRIANTO, 2018).

2.3 Trabalhos correlatos

Nesta seção, analisam-se pesquisas que utilizam IoT e Visão Computacional para a digitalização de medidores de consumo, como as soluções de computação de borda propostas por Vilela (2021) e Bravo, Javier e Tolentino (2025), além de arquiteturas de sensores inteligentes exploradas por Tedjojuwono e Jahja (2024). O objetivo é contextualizar as escolhas arquiteturais deste TCC, comparando modelos de processamento (borda vs nuvem) e protocolos de comunicação sem fio. A revisão fundamenta o estado da técnica e destaca os diferenciais desta proposta em termos de infraestrutura de dados e eficiência energética.

2.3.1 Trabalho 1: Solução IoT para Digitalização de Medidores de Consumo por Visão Computacional e MicroML

Vilela (2021) abordou o tema de soluções IoT para Medidores de Consumo. Para resolvê-lo, foi desenvolvido um sistema utilizando microprocessadores ESP32, baseado no método de Edge Computing. O autor desenvolveu um software próprio, otimizado para microprocessadores, focado em OCR, extraindo os dados, transmitindo-os via rede LoRaWan e armazenando-os localmente; posteriormente, os dados são apresentados em um simples dashboard apresentando o consumo extraído pelo sistema até o momento.

A metodologia utilizada foi a pesquisa experimental, na qual houve o desenvolvimento de um sistema que supre a necessidade de extração de dados de maneira digital.

Os resultados mostraram que o desenvolvimento deste tipo de ferramenta é totalmente viável e, em suma, o experimento foi bem-sucedido. No entanto, a solução apresenta limitações, principalmente no que tange ao desenvolvimento do dataset, no qual não foi implementado tratamento adequado ao enviesamento dos dados. Também existe uma carência de métricas para medição de consumo energético e de processamento pelo sistema.

O presente trabalho diferencia-se por aplicar uma comparação entre uma arquitetura clássica de Edge vs Cloud, e um ponto que atacamos mais profundamente é a apresentação de métricas importantíssimas no desenvolvimento de um sistema IoT além da acurácia de modelo, por exemplo, as métricas de consumo energético e impactos em manutenção por decisões tomadas.

2.3.2 Trabalho 2: Sistema de Gerenciamento Inteligente de Água Utilizando Sensores Baseados em IoT

Tedjojuwono e Jahja (2024) abordaram o tema de sistemas inteligentes para o manejo de recursos hídricos em áreas residenciais. Para resolvê-lo, os autores propuseram o desenvolvimento de um framework que substitui medidores mecânicos por sensores de fluxo eletrônicos integrados a uma arquitetura IoT. O sistema utiliza prototipagem com Arduino Uno e sensores de efeito Hall (YF-S201) para coletar dados de vazão em tempo real, enviando as informações via módulo 3G/GPRS para um servidor de banco de dados centralizado. Os dados processados alimentam uma aplicação web que permite o monitoramento do consumo, faturamento automatizado e a detecção precoce de anomalias na distribuição.

A metodologia utilizada seguiu os princípios do Design Thinking, incluindo uma fase qualitativa de coleta de requisitos através de entrevistas com usuários e a criação de mapas de empatia, seguida pelo desenvolvimento do protótipo e avaliação de usabilidade através da escala SUS (System Usability Scale).

Os resultados mostraram que a solução é viável e intuitiva, obtendo uma pontuação média de 72 na escala SUS, o que indica uma boa aceitação pelos usuários. No entanto, o

trabalho aponta limitações na abrangência dos dados, focando primariamente na taxa de fluxo, e destaca a necessidade de integrar mais tipos de sensores (como pressão e qualidade da água) para análises de Big Data mais robustas. Além disso, a precisão total do sistema de previsão depende de um volume maior de dados históricos que ainda precisam ser coletados.

O presente trabalho diferencia-se por focar na infraestrutura de comunicação LoRaWan, visando uma maior eficiência energética e alcance em comparação ao GPRS/3G utilizado pelos autores, além de explorar a integração direta com medidores analógicos já existentes através de telemetria, em vez da substituição completa do hardware de medição.

2.3.3 Trabalho 3: Rede de Sensores Sem Fio Habilitada para LoRa para Leitura e Faturamento de Consumo de Água Usando Reconhecimento Óptico de Caracteres

Bravo, Javier e Tolentino (2025) desenvolveram um sistema automatizado para leitura de hidrômetros analógicos utilizando uma combinação de tecnologias de baixo custo e comunicação de longo alcance. O objetivo central foi mitigar erros humanos e atrasos operacionais comuns em leituras manuais.

A metodologia consistiu na integração de um módulo ESP32-CAM para captura de imagens dos medidores e um algoritmo de OCR para a extração digital dos dados de consumo no próprio dispositivo. Para a transmissão, utilizou-se a tecnologia LoRa (módulo Ra-02), operando em uma topologia de rede que inclui repetidores para estender o alcance e um gateway central para envio dos dados à nuvem. O sistema também implementou estratégias de gerenciamento de energia através de ciclos de *deep sleep* programados para aumentar a vida útil da bateria.

Os resultados apontaram uma precisão de 99,71% no reconhecimento dos dígitos e um alcance de transmissão estável de até 1070 metros em condições de campo aberto. Os autores observaram que medidores mais antigos ou obstruídos podem elevar o tempo de processamento do OCR.

O presente trabalho diferencia-se diante dos seguintes tópicos:

- Metodologia de Pesquisa: Bravo, Javier e Tolentino (2025), aplicaram em seu trabalho a metodologia Design Science Research (DSR), desenvolvendo uma solução à problemática da coleta de dados em hidrômetros, em contrapartida, este trabalho possui como objetivo a comparação entre métodos de processamento de dados em sistemas IOT.

- Público-Alvo: O trabalho correlato, busca impactar principalmente empresas consorciadas à distribuição de recursos hídricos. O presente trabalho busca impactar

desenvolvedores de sistemas IOT, ao prover dados comparativos que podem pautar decisões arquitetônicas de novos sistemas.

- Especificação de Restrições Regulatórias: Este trabalho utiliza como base a regulação AU915, adotada pela ANATEL em 2017. O trabalho de Bravo, Javier e Tolentino (2025), não define qual restrição regulatória o sistema está seguindo, o que impacta diretamente sobre a viabilidade de operação do sistema desenvolvido.

Utilizaremos esse trabalho como referência no desenvolvimento do fluxo de borda para realizar uma comparação com a abordagem de processamento em nuvem, contribuindo para a discussão sobre o trade-off entre eficiência energética e precisão em sistemas de telemetria de baixo custo.

3 Desenvolvimento

3.1 Metodologia da pesquisa

Para avaliar a eficácia do sistema de telemetria proposto e garantir a objetividade dos resultados, foram adotados indicadores quantitativos divididos em três dimensões críticas: desempenho de rede, precisão de visão computacional, eficiência energética e acurácia preditiva.

3.1.1 Desempenho da Rede de Comunicação

A robustez da transmissão via LoRaWan será quantificada através da Packet Delivery Ratio (PDR). Conforme a metodologia de Bravo, Javier e Tolentino (2025), o PDR é definido pela razão entre o número de pacotes recebidos pelo *gateway* e o número de pacotes enviados pelo nó sensor:

$$PDR = \left(\frac{\text{Pacotes Recebidos}}{\text{Pacotes Enviados}} \right) \times 100 \tag{3.1}$$

Além do PDR, será medida a latência de ponta a ponta, compreendendo o tempo decorrido desde o disparo da captura da imagem no ESP32 até a disponibilidade do dado processado no *dashboard*.

3.1.2 Acurácia da Visão Computacional (OCR)

A validação do algoritmo de OCR, responsável pela extração dos dígitos dos medidores, será realizada sob diferentes condições de iluminação e ângulos de captura, visando simular cenários reais de instalação.

Conforme discutido por Barkat (2024), a eficácia de modelos de *Deep Learning* aplicados ao monitoramento deve ser medida pela sua capacidade de generalização. Para este trabalho, a métrica principal será a Acurácia de Leitura (Acc_{ocr}), definida pela razão entre as predições que coincidem exatamente com o valor real (verdade de campo) e o total de amostras testadas:

$$Acc_{ocr} = \frac{VP + VN}{VP + VN + FP + FN} \tag{3.2}$$

Em que VP, VN, FP e FN representam, respectivamente, os Verdadeiros Positivos, Verdadeiros Negativos, Falsos Positivos e Falsos Negativos obtidos através da Matriz de Confusão. Esta abordagem quantitativa permite identificar se erros de leitura estão associados a fatores externos, como reflexos na cúpula de vidro do medidor ou distorções de perspectiva.

3.1.3 Autonomia Energética

A eficiência do hardware será validada através da medição do consumo de corrente em diferentes estados de operação (*Deep Sleep*, captura de imagem e transmissão). A autonomia estimada (A) em dias será calculada pela relação entre a capacidade da bateria (C_{bat}) e o consumo médio diário ($I_{médio}$):

$$A = \frac{C_{bat}}{I_{médio} \times 24} \tag{3.3}$$

3.2 Detalhes da implementação

3.2.1 Visão geral da solução

O sistema desenvolvido realiza a leitura automática de hidrômetros analógicos do tipo ciclométrico, cujo mostrador é formado por roletes numerados, por meio de um nó sensor baseado no ESP32-CAM. O nó fotografa o mostrador do medidor e envia o resultado, via LoRaWan, até uma estação receptora conectada a um computador, que faz o papel de servidor da aplicação.

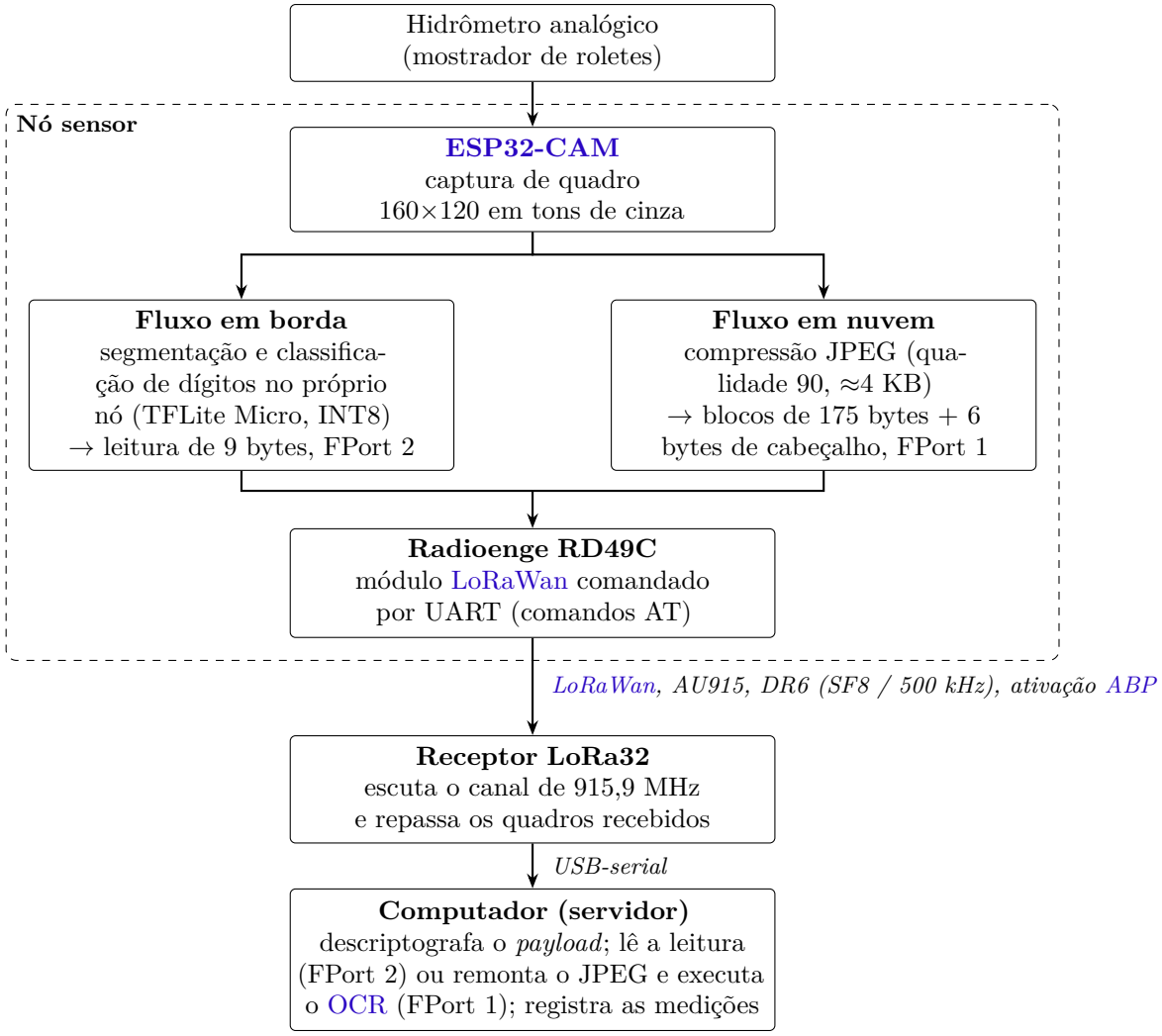
Para responder à pergunta de pesquisa, o mesmo nó foi implementado em dois fluxos de processamento, que diferem no local em que o OCR é executado e, por consequência, no conteúdo que trafega pelo enlace:

- **Fluxo em nuvem** (Cloud Computing): o nó captura a imagem, comprime-a no formato JPEG e a transmite fragmentada em blocos. O servidor remonta a imagem e executa o reconhecimento dos dígitos.
- **Fluxo em borda** (Edge Computing): o nó captura a imagem, executa localmente os modelos de reconhecimento e transmite apenas a leitura resultante, em um único pacote de 9 bytes.

Uma decisão central da implementação é que os dois fluxos utilizam os mesmos modelos de redes neurais, quantizados em inteiros de 8 bits (INT8), e a mesma lógica de leitura. Assim, as diferenças de latência, entrega de pacotes e consumo energético observadas entre eles decorrem apenas do local de processamento e do volume de dados transmitido, e não de modelos com capacidades distintas. O efeito da quantização sobre a acurácia, objeto da hipótese H1, é avaliado à parte, pela comparação entre as versões em ponto flutuante de 32 bits e em INT8 do mesmo modelo.

A Figura 10 apresenta a arquitetura do sistema e o caminho percorrido pelos dados em cada fluxo.

Figura 10 – Arquitetura do sistema e os dois fluxos de processamento



Fonte: Elaborado pelos autores (2026).

3.2.2 Componentes de hardware

O protótipo foi montado com componentes de baixo custo. A Tabela 10 resume a função de cada um.

Tabela 10 – Componentes de hardware do protótipo

Componente	**Função no sistema**
ESP32-CAM (AI-Thinker)	Nó sensor. Microcontrolador ESP32 a 240 MHz com 4 MB de PSRAM e câmera integrada. Captura a imagem e, no fluxo em borda, executa os modelos de OCR.
Radioenge RD49C	Módulo LoRaWan homologado pela ANATEL, ligado ao ESP32-CAM por UART (GPIO 15 e GPIO 14) e controlado por comandos AT. Opera no plano AU915 com potência de até +20 dBm.
Heltec/TTGO LoRa32 V2	Receptor. Rádio LoRa configurado no mesmo canal e na mesma taxa do transmissor; repassa cada quadro recebido ao computador pela porta serial, junto com o RSSI.
Computador	Servidor da aplicação. Descriptografa os quadros, remonta as imagens, executa o OCR do fluxo em nuvem e registra as medições. Também foi usado no treinamento dos modelos.
INA219 e Arduino Uno	Instrumentação. Sensor de corrente com resistor *shunt* de 0,1 Ω, lido por um Arduino Uno a 100 amostras por segundo, usado para medir o consumo do nó sensor.

Fonte: Elaborado pelos autores (2026).

3.2.3 Componentes de software

O código do projeto está reunido em um único repositório[1] e se divide em firmwares para os microcontroladores, escritos em C++ com o *core* Arduino-ESP32 3.0.7, e *scripts* em Python executados no computador. A Tabela 11 relaciona os principais componentes.

[1] Disponível em <https://github.com/Marcelo-122/Codigo-do-TCC2>.

Tabela 11 – Principais componentes de software

Componente	**Responsabilidade**
`transmissor_esp32cam`	Firmware do nó no fluxo em nuvem: captura, compressão JPEG, fragmentação e envio dos blocos pela porta 1 (FPort 1).
`transmissor_ocr_esp32cam`	Firmware do nó no fluxo em borda: captura, inferência dos modelos com TensorFlow Lite Micro e envio da leitura pela porta 2 (FPort 2).
`receptor_lora32`	Firmware do receptor: escuta o canal e escreve na serial cada quadro recebido.
`reconstrutor.py`, `enlace.py`	Lado do servidor: leitura da serial, descriptografia, remontagem da imagem ou decodificação da leitura e registro das medições.
`ocr/pipeline_nuvem.py`	OCR do fluxo em nuvem, com os mesmos modelos e a mesma lógica do firmware de borda.
`ModeloEDataset/`	Preparação dos conjuntos de dados, treinamento, quantização e exportação dos modelos para o firmware.
`medicoes/`	*Scripts* de medição de corrente e de latência e os dados brutos coletados.

Fonte: Elaborado pelos autores (2026).

Nos dois fluxos, os firmwares do nó compartilham o mesmo rádio, a mesma configuração de enlace, o mesmo ciclo de *deep sleep* e o mesmo formato de registro de tempos, de modo que as medições de consumo e de latência sejam obtidas pelos mesmos *scripts* e possam ser comparadas diretamente.

3.2.4 Simplificações em relação a uma implantação real

Algumas escolhas do protótipo simplificam a infraestrutura descrita na Seção 2.2 e devem ser consideradas na interpretação dos resultados:

- **Receptor de canal único no lugar de um *gateway***: o receptor LoRa32 escuta um único canal, fixo, e não implementa as funções de um *gateway* LoRaWan multicanal. Por isso, o transmissor também foi configurado para usar um único canal (Seção 3.2.5).
- **Ausência de servidor de rede**: não há servidor de rede nem servidor de aplicação dedicados. O dispositivo usa ativação ABP, com chaves de sessão configuradas previamente, e o próprio computador descriptografa o *payload* com a chave de aplicação (AppSKey). O código de integridade da mensagem (MIC) não é verificado, pois exigiria a chave de rede. Um protótipo inicial, que injetava os pacotes em um servidor ChirpStack sem uso de rádio, foi descartado.

- **Ambiente de bancada**: as medições de enlace foram feitas com transmissor e receptor a poucos metros de distância, com sinal forte, de modo que avaliam o comportamento dos fluxos, e não o alcance do enlace.

3.2.5 Enlace LoRaWAN

Esta seção descreve a configuração do enlace de rádio, o formato das mensagens trocadas nos dois fluxos, o preparo e a fragmentação da imagem e o tratamento dos quadros recebidos pelo servidor. Os mesmos parâmetros de rádio foram usados nos dois fluxos.

3.2.5.1 Estudo preliminar e escolha da configuração de rádio

Na fase inicial do trabalho, a viabilidade do envio de imagens foi estimada analiticamente para o plano AU915 com canais de 125 kHz e SF 8. Nessa configuração, o Ato nº 14.448 da ANATEL enquadra o LoRa como sistema de salto em frequência, que deve usar no mínimo 35 canais e limitar o tempo médio de ocupação de cada canal a 0,4 s em um intervalo de 14 s (ANATEL, 2017). Para respeitar esse limite, cada pacote foi restrito a 115 bytes na camada física; com isso, uma imagem JPEG de 3.182 bytes exigiria 38 pacotes de 348,7 ms cada, totalizando cerca de 13,3 s de rádio ativo.

Essa configuração foi abandonada na implementação por dois motivos. Primeiro, o salto entre pelo menos 35 canais é incompatível com o receptor disponível, que escuta um único canal (Seção 3.2.4). Segundo, o limite de 400 ms por transmissão restringe o tamanho do bloco de dados: um pacote com 181 bytes de *payload*, tamanho adotado adiante, ocuparia 543 ms de rádio com 125 kHz.

Por isso, o enlace foi configurado para operar com largura de banda de 500 kHz, no *data rate* DR6 do plano AU915, que corresponde a SF8 com 500 kHz (Tabela 7) e a um *payload* máximo de 222 bytes (Radioenge, 2022). Com essa largura de banda, o rádio se enquadra no item 10.3 do mesmo Ato, destinado a sistemas de modulação digital (Seção 2.2.4), cujos três requisitos são verificados a seguir: largura de faixa a 6 dB de, no mínimo, 500 kHz; potência de pico máxima de 1 W; e densidade espectral de potência máxima de 8 dBm em qualquer faixa de 3 kHz.

O módulo Radioenge RD49C transmite com potência máxima de +20 dBm (100 mW), sem comando AT que permita ajustá-la (Radioenge, 2022), abaixo do limite de 1 W. Supondo que essa potência se distribua de forma plana pelos 500 kHz, a densidade espectral fica em torno de $20 - 10\log_{10}(500/3) \approx -2{,}2$ dBm em 3 kHz, abaixo do limite de 8 dBm. Por ser enquadrado no item 10.3, o sistema dispensa o salto de frequência e o limite de ocupação por canal exigidos para os canais de 125 kHz, o que permite usar um canal fixo. O módulo é homologado pela ANATEL (certificado nº 02021-18-07215), mas duas ressalvas se aplicam: a largura de faixa a 6 dB do sinal LoRa com 500 kHz nominais

não consta do manual e deve ser confirmada nos ensaios de homologação, e antenas com ganho superior a 6 dBi exigiriam a redução da potência (item 10.5).

O canal escolhido foi o de 915,9 MHz, o primeiro canal de 500 kHz do plano AU915: canal 64 na numeração da especificação (LoRa Alliance Regional Parameters Workgroup, 2025), que começa em zero, e canal 65 na numeração do manual do módulo, que começa em um. No módulo, ele foi selecionado pela máscara de canais (`AT+CHMASK`), e o *data rate* foi fixado em DR6 com o ajuste adaptativo de taxa (ADR) desativado, pois o comando `AT+DR` só tem efeito com o ADR desligado (Radioenge, 2022). A Tabela 12 reúne os parâmetros usados por todos os componentes do enlace.

Tabela 12 – Parâmetros do enlace LoRaWAN

Parâmetro	**Valor**
Plano de canais	AU915
Canal	Fixo, 915,9 MHz (canal 64), sem salto de frequência
Data rate	DR6: SF 8, BW de 500 kHz, ADR desativado
Camada física	CR 4/5, preâmbulo de 8 símbolos, CRC ativado, cabeçalho explícito, palavra de sincronismo `0x34`
Potência de transmissão	+20 dBm (fixa no módulo)
Ativação	ABP
Payload máximo em DR6	222 bytes
Portas de aplicação	FPort 1 (blocos de imagem) e FPort 2 (leitura do OCR)
Bloco de imagem	175 bytes de dados + 6 bytes de cabeçalho = 181 bytes
Intervalo entre pacotes	7 s

Fonte: Elaborado pelos autores (2026).

3.2.5.2 Formato das mensagens

Como apresentado na imagem 6, cada mensagem LoRaWan acrescenta 13 bytes ao conteúdo da aplicação, devido aos campos: o cabeçalho MAC, o endereço do dispositivo, o campo de controle FCtrl, o contador FCnt, a porta de aplicação FPort e o código de integridade MIC. O conteúdo da aplicação (FRMPayload) é criptografado com a chave de sessão de aplicação (AppSKey). Os dois fluxos usam portas diferentes, o que permite ao servidor distinguir as mensagens sem nenhum outro campo. A Tabela 13 apresenta o formato do conteúdo de cada porta.

Tabela 13 – Formato do *payload* de aplicação em cada fluxo

Campo	**Tamanho**	**Conteúdo**
FPort 1 — bloco de imagem (fluxo em nuvem)		
Identificador da leitura	2 bytes	Número aleatório sorteado a cada captura
Índice do bloco	2 bytes	Posição do bloco na imagem, a partir de 0
Total de blocos	2 bytes	Número de blocos da imagem
Dados	até 175 bytes	Trecho do arquivo JPEG
FPort 2 — leitura (fluxo em borda)		
Identificador da leitura	2 bytes	Número aleatório sorteado a cada leitura
Indicadores	1 byte	Bit 7: janela dos dígitos encontrada; bits 0 a 6: número N de dígitos
Dígitos	N bytes	Um caractere ASCII por rolete: `0` a `9`; `?` para dígito não classificado; `-` quando a janela não foi encontrada

Fonte: Elaborado pelos autores (2026).

Os campos numéricos são gravados em *big-endian*. O identificador da leitura impede que blocos de capturas diferentes sejam misturados na remontagem, e o par índice–total permite ao servidor saber quais blocos faltam. No fluxo em borda, com o medidor de seis roletes usado nos testes, a mensagem tem $3 + 6 = 9$ bytes e é enviada em um único *uplink*.

3.2.5.3 Preparo da imagem e fragmentação

No fluxo em nuvem, a câmera é configurada na resolução QQVGA (160×120 pixels) em tons de cinza, de modo que cada quadro bruto ocupa 19.200 bytes (um byte por pixel). Esse quadro é comprimido em JPEG pelo próprio ESP32-CAM, por software, com qualidade 90 em uma escala de 1 a 100.

A escolha do formato foi apoiada por uma prova de conceito em C++ com a biblioteca OpenCV, que mediu o tamanho de três imagens de hidrômetro em 160×120 pixels após a compressão JPEG em diferentes qualidades, em três versões: colorida, em tons de cinza e binarizada pelo método de Otsu. A Tabela 14 mostra as médias obtidas para algumas qualidades.

Tabela 14 – Tamanho médio do JPEG, em bytes, de três imagens de 160×120 pixels

Qualidade JPEG	**Colorida**	**Tons de cinza**	**Binarizada**
30	2.028	1.626	2.773
50	2.338	1.949	3.433
80	2.910	2.518	4.725
90	3.466	3.073	5.895

Fonte: Elaborado pelos autores (2026).

A conversão para tons de cinza reduziu o arquivo em relação à imagem colorida, enquanto a binarização o aumentou em quase o dobro, pois as bordas abruptas entre preto e branco são pouco compressíveis pelo JPEG. Por isso, a imagem é transmitida em tons de cinza, sem binarização. A qualidade 90 foi mantida para que a compressão não prejudicasse o reconhecimento dos dígitos no servidor. Nas 60 leituras das duas corridas de medição do fluxo em nuvem, o JPEG variou de 3.323 a 4.568 bytes (mediana de 4.323 bytes), pois o tamanho depende da cena e da iluminação. Para o dimensionamento, adotou-se como referência uma imagem de 4.300 bytes, valor definido a partir das primeiras capturas, antes das corridas de medição, e próximo da mediana observada depois.

O arquivo JPEG é dividido em blocos de 175 bytes, que recebem o cabeçalho de 6 bytes da Tabela 13, totalizando 181 bytes de *payload*, abaixo do máximo de 222 bytes permitido em DR6. Uma versão anterior usava blocos de 100 bytes; o aumento para 175 bytes reduziu o número de pacotes por imagem a pouco mais da metade, mantendo uma folga de 41 bytes em relação ao máximo.

Os blocos são enviados ao módulo de rádio, um a um, pelo comando `AT+SENDB` na porta 1, com intervalo de 7 s entre eles. O item 10.3 não impõe intervalo mínimo entre transmissões em um canal fixo de 500 kHz; o valor de 7 s é conservador e foi mantido para não sobrecarregar o módulo. Se o módulo não confirmar a transmissão em até 2 s, o bloco é reenviado, até cinco tentativas.

3.2.5.4 Tempo no ar

O tempo no ar de cada pacote foi calculado pela fórmula da Semtech para o modem LoRa (Semtech Corporation, 2013). A duração de um símbolo é dada pela Equação 3.4:

$$T_{\text{sym}} = \frac{2^{SF}}{BW} = \frac{2^8}{500\,000\ \text{Hz}} = 0{,}512\ \text{ms} \tag{3.4}$$

O número de símbolos do *payload* é dado pela Equação 3.5:

$$n_{\text{payload}} = 8 + \max\left(\left\lceil \frac{8 \cdot PL - 4 \cdot SF + 28 + 16 \cdot CRC - 20 \cdot H}{4 \cdot (SF - 2 \cdot DE)} \right\rceil \cdot (CR + 4),\ 0\right) \tag{3.5}$$

em que PL é o tamanho do pacote na camada física, em bytes; $CRC = 1$ indica CRC ativado; $H = 0$ indica cabeçalho explícito; $DE = 0$ indica a otimização para baixas taxas desativada; e $CR = 1$ corresponde ao código 4/5. O tempo no ar do pacote é dado pela Equação 3.6, com preâmbulo de $n_{\text{pre}} = 8$ símbolos:

$$T_{\text{pacote}} = (n_{\text{pre}} + 4{,}25 + n_{\text{payload}}) \cdot T_{\text{sym}} \tag{3.6}$$

Para um bloco de imagem completo, $PL = 13 + 181 = 194$ bytes, o que resulta em $n_{\text{payload}} = 8 + \lceil 1564/32 \rceil \cdot 5 = 253$ símbolos e $T_{\text{pacote}} = (8 + 4{,}25 + 253) \cdot 0{,}512 \approx 135{,}8$ ms. Para a mensagem de leitura do fluxo em borda, $PL = 13 + 9 = 22$ bytes, o que resulta

em $T_{\text{pacote}} \approx 25{,}7$ ms. A Tabela 15 apresenta o número de pacotes e os tempos para os tamanhos mínimo, mediano e máximo de JPEG medidos e para a mensagem do fluxo em borda.

Tabela 15 – Pacotes, tempo de rádio e tempo total de envio por leitura

Conteúdo	**Bytes**	**Pacotes**	**Rádio (s)**	**Envio total (s)**
JPEG, menor medido	3.323	19	2,58	126
JPEG, mediana medida	4.323	25	3,36	168
JPEG, maior medido	4.568	27	3,57	182
Leitura do fluxo em borda	9	1	0,03	0,03

Fonte: Elaborado pelos autores (2026). O envio total soma o tempo no ar ao intervalo de 7 s entre pacotes.

Os valores mostram que, no fluxo em nuvem, o tempo de rádio é pequeno diante do tempo total de envio, que é dominado pelos intervalos de 7 s entre os pacotes. O volume transmitido por leitura também difere em mais de duas ordens de grandeza entre os fluxos: cerca de 4 KB de imagem contra 9 bytes de leitura. Essa diferença é a base das hipóteses H2 e H3.

3.2.5.5 Recepção, descriptografia e remontagem

O receptor LoRa32 foi configurado com os mesmos parâmetros de camada física do transmissor (Tabela 12). Para cada quadro recebido com CRC válido, ele escreve na porta serial uma linha com o conteúdo do quadro em hexadecimal e o RSSI medido. Como não há servidor de rede, o processamento LoRaWan é feito no computador pelo *script* `reconstrutor.py`, em quatro etapas:

1. **Separação dos campos**: o quadro é dividido em endereço do dispositivo, contador FCnt, porta e conteúdo; quadros de outros endereços são descartados.

2. **Descriptografia**: o conteúdo é descriptografado com a AppSKey, usando o AES-128 em modo contador definido pela especificação LoRaWan, que combina o endereço do dispositivo e o FCnt na geração do fluxo de chaves. O MIC não é verificado, pois o computador não dispõe da chave de rede.

3. **Tratamento por porta**: na porta 2, a leitura é decodificada e validada (tamanho coerente com o número de dígitos e apenas caracteres permitidos). Na porta 1, os blocos são agrupados pelo identificador da leitura, e blocos repetidos são ignorados.

4. **Remontagem e registro**: quando todos os blocos de uma leitura chegam, o arquivo JPEG é gravado e pode seguir para o OCR. Se algum bloco faltar, a imagem é gravada como parcial, com zeros no lugar dos dados ausentes. Cada leitura é registrada em um arquivo CSV com os blocos recebidos, o RSSI e os tempos usados nas medições.

No modo autônomo, usado nas medições, nenhum dos dois firmwares retransmite mensagens perdidas: o nó envia a leitura ou a imagem uma única vez e volta a dormir. O firmware do fluxo em nuvem aceita um comando de reenvio de blocos específicos, mas ele só foi usado em testes manuais. Assim, no fluxo em nuvem, a perda de um único bloco inutiliza a imagem inteira, e, no fluxo em borda, a perda do único pacote inutiliza a leitura. Esse comportamento é considerado na análise da taxa de entrega.

3.2.6 Reconhecimento dos dígitos

O OCR implementado divide a leitura em duas etapas: primeiro, localizar no quadro a janela onde ficam os roletes do hidrômetro; depois, classificar cada dígito dessa janela separadamente. Os mesmos modelos e a mesma lógica de leitura são usados no ESP32-CAM, no fluxo em borda, e no servidor, no fluxo em nuvem. Esta seção descreve como essa solução foi escolhida, os dados usados, os dois modelos, a quantização e a forma de leitura adotada.

3.2.6.1 Evolução das abordagens

3.2.6.1.1 Teste de viabilidade com OCR genérico.

O primeiro experimento verificou se um OCR de propósito geral conseguiria ler o mostrador em uma imagem tão pequena quanto a capturada pelo ESP32-CAM (160×120 pixels, em tons de cinza). Em um *notebook* no Google Colab, o Tesseract 4.1.1 e o EasyOCR 1.7.2 foram aplicados a uma foto do mostrador, cujo valor visível era `353485`. Aplicados à linha inteira, nenhum dos dois leu o valor: o Tesseract devolveu sequências sem sentido, como `flslalalals`, e o EasyOCR devolveu `1581`. A leitura só foi correta quando a imagem foi separada em um recorte por dígito, por análise de componentes conectados com o OpenCV, e cada recorte foi reconhecido isoladamente. Por usar uma única foto, o teste serviu apenas como prova de viabilidade, mas indicou a estrutura adotada: localizar os dígitos e classificá-los um a um. Os OCR genéricos não foram usados nas etapas seguintes.

3.2.6.1.2 Pipelines avaliados.

A partir dessa estrutura, três pipelines de OCR foram desenvolvidos em paralelo, e apenas um foi mantido. A Tabela 16 resume os três.

Tabela 16 – Pipelines de OCR desenvolvidos

Pipeline	**Localização da janela**	**Leitura dos dígitos**	**Situação**
A	U-Net de 384 × 384 pixels, treinada com QAT	CRNN com CTC, lendo a janela inteira de uma vez (48 × 224 pixels)	Descartado
B	Rede codificador–decodificador de 192 × 192 pixels, INT8	CNN de um dígito (28 × 28 pixels) aplicada a cada rolete	**Adotado**
C	A mesma U-Net do pipeline A	CNN de um dígito (68 × 68 pixels), treinada com dígitos de fontes genéricas	Descartado

Fonte: Elaborado pelos autores (2026).

O pipeline A aplicava a arquitetura CRNN com função de perda CTC, apresentada na Seção 2.2.5.5, e foi testado diretamente no ESP32-CAM. O teste mostrou que ele não pode ser executado no microcontrolador: o modelo de localização exigiu 15.335.424 bytes de memória de trabalho (*arena*) do TensorFlow Lite Micro, 3,8 vezes os cerca de 4 MB de PSRAM da placa, e os modelos de leitura usam o operador `REDUCE_PROD`, ausente na versão da biblioteca disponível no *core* Arduino-ESP32 3.0.7. O descarte vale para os modelos como foram exportados e não invalida a abordagem com CTC, que permanece como possibilidade de trabalho futuro. O pipeline C foi descartado por rodar apenas no computador e por ter o classificador treinado com dígitos diretamente retirados dos visores (KUCEV, 2020) e visores montados sinteticamente com dígitos manuscritos (IQBAL, 2019), e não com imagens de hidrômetros puras. Devido à complexidade arquitetural e aos desafios de implementação em hardware desses modelos, os detalhes técnicos dos pipelines A e C são aprofundados a seguir.

3.2.6.2 Detalhamento dos modelos explorados (U-Net e CTC)

3.2.6.2.1 O modelo de localização U-Net (Pipelines A e C).

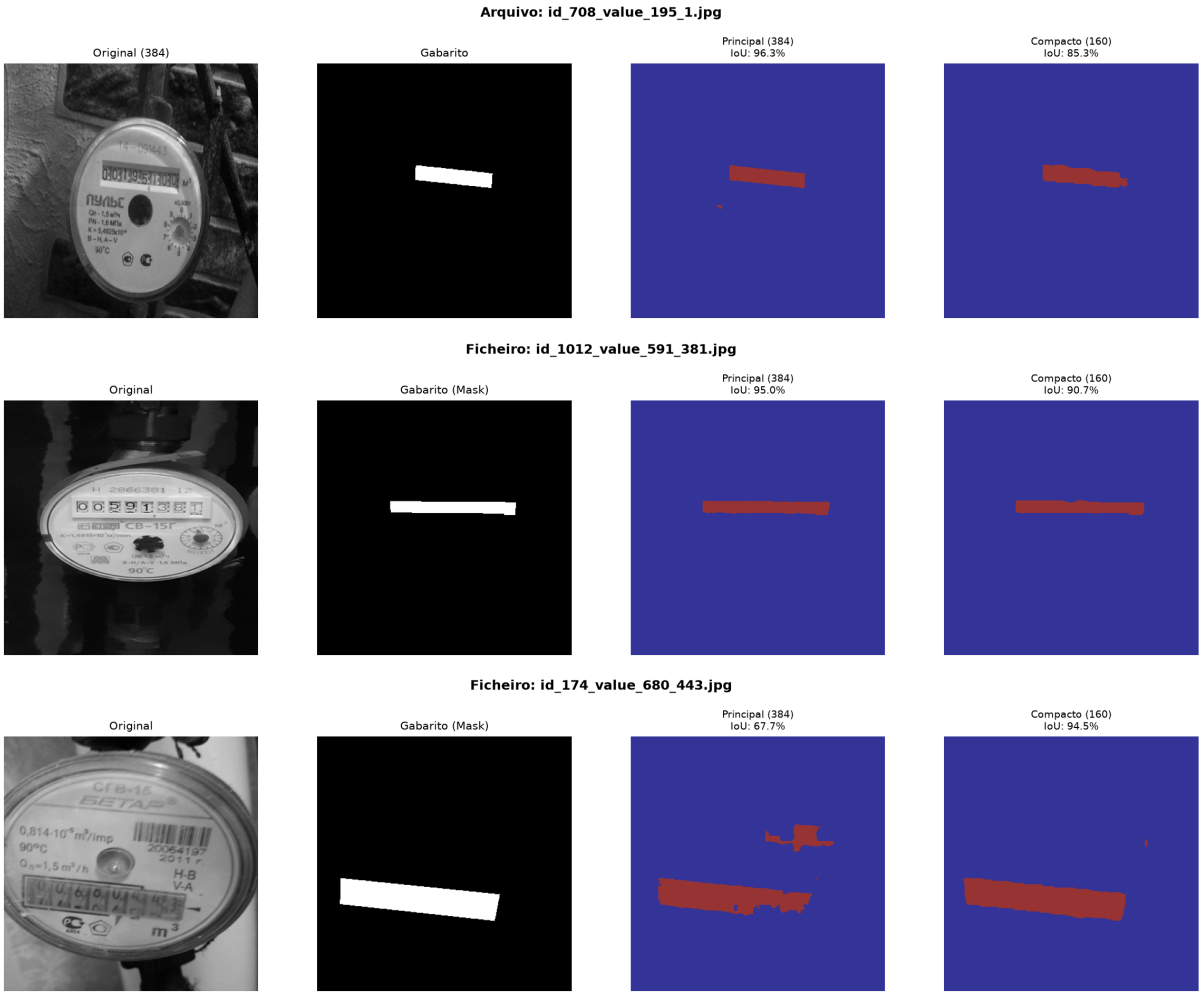
O modelo de localização inicial baseava-se na arquitetura U-Net, projetada para receber imagens de 384 × 384 pixels e extrair uma máscara precisa do visor. Em testes iniciais em ambiente de desenvolvimento, esta rede apresentou uma sobreposição (IoU) frequentemente superior a 95%. No entanto, ao ser convertido para execução em borda, o modelo exigiu cerca de 15 MB de memória RAM (arena), inviabilizando sua execução no ESP32-CAM, que dispõe de aproximadamente 4 MB de PSRAM.

O treinamento desta rede foi realizado em duas fases: ponto flutuante e *Quantization-Aware Training* (QAT). Para lidar com o forte desbalanceamento entre os pixels de fundo e os da janela do visor, adotou-se uma função de perda composta pela soma da Entropia Cruzada Binária (BCE) e da *Dice Loss*. Utilizou-se o otimizador Adam com taxa de aprendizado inicial de 1×10^{-3}, reduzida para 1×10^{-5} na fase de quantização. Para

aumentar a robustez, aplicou-se aumento de dados (*data augmentation*) com espelhamento horizontal, variações de brilho e contraste, além de recortes aleatórios. O treinamento ocorreu por até 250 épocas na fase flutuante e 100 épocas na fase QAT, com parada antecipada (*early stopping*) para evitar sobreajuste.

Na tentativa de contornar o problema de memória, desenvolveu-se uma versão compacta da U-Net, reduzindo a profundidade dos filtros e a resolução de entrada para 160 × 160 pixels, o que diminuiu a arena de memória para cerca de 1,7 MB. Essa versão apresentou um comportamento curioso: em alguns testes, obteve métricas de segmentação superiores à versão grande, ignorando ruídos visuais externos (como códigos de barras no mostrador). Contudo, o modelo compacto apresentou alta instabilidade. Devido à redução agressiva de resolução e ao pós-processamento via contornos conectados, a rede falhava em aproximadamente 15% dos casos, retornando uma máscara vazia. Essa incerteza motivou o abandono da U-Net em favor da rede codificador–decodificador simples descrita na Seção 3.2.6.5.

Figura 11 – Comparação do desempenho do modelo U-Net Principal (384 × 384) e Compacto (160 × 160) na segmentação de visores.



Fonte: Elaborado pelos autores (2026). A primeira coluna mostra a imagem original, a segunda o gabarito (máscara correta) e as duas últimas as previsões dos modelos com seus respectivos IoU.

3.2.6.2.2 O reconhecedor CRNN com CTC (Pipeline A).

O reconhecedor do pipeline A utilizou uma arquitetura CRNN (*Convolutional Recurrent Neural Network*) aliada à perda CTC (*Connectionist Temporal Classification*). Para contornar as restrições de memória, as camadas recorrentes tradicionais (LSTMs) foram substituídas por convoluções temporais 1D com *kernels* largos. O modelo recebia a janela de dígitos redimensionada para 48×224 pixels e previa a sequência completa de até oito dígitos de uma só vez.

O treinamento do modelo CTC utilizou uma abordagem híbrida de dados, combinando imagens reais de hidrômetros (recortadas e rotuladas) com imagens sintéticas geradas a partir de dígitos manuscritos e impressos. A função de perda foi a própria CTC Loss, que resolve o alinhamento temporal sem a necessidade de segmentação prévia. O otimizador utilizado foi o Adam (taxa de aprendizado de 3×10^{-4} na fase flutuante e 5×10^{-5} na fase QAT), com recorte de norma do gradiente (*clipnorm*) fixado em 1.0 para estabilidade. Aplicou-se um aumento de dados agressivo para simular condições reais, incluindo ruído gaussiano, variações extremas de brilho e contraste, além de oclusões aleatórias (blocos pretos) para simular sujeira no visor. O treinamento durou até 600 épocas na fase flutuante e 100 épocas na fase QAT.

A avaliação do modelo CTC em um conjunto de teste composto por 30 imagens reais de hidrômetros (executado em servidor) demonstrou a força da abordagem. Conforme detalhado na Tabela 17, o modelo obteve 86,67% de acerto completo da sequência (26 de 30 imagens) e 94,58% de acurácia por dígito na posição exata.

Tabela 17 – Resumo da avaliação do modelo CRNN com CTC no conjunto de teste

Métrica	**Resultado**
Acurácia da sequência completa (até 8 dígitos)	86,67% (26/30)
Acurácia por dígito (posição exata)	94,58%
Acurácia por dígito (edição/Levenshtein)	95,42%
Acurácia do valor lógico (ignorando zeros à esquerda)	86,67%

Fonte: Elaborado pelos autores (2026).

3.2.6.2.3 Inviabilidade do Pipeline A no hardware alvo.

Apesar do excelente desempenho do CTC em software, a arquitetura completa do pipeline A não pôde ser executada no ESP32-CAM. O motivo principal não foi o consumo do CTC (que, isoladamente, exige cerca de 500 KB de memória, viável para a placa), mas sim a dependência do modelo U-Net para recortar a janela. Como a U-Net estourava a memória e utilizava o operador `REDUCE_PROD` (ausente na biblioteca do microcontrolador), todo o pipeline ficou inviabilizado. A arquitetura CRNN com CTC permanece guardada

como uma alternativa de alta performance para hardwares mais robustos ou trabalhos futuros.

3.2.6.3 Detalhes do Pipeline B (Adotado)

Como as abordagens anteriores falharam em cenários de borda, o pipeline B foi concebido para ser leve e independente de modelos pesados de segmentação.

3.2.6.4 Conjuntos de dados

A principal dificuldade para treinar um classificador de dígitos de hidrômetro é a rotulagem: os conjuntos públicos informam a leitura completa do medidor, e não o valor de cada rolete. Três fontes de dados foram combinadas, cada uma com um tratamento próprio para gerar recortes rotulados de um único dígito:

1. **Water Meters Dataset** (KUCEV, 2020): 1.244 fotos de hidrômetros variados, tiradas em ângulos arbitrários, com a máscara da janela de dígitos e a leitura no nome do arquivo. Foi usado no treinamento do modelo de localização e, para o classificador, passou por um processo automático: a janela foi retificada a partir da máscara e dividida em células por projeção vertical. Para evitar rótulos errados, só foram aceitas as fotos em que a divisão produziu exatamente oito células de larguras semelhantes; as demais foram descartadas. Restaram 195 fotos, que geraram 1.560 recortes.

2. **Word-Wheel Water Meter Dataset** (ZHAO et al., 2026): conjunto público cujo subconjunto de reconhecimento traz recortes padronizados de 200×60 pixels da janela de dígitos, com a leitura completa. Foram descartadas as imagens com um rolete no meio da troca entre dois dígitos, cujo rótulo é ambíguo, e as imagens espelhadas foram invertidas. Como os recortes já vêm retos, cada um foi dividido em fatias de largura igual. Apenas as pastas de treino foram usadas, gerando 47.478 recortes; as pastas de teste, com 1.894 imagens, foram reservadas como teste independente.

3. **Fotos do próprio ESP32-CAM**: para aproximar o modelo das condições do sensor, da lente e da compressão do dispositivo, três fotos capturadas pelo ESP32-CAM, de dois medidores diferentes, tiveram a janela marcada manualmente e foram fatiadas e rotuladas à mão, gerando 24 recortes. Esses recortes foram usados apenas no ajuste fino (*fine-tuning*) do classificador.

Os dois conjuntos públicos somam 49.038 recortes, com forte desbalanceamento: o dígito 0 tem 20.398 exemplos, pois os roletes mais significativos do medidor costumam marcar zero, enquanto os demais dígitos têm entre 2.469 e 4.991. Para a validação, a divisão foi feita por foto, e não por recorte, para que dígitos de uma mesma foto não

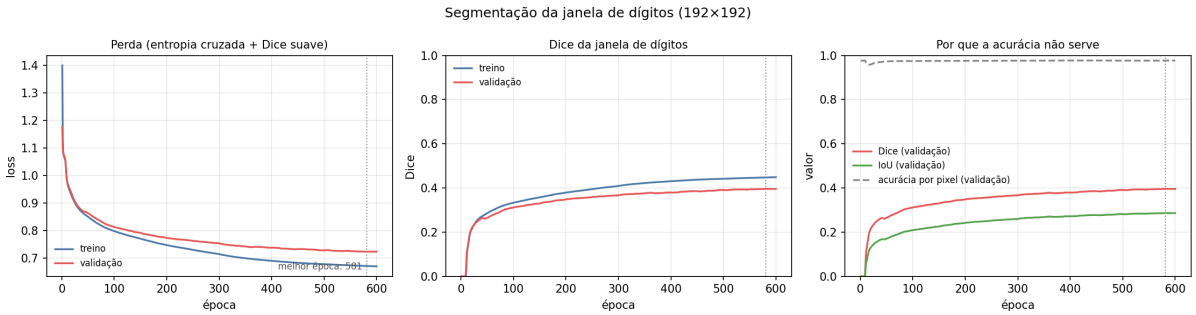
aparecessem ao mesmo tempo no treino e na validação: 15% das fotos foram separadas, resultando em 41.660 recortes de treino e 7.378 de validação.

3.2.6.5 Modelo de localização da janela

O modelo de localização recebe o quadro redimensionado para 192 × 192 pixels e produz uma máscara binária que indica os pixels da janela de dígitos. Trata-se de uma rede codificador–decodificador inspirada na U-Net, porém reduzida para caber no microcontrolador e sem as conexões de salto: duas camadas convolucionais (3 × 3, com 8 e 16 filtros), cada uma seguida de *max pooling*, duas convoluções transpostas que restauram a resolução e uma convolução 1 × 1 com ativação sigmoide na saída. Quantizada em INT8, ocupa 11,8 KB.

O modelo foi treinado com o *Water Meters Dataset*, dividido em 70% para treino, 15% para validação e 15% para teste. Como a janela ocupa uma fração pequena da imagem, a entropia cruzada sozinha levava a rede a classificar quase tudo como fundo; por isso, a perda adotada combina entropia cruzada e Dice suave, o que elevou o Dice de validação de 0,306 para 0,396 com o mesmo número de épocas. O treino durou 600 épocas, com lotes de 16 imagens, e a melhor época foi a 581. A Figura 12 mostra a evolução do treino.

Figura 12 – Evolução do treino do modelo de localização da janela



Fonte: Elaborado pelos autores (2026).

No conjunto de teste, com 186 fotos não vistas no treino, o modelo em INT8 obteve Dice de 0,39 e IoU de 0,28. O resultado é fraco. A acurácia por pixel supera 98%, mas não serve como medida, pois o fundo domina a imagem, como mostra o terceiro gráfico da Figura 12. As curvas de treino e validação próximas indicam subajuste: a rede, minúscula e sem conexões de salto, não tem capacidade para localizar a janela com precisão.

No uso, a caixa da janela é o menor retângulo que contém os pixels marcados pela máscara, convertido para as coordenadas do quadro de 160 × 120 pixels. Como a máscara prevista transborda a janela, essa caixa fica muito maior do que deveria. Nas fotos do conjunto de teste comum, descrito adiante, a caixa prevista teve mediana de 130 × 90 pixels, cobrindo quase todo o mostrador, enquanto a janela real mede cerca de 97 × 18

pixels. Com essa caixa, cada fatia contém pedaços de vários roletes e do mostrador, e a leitura falha.

3.2.6.6 Classificador de dígitos

O classificador recebe a imagem de um único rolete, redimensionada para 28×28 pixels em tons de cinza, e devolve a probabilidade de cada um dos dez dígitos. É uma rede neural convolucional com 72.442 parâmetros, cuja arquitetura é apresentada na Tabela 18. Quantizado em INT8, o modelo ocupa 81,5 KB.

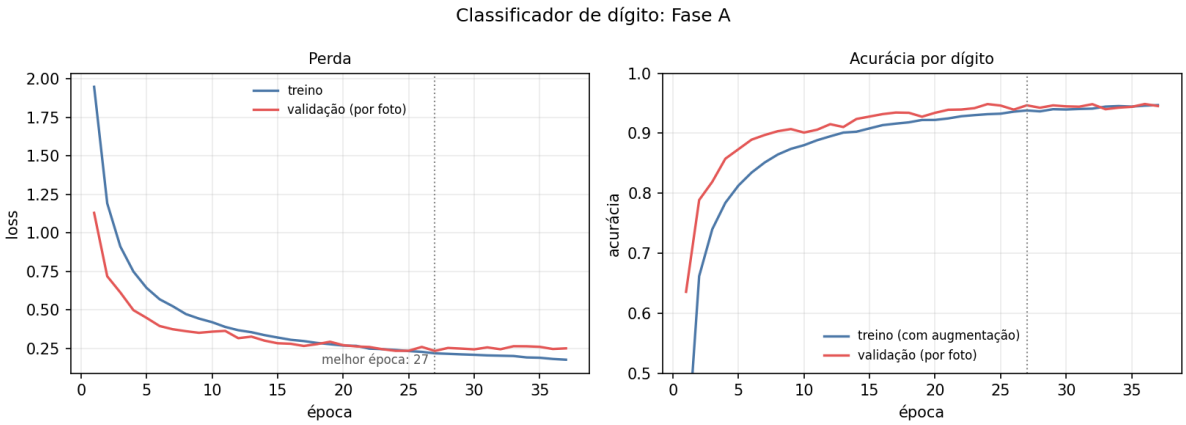
Tabela 18 – Arquitetura do classificador de dígitos

Camada	**Configuração**	**Saída**
Entrada	imagem em tons de cinza	$28 \times 28 \times 1$
Convolução $\times 2$	16 filtros 3×3, ReLU	$28 \times 28 \times 16$
Max pooling	2×2	$14 \times 14 \times 16$
Convolução $\times 2$	32 filtros 3×3, ReLU	$14 \times 14 \times 32$
Max pooling	2×2	$7 \times 7 \times 32$
Convolução	64 filtros 3×3, ReLU	$7 \times 7 \times 64$
Max pooling	2×2	$3 \times 3 \times 64$
Achatamento	—	576
Densa	64 neurônios, ReLU, *dropout* de 30%	64
Densa (saída)	10 neurônios, *softmax*	10

Fonte: Elaborado pelos autores (2026).

O treinamento foi feito em duas fases. Na fase A, o modelo foi treinado com os recortes dos dois conjuntos públicos, com o otimizador Adam (taxa de aprendizado de 5×10^{-4}), lotes de 128 recortes e pesos por classe para compensar o excesso de zeros. Para aproximar os recortes das imagens do ESP32-CAM, foi aplicado aumento de dados com redução e ampliação da resolução, ruído gaussiano, pequenas rotações e translações e variação de contraste. O treino foi interrompido por parada antecipada após 37 épocas, com a melhor época na 27.ª, e atingiu 94,65% de acurácia na validação (Figura 13).

Figura 13 – Evolução do treino do classificador de dígitos (fase A)



Fonte: Elaborado pelos autores (2026).

Na fase B, o modelo passou por um ajuste fino de cinco épocas, com taxa de aprendizado dez vezes menor (5×10^{-5}), usando os 24 recortes do ESP32-CAM, fortemente aumentados e repetidos, junto com uma parte dos recortes da fase A, para que o modelo não esquecesse o que já havia aprendido. Ao final, a acurácia de validação do modelo em ponto flutuante foi de 95,46%.

3.2.6.7 Quantização e exportação

Os dois modelos foram quantizados em INT8 após o treinamento (*post-training quantization*), com um conjunto representativo de imagens para calibrar as escalas. A quantização foi completa: pesos, ativações, entrada e saída em inteiros de 8 bits, usando apenas operadores INT8 do TensorFlow Lite. Na entrada, a escala de 1/255 com ponto zero de -128 permite converter cada pixel diretamente, subtraindo 128 do seu valor. Diferente do pipeline A, que usou treinamento consciente da quantização (QAT), o pipeline B não precisou dessa técnica: a perda de acurácia na quantização foi desprezível, como mostra a avaliação da hipótese H1 nos resultados. Por fim, cada modelo `.tflite` foi convertido em um vetor de bytes em um arquivo de cabeçalho C (`.h`), incluído no firmware.

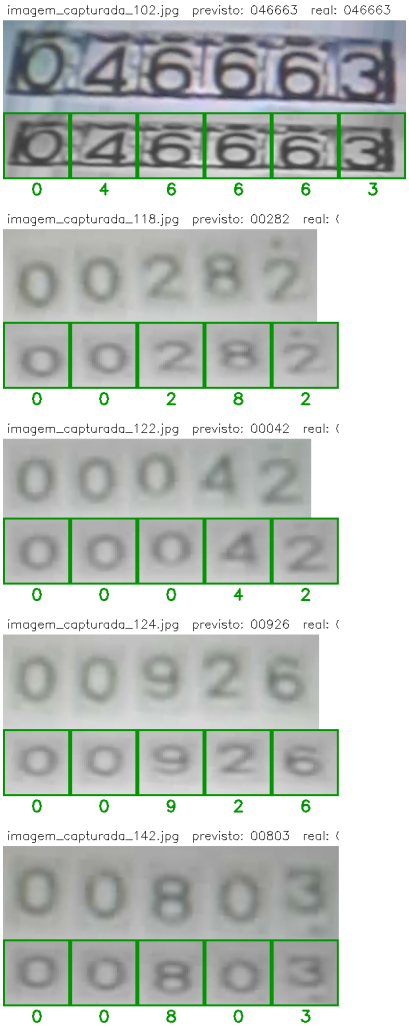
3.2.6.8 Leitura com janela calibrada na instalação

Dada a caixa da janela, a leitura é feita da mesma forma no firmware e no servidor: a caixa é dividida em N fatias de largura igual, uma por rolete; cada fatia é redimensionada para 28×28 pixels por vizinho mais próximo, convertida para INT8 e classificada; e os dígitos de maior probabilidade são concatenados na leitura. O número N de roletes é um parâmetro configurado por medidor: seis no medidor usado nos testes de hardware.

Como o modelo de localização não encontra a janela com precisão, os resultados com a caixa prevista por ele são próximos de zero. Esses resultados **não refletem a capacidade de leitura do classificador**, e sim o erro da localização: com a caixa errada, as fatias

não correspondem aos roletes. Quando a janela é informada corretamente, o mesmo classificador lê com alta acurácia os dígitos escuros sobre fundo claro, que predominam no treino; a leitura de medidores com dígitos claros em fundo escuro é discutida nos resultados (Seção **??**). A Figura 14 mostra cinco capturas do ESP32-CAM em que a janela foi marcada à mão: todas as cinco leituras e os 26 dígitos foram reconhecidos corretamente.

Figura 14 – Leitura com a janela de dígitos marcada manualmente em cinco capturas do ESP32-CAM



Fonte: Elaborado pelos autores (2026). Em cada captura, a faixa superior é a janela recortada, a inferior mostra as fatias e, abaixo, o dígito previsto para cada uma.

Por isso, a solução adotada dispensa a localização automática. Em uma instalação real, a câmera fica fixa diante do medidor, de modo que a janela de dígitos ocupa sempre a mesma região do quadro. Assim, a região de interesse (ROI) e a inclinação dos roletes são calibradas uma única vez, na instalação, e usadas em todas as leituras seguintes. Essa calibração precisa ser precisa: a avaliação apresentada nos resultados indica que a ROI deve estar a cerca de um ou dois pixels da posição correta no quadro de 160×120 pixels, em que cada dígito tem aproximadamente 16 pixels de largura.

Nas medições de latência e consumo em hardware, o firmware de borda ainda executava o modelo de localização a cada leitura, com a câmera posicionada à mão para que a janela

ocupasse a maior parte do quadro e a caixa prevista ficasse próxima dela. Essa configuração é descrita na próxima seção, e o efeito de remover a localização é discutido nos resultados.

3.2.7 Firmware do nó sensor

Os dois firmwares do nó, um para cada fluxo, foram escritos para o ESP32-CAM com o *core* Arduino-ESP32 3.0.7 e compartilham a mesma estrutura: configuração da câmera, comunicação com o módulo de rádio, modo autônomo com *deep sleep* e registro dos tempos de cada etapa. Além do modo autônomo, ambos aceitam comandos pela porta serial, usados nos testes e nas medições: disparo de uma leitura, ativação e desativação do modo autônomo, *deep sleep* avulso e repasse de comandos AT ao módulo de rádio.

3.2.7.1 Ciclo de operação

No modo autônomo, o nó executa ciclos de leitura separados por períodos de *deep sleep*. Ao despertar pelo temporizador, o microcontrolador reinicia, configura a câmera em tons de cinza na resolução de 160×120 pixels, captura um quadro e, logo em seguida, desliga a câmera, pois ela não é necessária no restante do ciclo. A partir desse ponto, cada fluxo segue um caminho, como resume a Tabela 19.

Tabela 19 – Etapas de um ciclo de leitura em cada fluxo

Etapa	**Fluxo em nuvem**	**Fluxo em borda**
Despertar	Reinício pelo temporizador e configuração da câmera	
Captura	Quadro de 160×120 pixels em cinza; câmera desligada em seguida	
Processamento	Compressão JPEG e divisão em blocos de 175 bytes	Localização da janela e classificação dos dígitos com TensorFlow Lite Micro
Envio	Um *uplink* por bloco, com 7 s de intervalo (19 a 27 blocos)	Um único *uplink* de 9 bytes
Registro	Linha de fim de ciclo na serial, com os tempos de cada etapa	
Repouso	*Deep sleep* até o próximo ciclo	

Fonte: Elaborado pelos autores (2026).

Ao fim de cada ciclo, o firmware escreve na serial uma linha padronizada com o número do ciclo, o resultado do envio e a duração de cada etapa (captura, processamento e envio). Essa linha é lida pelos *scripts* de medição, o que permite comparar os dois fluxos com as mesmas ferramentas.

Antes de entrar em *deep sleep*, o firmware desativa o driver da câmera, coloca o sensor em modo de baixo consumo pelo pino de *power-down* (GPIO 32), apaga o LED de *flash* (GPIO 4) e mantém o nível desses pinos durante o repouso. O contador de ciclos e o estado do modo autônomo ficam na memória RTC, que é preservada durante o *deep sleep*. Apenas o ESP32 é colocado em repouso: o módulo Radioenge continua alimentado e não

recebe comando para entrar em modo suspenso, o que mantém nele a sessão LoRaWan e o contador de quadros, mas também mantém o seu consumo durante o repouso, como discutido nos resultados. Nas medições, o intervalo de *deep sleep* foi de 30 s nos dois fluxos.

3.2.7.2 Execução dos modelos no microcontrolador

No fluxo em borda, os modelos são executados com o TensorFlow Lite Micro, na versão `esp-tflite-micro` 1.3.1 com a biblioteca de otimização `esp-nn` 1.1.0, ambas incluídas no *core* 3.0.7. Os dois modelos `.tflite` ficam na memória *flash* como vetores de bytes, e cada um tem o seu próprio interpretador, criado com um registro de operadores que contém apenas os operadores usados pelo modelo: sete na localização da janela e oito no classificador. O firmware compilado ocupa 563 KB, cerca de 17% da memória de programa.

O TensorFlow Lite Micro não aloca memória dinamicamente: cada interpretador recebe uma área de trabalho (*arena*) de tamanho fixo, onde ficam os tensores intermediários. As duas *arenas* foram alocadas na PSRAM de 4 MB da placa. A Tabela 20 mostra a memória reservada e a efetivamente usada, medida na placa durante 61 capturas.

Tabela 20 – Uso da PSRAM pelo fluxo em borda

Item	**Reservado (bytes)**	**Usado (bytes)**
Buffer de quadro da câmera	—	≈19.216
Arena da localização da janela	2.097.152	1.623.852
Arena do classificador de dígitos	786.432	28.556
PSRAM livre após as alocações		1.289.292

Fonte: Elaborado pelos autores (2026).

O dimensionamento das *arenas* exigiu ajustes durante os testes em placa. A primeira estimativa, baseada no maior tensor intermediário do modelo de localização, reservou 1 MB, e a alocação falhou: o interpretador mantém vários tensores intermediários ativos ao mesmo tempo, e o modelo precisou de cerca de 1,6 MB. A *arena* foi então ampliada para 2 MB. A *arena* do classificador, de 768 KB, mostrou-se superdimensionada, pois o modelo usa menos de 29 KB. Para evitar novas tentativas na placa, foi escrito um *script* que reproduz o planejador de memória do TensorFlow Lite Micro a partir do arquivo `.tflite`; para o modelo de localização, ele estimou 1.622.016 bytes, contra 1.623.852 bytes medidos. Esse mesmo *script* confirmou que o modelo de localização do pipeline A, que pediu 15,3 MB, não caberia na placa.

A leitura no firmware segue os passos descritos na Seção 3.2.6.8. O quadro capturado é copiado para um buffer próprio e a câmera é desligada antes da inferência. Para o modelo de localização, o quadro é redimensionado por vizinho mais próximo para 192×192 pixels diretamente no tensor de entrada, já convertido para INT8 pela subtração de 128 de cada pixel. Na saída, os valores INT8 maiores que zero, que correspondem a probabilidade

superior a 0,5, formam a máscara, e a caixa é o menor retângulo que a contém. Cada uma das N fatias da caixa é redimensionada para 28×28 pixels e classificada, e o dígito de maior pontuação é escrito na leitura. Se a máscara vier vazia, a leitura é enviada com traços no lugar dos dígitos.

O tempo de inferência medido na placa foi muito maior do que o esperado para modelos tão pequenos. Em 61 capturas, a localização da janela levou mediana de 15.564 ms, com variação de apenas 4 ms entre capturas, e a classificação dos seis dígitos levou 1.995 ms, para um total de 17.838 ms por leitura. A causa não foi investigada; uma hipótese é que parte dos operadores do modelo de localização, como a convolução transposta, não tenha versão otimizada na `esp-nn` e seja executada pelas implementações de referência, em C genérico. Durante essas capturas, a temperatura interna do chip permaneceu entre 72 e 74 °C.

Como discutido na Seção 3.2.6.8, a solução adotada dispensa o modelo de localização e usa uma ROI calibrada na instalação. O firmware usado nas medições, porém, ainda executa a localização a cada leitura; a versão com ROI fixa não foi implementada. Sem a localização, a leitura no nó ficaria restrita à classificação dos dígitos, de cerca de 2 s, o que é uma estimativa, e não uma medição.

3.2.7.3 Firmware de depuração e calibração

Um terceiro firmware, sem rádio, foi usado para posicionar a câmera diante do medidor e para medir o desempenho dos modelos na placa. A cada 5 s, ele captura um quadro, executa a leitura completa, desenha sobre a imagem a caixa encontrada e as divisões das fatias e envia ao computador a imagem anotada, a leitura e métricas de memória, tempo e temperatura. Um *script* no computador grava as imagens e as métricas em arquivo. Com ele, a câmera foi ajustada à mão até que a janela de dígitos ocupasse o quadro e a caixa prevista coincidisse com ela; a partir desse ponto, a leitura do medidor de teste se estabilizou. Os dados de memória e de tempo de inferência apresentados nesta seção vêm desse firmware.

3.2.8 Servidor da aplicação

No servidor, a recepção é feita pelo *script* `reconstrutor.py`, descrito na Seção 3.2.5.5: no fluxo em borda, a leitura chega pronta na porta 2; no fluxo em nuvem, a imagem remontada é gravada em disco. O OCR do fluxo em nuvem é executado sobre essas imagens pelo *script* `pipeline_nuvem.py`.

Para que a única diferença entre os fluxos seja o local do processamento, o servidor executa os mesmos arquivos `.tflite` INT8 do firmware, com o interpretador LiteRT 2.3.0, e reproduz a mesma lógica: o mesmo redimensionamento por vizinho mais próximo, com o mesmo cálculo de índices, a mesma conversão para INT8, o mesmo limiar na máscara

e o mesmo fatiamento. O acelerador XNNPACK do LiteRT foi desativado, para que os operadores sejam executados pelas implementações de referência, como no microcontrolador. A única diferença no dado de entrada é a compressão JPEG com qualidade 90, que o fluxo em borda não sofre.

3.2.8.1 Tempo de inferência no servidor

Na implementação atual, o OCR do servidor não é disparado automaticamente quando a imagem fica completa: ele é executado depois, sobre as imagens gravadas. Por isso, a latência medida no fluxo em nuvem termina no momento em que a imagem fica disponível no servidor, e o tempo de inferência foi medido à parte. Em um computador com processador AMD Ryzen 5 5600GT, as 21 imagens completas recebidas pelo enlace foram lidas cinco vezes cada. A Tabela 21 compara os tempos com os do microcontrolador.

Tabela 21 – Tempo de inferência por leitura no nó e no servidor (mediana)

Etapa	**ESP32-CAM (ms)**	**Servidor (ms)**
Localização da janela	15.564	4,30
Leitura completa (localização e seis dígitos)	17.838	6,25
Leitura com ROI fixa (só os seis dígitos)	≈1.995	1,91

Fonte: Elaborado pelos autores (2026). No servidor, os tempos incluem a decodificação do JPEG; no ESP32-CAM, a leitura com ROI fixa corresponde ao tempo medido da classificação dos dígitos.

O servidor executa a leitura completa cerca de 2.800 vezes mais rápido que o microcontrolador. Diante da latência de envio da imagem, de mais de dois minutos, o tempo de inferência no servidor é desprezível, e a omissão dessa etapa na latência medida não altera a comparação entre os fluxos.

3.2.8.2 Recursos disponíveis apenas no servidor

O baixo custo da inferência no servidor permite etapas que seriam caras no microcontrolador. Duas foram implementadas e avaliadas:

- **Correção da inclinação**: a fileira de roletes costuma aparecer levemente inclinada no quadro. O servidor detecta as bordas da imagem (Canny), extrai os segmentos de reta pela transformada de Hough, toma a mediana ponderada pelo comprimento dos ângulos dos segmentos quase horizontais (até ±20°) e gira o quadro para endireitá-lo antes da leitura.
- **Votação entre caixas deslocadas**: para tolerar pequenos erros na posição da ROI, o classificador é aplicado a cinco versões da caixa, a original e outras quatro deslocadas de 2 pixels na horizontal e de 1 pixel na vertical, e as pontuações de cada

fatia são somadas antes da escolha do dígito. No microcontrolador, isso multiplicaria por cinco os cerca de 2 s da classificação.

A ROI é informada ao servidor como um retângulo em coordenadas do quadro de 160×120 pixels e é calibrada uma vez por instalação. Para isso, um *script* auxiliar desenha uma grade de coordenadas sobre o quadro ampliado, a partir da qual a janela é lida à mão. Ainda não existe uma ferramenta que faça essa calibração de forma automática ou guiada, o que é uma limitação da solução atual. O efeito da correção da inclinação, da votação e da precisão da ROI sobre a acurácia é apresentado nos resultados.

3.3 Experimentos e cenário de testes

Os experimentos foram planejados para responder às três hipóteses da Seção 1.3.3. A Tabela 22 relaciona cada experimento à hipótese avaliada, aos dados utilizados e às métricas principais. Os experimentos de reconhecimento foram executados no computador, sobre conjuntos de imagens fixos e versionados no repositório. Os de latência, entrega e consumo foram executados com o protótipo em bancada, com os firmwares dos dois fluxos descritos na Seção 3.2.7.

Tabela 22 – Plano de experimentação

Experimento	**Hipótese**	**Dados e condição**	**Métricas**
Efeito da quantização	H1	Mesmo classificador em ponto flutuante e em INT8; 7.378 recortes de validação e 1.894 imagens de teste independentes	Acurácia por recorte, leitura exata, acurácia por dígito, CER
Leitura de ponta a ponta	—	39 fotos do conjunto de teste comum e 21 fotos de hidrômetros físicos; janela prevista pelo modelo e janela marcada à mão	Leitura exata, acurácia por dígito, CER, Dice e IoU
Latência e entrega	H2	Nó em ciclo autônomo com *deep sleep*: 30 leituras no fluxo em nuvem e 34 no fluxo em borda; 30 envios adicionais de imagem com o nó sempre ligado	Latência ponta a ponta, PDR por pacote, taxa de entrega por leitura
Consumo de energia	H3	Corrente do nó medida em 30 ciclos do fluxo em nuvem e 32 do fluxo em borda	Carga por leitura (mAh), corrente acordado e em repouso, autonomia estimada

Fonte: Elaborado pelos autores (2026).

Como os dois fluxos usam os mesmos modelos INT8 e a mesma lógica de leitura (Seção 3.2.1), a acurácia do reconhecimento não é uma variável entre eles. Por isso, a hipótese H1 é avaliada pela comparação entre as versões em ponto flutuante e em INT8 do mesmo modelo, e não entre os fluxos. Já nos experimentos das hipóteses H2 e H3, a única variável é o fluxo: o hardware, o enlace, o intervalo de repouso e a bancada são os mesmos.

3.3.1 Métricas

3.3.1.1 Reconhecimento

Para cada imagem, a leitura prevista $\hat{y}$ é comparada com a leitura real y, ambas sequências de N dígitos. São usadas três métricas sobre um conjunto de M imagens: a taxa de leitura exata, que conta apenas as leituras com todos os dígitos corretos; a acurácia por dígito, que compara posição a posição; e a taxa de erro de caracteres (*character error rate*, CER), baseada na distância de edição de Levenshtein d_{L}, isto é, o número mínimo de inserções, remoções e substituições que transformam uma sequência na outra:

$$Acc_{\text{leitura}} = \frac{1}{M}\sum_{m=1}^{M}\mathbf{1}[\hat{y}_m = y_m] \qquad Acc_{\text{dígito}} = \frac{\sum_m \sum_{k=1}^{N_m}\mathbf{1}[\hat{y}_{m,k} = y_{m,k}]}{\sum_m N_m} \tag{3.7}$$

$$CER = \frac{\sum_m d_{\mathrm{L}}(\hat{y}_m, y_m)}{\sum_m N_m} \tag{3.8}$$

A taxa de leitura exata é a métrica mais exigente e a que interessa à aplicação: uma leitura com um único dígito errado é uma leitura errada. Para o classificador isolado, a métrica é a acurácia por recorte, a fração de recortes de um único dígito classificados corretamente, acompanhada da acurácia por classe e da matriz de confusão.

A qualidade da localização da janela é medida pela sobreposição entre a máscara prevista P e a máscara de referência G, pelos coeficientes Dice e IoU (*intersection over union*):

$$Dice = \frac{2\,|P \cap G|}{|P| + |G|} \qquad IoU = \frac{|P \cap G|}{|P \cup G|} \tag{3.9}$$

3.3.1.2 Latência e entrega

A latência ponta a ponta de uma leitura é o intervalo entre o disparo no nó e o instante em que o dado fica disponível no servidor: a imagem remontada, no fluxo em nuvem, ou a leitura decodificada, no fluxo em borda. A entrega é avaliada de duas formas. A primeira é o PDR por pacote, definido na Equação (3.1). A segunda é a taxa de entrega por leitura, a fração das leituras disparadas cujo dado chegou completo ao servidor:

$$E_{\text{leitura}} = \frac{\text{leituras disponíveis no servidor}}{\text{leituras disparadas}} \tag{3.10}$$

As duas métricas diferem no fluxo em nuvem: como nenhum dos firmwares retransmite blocos no modo autônomo, uma imagem com um único bloco perdido não é remontada, e a leitura se perde mesmo com um PDR alto. No fluxo em borda, em que cada leitura ocupa um pacote, as duas coincidem. As taxas de entrega por leitura dos dois fluxos são comparadas pelo teste exato de Fisher, adequado a proporções com amostras pequenas.

3.3.1.3 Energia

A carga consumida em uma leitura é a integral da corrente $i(t)$ ao longo de um ciclo completo, de um despertar ao seguinte, incluindo o repouso, calculada pela regra do trapézio sobre as amostras e expressa em mAh:

$$Q_{\text{leitura}} = \frac{1}{3600} \int_{t_{\text{despertar},k}}^{t_{\text{despertar},k+1}} i(t)\, dt \tag{3.11}$$

com i em mA e t em segundos. Para estimar a autonomia em outros intervalos entre leituras, o ciclo é modelado por dois estados: acordado, com corrente média I_{ac} durante t_{ac}, e em repouso, com corrente I_{rep} no restante do intervalo T. A autonomia A, em dias, para uma bateria de capacidade C_{bat} em mAh, é então:

$$Q_{\text{leitura}}(T) = \frac{I_{\text{ac}}\, t_{\text{ac}} + I_{\text{rep}}\, (T - t_{\text{ac}})}{3600} \qquad A = \frac{C_{\text{bat}}}{Q_{\text{leitura}}(T)} \cdot \frac{T}{86400} \tag{3.12}$$

O modelo é ideal: não considera as perdas do regulador, a autodescarga nem a variação da capacidade da bateria com a temperatura, de modo que a autonomia calculada é um limite superior.

3.3.2 Cenário de bancada

Todas as medições em hardware foram feitas em 27 de outubro de 2026, em bancada, com o nó sensor e o receptor a cerca de 2 m de distância. Nessa condição, o RSSI médio dos quadros recebidos ficou entre −34 e −27 dBm, um sinal muito forte. Como antecipado na Seção 3.2.4, o cenário avalia o comportamento dos fluxos, e não o alcance do enlace. O enlace usou a configuração da Tabela 12 nos dois fluxos, e a câmera ficou fixa, apontada para o mostrador de um hidrômetro.

Nos experimentos de latência, entrega e consumo, os dois firmwares operaram no modo autônomo com repouso de 30 s (`AUTO:30`), descrito na Seção 3.2.7.1, e sem a janela para receber o comando de parada após o despertar. O intervalo de 30 s não representa uma instalação real, em que as leituras seriam espaçadas de minutos ou horas; ele foi escolhido para obter pelo menos 30 ciclos de cada fluxo em poucas horas de medição. O efeito de intervalos mais longos é obtido pelo modelo da Equação (3.12).

O nó foi alimentado pelos 5 V da porta USB da placa de gravação ESP32-CAM-MB, que também fornecia a serial usada para acompanhar o log do firmware. O computador lia

ao mesmo tempo a serial do nó e a do receptor, de modo que todos os instantes foram marcados por um único relógio, o do computador. A incerteza desses instantes, de alguns milissegundos, é desprezível diante das latências medidas.

3.3.3 Avaliação do reconhecimento

3.3.3.1 Efeito da quantização (H1)

O classificador de dígitos treinado (Seção 3.2.6.6) foi salvo em ponto flutuante de 32 bits e, após a quantização (Seção 3.2.6.7), em INT8. As duas versões foram avaliadas sobre os mesmos 7.378 recortes de validação, da divisão por foto descrita na Seção 3.2.6.4, por um *script* que apenas carrega os modelos já treinados e mede, sem treinar novamente. Como os pesos de origem e os dados são os mesmos, a diferença de acurácia entre as versões isola o efeito da quantização.

Como os recortes de validação vêm das mesmas fontes usadas no treino, o modelo INT8 também foi avaliado nas pastas de teste do *Word-Wheel Water Meter Dataset*, que ficaram fora de todo o treinamento: 1.894 imagens da janela de dígitos, de 200×60 pixels. Cada imagem foi dividida em N fatias iguais, como no firmware, e cada fatia foi classificada. Como essas imagens já são recortes retos da janela, esse teste mede o classificador e o fatiamento, sem a etapa de localização. O teste foi repetido com o modelo ao fim da fase A e com o modelo final, após o ajuste fino da fase B, para verificar se o ajuste às imagens do ESP32-CAM prejudicou a generalização.

3.3.3.2 Leitura de ponta a ponta

Para avaliar o pipeline completo, da imagem da câmera até a leitura, foram usados dois conjuntos de fotos capturadas pelo ESP32-CAM, nenhum deles usado no treino ou no ajuste dos modelos.

O **conjunto de teste comum** reúne 39 fotos de um monitor que exibia imagens de hidrômetros, com 19 leituras distintas de dois modelos de medidor, um de cinco e outro de seis roletes. As fotos foram selecionadas entre 152 capturas, com dois critérios: sem rotação de 90° ou 180° e sem nenhum rolete parado entre dois números, caso em que a leitura real é ambígua. Por comparação de *hash*, verificou-se que nenhuma delas é idêntica às fotos usadas no treino. O gabarito de cada foto foi conferido manualmente, sobre uma folha que imprime a leitura esperada sobre a imagem, e para cada foto também foi marcada à mão a janela de dígitos.

O **conjunto de hidrômetros físicos** reúne 21 fotos de medidores reais, de diferentes modelos, com cinco a oito roletes, capturadas pelo ESP32-CAM no quadro de 160×120 pixels. Ao contrário do conjunto comum, essas fotos têm reflexos no vidro, iluminação variada e ângulos diversos. A leitura de cada foto foi anotada manualmente. Para cada foto, foram marcados também a inclinação da fileira de roletes, a janela de dígitos no quadro

endireitado e a polaridade da janela: dígitos escuros em fundo claro ou dígitos claros em fundo escuro. A inclinação foi a estimada pela transformada de Hough quando ela deixava a fileira reta, em 16 fotos, e corrigida à mão nas outras cinco. A janela foi marcada pela geometria da imagem, sobre o quadro ampliado com uma grade de coordenadas, a partir das bordas da janela e dos centros dos roletes, de modo que as divisões das fatias caíssem entre eles. Todas as marcações foram concluídas antes da execução da leitura e não foram ajustadas pelo resultado.

Cada foto foi avaliada nos dois fluxos. No fluxo em borda, o quadro de 160×120 pixels é lido diretamente; no fluxo em nuvem, o mesmo quadro é antes codificado em JPEG com qualidade 90 e decodificado, como ocorre na transmissão. Essa avaliação é uma emulação em Python, com os mesmos arquivos `.tflite` INT8, e não a execução na placa. No conjunto comum, as fotos foram lidas em três configurações:

1. **Janela prevista pelo modelo**: o pipeline completo, como no firmware, com a caixa obtida da máscara do modelo de localização.

2. **Janela marcada à mão**: a caixa é informada diretamente e o modelo de localização não é usado. Essa configuração corresponde à ROI calibrada na instalação (Seção 3.2.6.8) e mede o classificador e o fatiamento quando a janela está correta.

3. **Janela marcada à mão com correção de inclinação**: o quadro é antes endireitado no servidor pelo ângulo estimado com a transformada de Hough, recurso disponível apenas no fluxo em nuvem (Seção 3.2.8.2).

A distinção entre essas configurações é essencial para interpretar os resultados. Quando a janela é prevista pelo modelo, o erro da leitura é dominado pelo erro da localização: com uma caixa que cobre quase todo o mostrador, as fatias não correspondem aos roletes, e nenhum classificador acertaria a leitura. Assim, os resultados da primeira configuração medem o modelo de localização, e não a capacidade de leitura do classificador, que é medida pelas configurações com a janela marcada à mão.

O conjunto de hidrômetros físicos, cujas fotos têm inclinações de até cerca de 55°, foi lido com a janela prevista pelo modelo e com a janela e a inclinação marcadas à mão. Nesse caso, a rotação do quadro foi aplicada nos dois fluxos, o que no firmware exigiria uma rotação com ângulo fixo, ainda não implementada. Como variante de diagnóstico, as fotos de fundo escuro foram lidas também com os tons da janela invertidos, o que equivale a informar a polaridade do medidor na instalação, assim como o número de roletes. Depois da avaliação do conjunto completo, os resultados também foram calculados para o subconjunto de fotos que atendem às condições de uma instalação adequada: câmera frontal, dígitos escuros em fundo claro e todos os dígitos visíveis no quadro. Por ter sido definido após a avaliação, esse subconjunto é apresentado como complemento, e o conjunto completo continua sendo a referência.

Por fim, como a ROI de uma instalação real é calibrada uma única vez e pode ter pequenos desvios, foi medida a tolerância da leitura ao erro de posição da janela. Partindo da janela marcada à mão nas 39 fotos do conjunto comum, já endireitadas e no fluxo em nuvem, cada uma das quatro coordenadas da caixa recebeu um erro inteiro sorteado uniformemente entre $-d$ e $+d$ pixels, para $d \in \{0, 1, 2, 3, 4, 6\}$, com três sementes aleatórias. Cada caso foi lido com fatias iguais e com a votação entre cinco caixas deslocadas.

3.3.4 Latência e entrega (H2)

A latência e a entrega foram medidas com o *script* `latencia_ponta_a_ponta.py`, com o nó no modo autônomo. Nesse modo, o computador apenas escuta: o disparo é a primeira linha que o firmware escreve na serial após o despertar, e o fim é o instante em que a imagem é remontada ou a leitura é decodificada. Como o boot da ROM do ESP32 não aparece na serial, o disparo marcado ocorre cerca de 0,3 a 0,5 s após o despertar real, o que subestima a latência nesse valor nos dois fluxos. No fluxo em nuvem, a latência não inclui a execução do OCR no servidor, que é feita depois sobre as imagens salvas; como mostra a Tabela 21, esse tempo é da ordem de milissegundos e não altera a comparação. Foram feitas duas corridas:

- **Fluxo em nuvem**: 31 despertares, das 9h34 às 11h20, simultâneos à medição de corrente. Em 30 deles o nó enviou uma imagem; no outro, o nó acordou e voltou a dormir sem enviar nada, e esse despertar foi excluído.
- **Fluxo em borda**: 34 despertares, das 12h59 às 13h27.

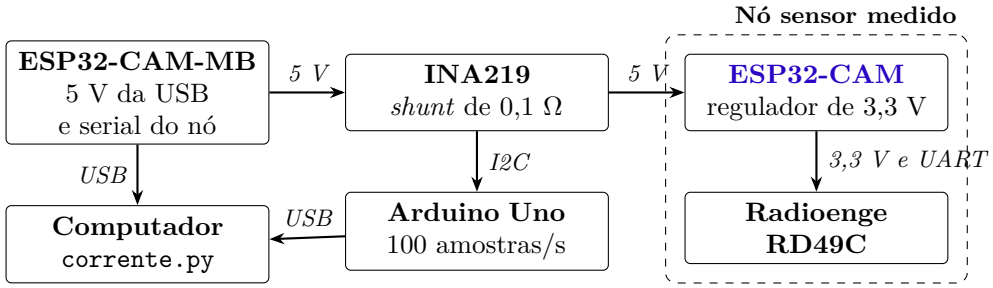
Para conferir o PDR do fluxo em nuvem com uma contagem exata dos pacotes enviados, foi feita uma terceira corrida com o *script* `reconstrutor.py` no modo controlado. Nele, o nó fica sempre ligado, sem *deep sleep*, e o computador envia o comando de captura, espera o fim do envio e, após uma pausa de 10 s, dispara a leitura seguinte. Foram 30 envios, das 14h59 às 16h44. O número de pacotes enviados vem do log do próprio nó, que registra cada tentativa de transmissão, e não da lacuna no contador de quadros (FCnt), que não detecta perdas no início nem no fim de uma imagem. Cada corrida grava todos os quadros recebidos, os logs das duas seriais e as imagens remontadas, que ficam versionadas no repositório. O tempo no ar de cada envio é calculado pela Equação (3.6), e não medido.

Em algumas execuções, o receptor deixou de repassar quadros ao computador por 15 a 25 minutos seguidos, enquanto o nó registrava todos os envios como aceitos pelo módulo. Isso ocorreu em uma primeira tentativa do fluxo em borda, das 12h25 às 12h46, e em cinco execuções do fluxo em nuvem, das 13h41 às 14h55. Essas execuções foram interrompidas e mantidas no repositório em arquivos separados, como registro, mas não entram nas estatísticas: as perdas nelas vêm de uma falha do receptor, cuja causa não foi identificada, e não do enlace. Essa exclusão é retomada na discussão das limitações.

3.3.5 Consumo de energia (H3)

A corrente do nó foi medida com um sensor INA219, cujo resistor *shunt* de 0,1 Ω foi ligado em série na alimentação de 5 V, antes da ESP32-CAM. Como o Radioenge é alimentado pelo regulador de 3,3 V da própria placa, a medição inclui o nó inteiro: microcontrolador, câmera, módulo de rádio, regulador e LED. Um capacitor de 100 µF, ligado depois do resistor, evita a queda de tensão nos picos de transmissão. O INA219 foi lido por I2C por um Arduino Uno, que enviava ao computador uma amostra a cada 10 ms (100 Hz), com a tensão no resistor e a tensão de alimentação. A Figura 15 mostra a montagem.

Figura 15 – Montagem da medição de corrente do nó sensor



Fonte: Elaborado pelos autores (2026).

O INA219 foi configurado com o ganho que admite até 320 mV no resistor (3,2 A), para não saturar nos picos de transmissão. A corrente é calculada no computador pela tensão no resistor, cujo passo de 10 µV corresponde a 0,1 mA. Com o nó desconectado, a leitura foi de 0,00 mV, de modo que nenhum *offset* foi descontado. Considerando a tolerância do resistor e o *offset* típico do sensor, a incerteza é de cerca de 1 mA mais 1% da leitura, o que torna confiáveis apenas as correntes de repouso acima de alguns miliampères. A taxa de 100 amostras por segundo registra cerca de 13 amostras em cada pacote de 135,8 ms no ar.

A análise, feita pelo *script* `corrente.py`, separa os ciclos pelos trechos de repouso. O nível de repouso é estimado pela mediana dos 10% menores valores da captura, e todo trecho abaixo de um limiar, de 5 mA ou 50% acima desse nível, o que for maior, com pelo menos 12 s de duração, é considerado repouso. Cada ciclo vai do fim de um repouso ao fim do seguinte, e a carga do ciclo é calculada pela Equação (3.11). A duração mínima de 12 s foi necessária porque picos isolados de uma única amostra, de cerca de 61 mA, partiam alguns trechos de repouso em dois e criavam ciclos falsos. Foram feitas duas capturas principais:

- **Fluxo em nuvem**: 685.000 amostras em 6.850 s, a partir das 9h29. Dos 31 ciclos encontrados, um foi excluído: o mesmo despertar sem envio excluído na medição de latência, em que o nó ficou acordado por 0,97 s. Restaram 30 ciclos completos.

- **Fluxo em borda**: 169.738 amostras em 1.697 s, das 12h22 às 12h49, com 32 ciclos completos.

Antes dessas capturas, um teste curto de quatro ciclos com uma versão anterior do firmware do fluxo em nuvem, que mantinha a câmera ligada durante todo o envio, foi usado como referência para medir o efeito de desligá-la após a captura.

A partir das médias medidas de corrente e tempo acordado de cada fluxo, a autonomia foi estimada pela Equação (3.12) para uma bateria hipotética de 2.000 mAh e para intervalos de 10 minutos e de 1 hora entre leituras. A corrente de repouso medida, de cerca de 24 mA, inclui o Radioenge, que permanece ligado durante o *deep sleep* do ESP32 (Seção 3.2.7.1), e os demais componentes da placa. A medição que separaria essas parcelas, com o módulo desconectado, não foi realizada. Por isso, a estimativa foi repetida com correntes de repouso hipotéticas de 5, 1 e 0,1 mA, para mostrar como o resultado depende desse valor.

3.4 Resultados e análises

Esta seção apresenta os resultados dos experimentos descritos na Seção 3.3 e os analisa à luz das hipóteses da Seção 1.3.3. Salvo indicação em contrário, os valores foram calculados a partir dos dados brutos versionados no repositório do projeto, com os *scripts* de análise descritos naquela seção. A Seção **??** reúne o resultado de cada hipótese, e a Seção **??**, as limitações que devem acompanhar a sua leitura.

3.4.1 Reconhecimento dos dígitos e efeito da quantização (H1)

A Tabela **??** apresenta o desempenho do classificador de dígitos. Nos 7.378 recortes de validação, o modelo em ponto flutuante acertou 7.043 (95,46%) e o modelo em INT8, 7.037 (95,38%): a quantização custou seis recortes, ou 0,08 ponto percentual.

Tabela 23 – Desempenho do classificador de dígitos em ponto flutuante e em INT8

Conjunto	Modelo	Leitura exata	Acerto por dígito	CER
Validação (7.378 recortes)	Ponto flutuante	—	95,46%	—
	INT8	—	95,38%	—
Teste independente *Word-Wheel* (1.894 imagens, 9.719 dígitos)	INT8, fase A	92,4%	97,3%	0,026
	INT8, fases A e B	92,8%	97,7%	0,023

Fonte: Elaborado pelos autores (2026). Na validação, cada recorte é um único dígito, de modo que a acurácia por recorte coincide com o acerto por dígito.

Como os dois modelos foram avaliados sobre os mesmos recortes, a comparação foi feita de forma pareada. As duas versões deram a mesma resposta em 99,62% dos recortes. Dos que divergiram, 14 foram acertados apenas pelo modelo em ponto flutuante e 8 apenas pelo modelo INT8. Pelo teste exato de McNemar, que considera apenas esses pares discordantes, a diferença não é estatisticamente significativa ($p = 0{,}29$). A ordem de grandeza da perda também aparece em outro detalhe: executado com os *kernels* otimizados XNNPACK, em vez dos *kernels* sem esse delegado usados no restante do trabalho, o mesmo arquivo INT8 acertou 7.040 recortes. A troca da implementação dos operadores altera o resultado na mesma escala que a própria quantização.

O resultado é coerente com o tamanho e a forma do modelo. A quantização completa após o treinamento, calibrada com um conjunto representativo, foi suficiente para um classificador convolucional pequeno, com ativações ReLU, e o pipeline B não precisou do treinamento consciente da quantização descrito na Seção 2.2, que no pipeline A havia sido usado justamente para mitigar a degradação (FERREIRA, 2022). Os dois modelos INT8 somam 93,3 KB (81,5 KB do classificador e 11,8 KB da localização), na faixa de centenas de quilobytes relatada para modelos de OCR de medidores em hardware embarcado (ADEMOLA; PETLENKOV; LEIER, 2023; JAWAS; INDRIANTO, 2018).

Por classe, a acurácia em INT8 variou de 90,84% no dígito 7 a 97,21% no dígito 0, que é também o mais frequente. Os dígitos 7, 9 e 6 foram os mais difíceis (90,84%, 91,64% e 92,18%). Dos 341 erros, 110 (32%) foram classificações como 0, o que indica que os pesos por classe compensaram apenas em parte o excesso de zeros no treino.

No teste independente, com as 1.894 imagens que ficaram fora de todo o treinamento, o modelo INT8 leu 92,8% das leituras inteiras e 97,7% dos dígitos, com CER de 0,023. O ajuste fino com as imagens do ESP32-CAM (fase B) não prejudicou a generalização: melhorou 0,4 ponto percentual a leitura exata em relação ao modelo da fase A. Esse número é inferior aos 95,6% de leituras exatas que uma versão anterior do classificador obtinha nesse mesmo conjunto, mas aquela versão havia sido treinada com essas imagens. A diferença mostra o quanto um teste contaminado pelo treino superestima o desempenho, e é por isso que o valor adotado é o do teste independente.

Quanto à hipótese H1, o modelo INT8 teve, de fato, acurácia menor que a do modelo em ponto flutuante, como previsto. A redução, porém, é de 0,08 ponto percentual e não é estatisticamente significativa, de modo que, na prática, a quantização não penalizou o reconhecimento. Além disso, como os dois fluxos executam os mesmos modelos INT8 (Seção 3.2.1), a acurácia não é um fator de diferença entre eles: o servidor poderia executar o modelo em ponto flutuante sem restrição de memória, mas ganharia no máximo essa mesma fração.

3.4.2 Leitura de ponta a ponta

A Tabela **??** apresenta a leitura completa, da imagem da câmera até a sequência de dígitos, nas configurações descritas na Seção 3.3.3.2.

Tabela 24 – Leitura de ponta a ponta nos dois fluxos

		Fluxo em borda		**Fluxo em nuvem**	
Conjunto	**Janela de dígitos**	**Leituras**	**Dígitos**	**Leituras**	**Dígitos**
Comum (39 fotos, 222 dígitos)	Prevista pelo modelo	0%	15%	0%	16%
	Marcada à mão	46%	85%	46%	85%
	Marcada à mão, quadro endireitado	—	—	82%	92%
Físico (21 fotos, 152 dígitos)	Prevista pelo modelo	10%	31%	0%	27%
	Marcada à mão, quadro endireitado	10%	36%	10%	37%
	Marcada à mão, endireitado, tons invertidos no fundo escuro	14%	56%	14%	56%

Fonte: Elaborado pelos autores (2026). Leituras: taxa de leitura exata; dígitos: acerto por dígito. No conjunto comum, a inclinação é estimada pela transformada de Hough e só é corrigida no servidor; no físico, é marcada à mão e corrigida nos dois fluxos.

3.4.2.1 O gargalo é a localização da janela

Com a janela prevista pelo modelo de localização, nenhuma das 39 fotos do conjunto comum foi lida corretamente, em nenhum dos dois fluxos, e apenas 15% a 16% dos dígitos coincidiram com o gabarito. Esse resultado **não mede o classificador**. Nesse conjunto, a máscara prevista teve Dice de 0,23 e IoU de 0,13 em relação à janela real, valores ainda menores que os do teste do próprio modelo (Seção 3.2.6.5), e a caixa prevista teve mediana de 130×90 pixels, contra cerca de 97×18 pixels da janela real. Com uma caixa que cobre quase todo o mostrador, as fatias não correspondem aos roletes. Endireitar o quadro no servidor antes da localização também não resolveu: a caixa continuou com cerca de 90 pixels de altura, e a leitura exata continuou em 0%.

Quando a janela é marcada à mão, o mesmo classificador, com o mesmo fatiamento, leu 46% das leituras e 85% dos dígitos; com o quadro endireitado pelo servidor, chegou a 82% das leituras e 92% dos dígitos. No medidor de seis roletes, com 27 fotos, foram 85% das leituras e 97% dos dígitos. A diferença entre 0% e 82% vem inteiramente da janela: o modelo de reconhecimento é o mesmo nas duas linhas. As sete leituras que ainda falharam nessa configuração são as três fotos de uma mesma leitura com contraste muito baixo e

quatro fotos de duas leituras do medidor de seis roletes, cada uma com um ou dois dígitos trocados.

Esse comportamento se repetiu no hardware. Nas primeiras capturas do firmware de depuração, antes do ajuste da posição da câmera, a caixa prevista era enorme e a leitura, errada. Depois que a câmera foi posicionada à mão, de modo que a janela ocupasse a maior parte do quadro, a caixa encolheu, e as doze últimas capturas leram a leitura real do medidor de teste, `046663`. Nas cinco capturas do ESP32-CAM com a janela marcada à mão (Figura 14), todas as leituras e os 26 dígitos foram reconhecidos.

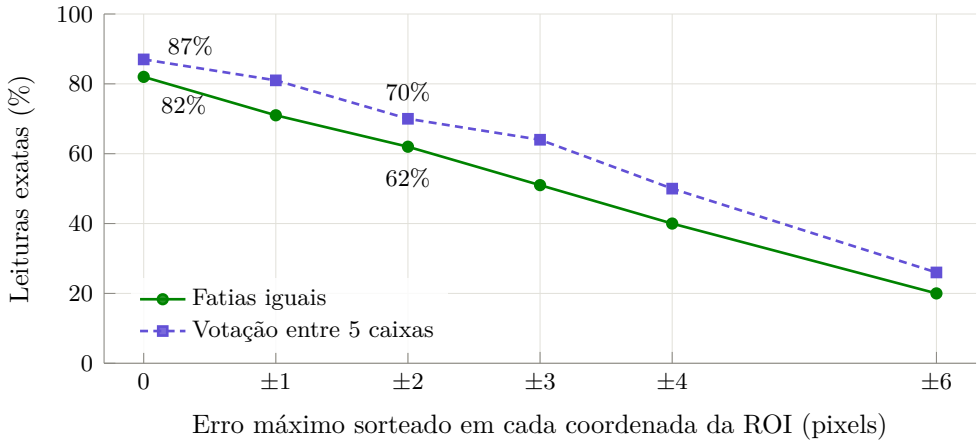
3.4.2.2 A compressão JPEG não afeta a leitura

Com a janela marcada à mão, os dois fluxos tiveram exatamente o mesmo número de leituras exatas (18 de 39) e de dígitos corretos (189 de 222). A compressão JPEG com qualidade 90, que o fluxo em nuvem aplica antes da transmissão, não alterou a leitura. Com a janela prevista pelo modelo, por outro lado, os dois fluxos deram a mesma leitura em apenas 10 das 39 fotos: pequenas diferenças introduzidas pela compressão mudam a máscara prevista e, com ela, a caixa. A compressão afeta o modelo de localização, que já falha, e não o classificador.

3.4.2.3 Precisão exigida da ROI

Como a solução adotada substitui a localização automática por uma ROI calibrada na instalação, a Figura **??** mostra quanto erro de posição a leitura tolera. Sem erro, a leitura exata é de 82% com fatias iguais e de 87% com a votação entre cinco caixas deslocadas. Com até 2 pixels de erro em cada coordenada, cai para 62% e 70%; com até 6 pixels, para 20% e 26%. Como cada dígito tem cerca de 16 pixels de largura no quadro de 160×120 pixels, um erro de 2 pixels já corresponde a um oitavo de dígito. A votação compensa cerca de 1 pixel de erro: com ela, a leitura com ± 2 pixels fica próxima da leitura sem votação com ± 1 pixel. O custo é executar o classificador cinco vezes.

Figura 16 – Leitura exata em função do erro de posição da ROI



Fonte: Elaborado pelos autores (2026). Conjunto comum, fluxo em nuvem, quadro endireitado; média de três sorteios do erro.

Calibrar a ROI com uma única leitura e aplicá-la às demais fotos do mesmo medidor rendeu apenas 26% a 30% das leituras no medidor de seis roletes e 0% no de cinco. Esse cenário, porém, é pessimista: as fotos do conjunto comum são de um monitor cujo zoom e cuja posição mudaram entre leituras, o que deslocou a janela em 3,5 a 5 pixels, em média, no primeiro medidor e em 12 a 18 pixels no segundo. Os valores confirmam que a precisão da calibração é o fator que decide a leitura: em uma instalação com a câmera fixada mecanicamente, espera-se que a janela não se desloque entre leituras, mas essa condição não foi testada.

3.4.2.4 Hidrômetros físicos

No conjunto de hidrômetros físicos, com a janela prevista pelo modelo, o fluxo em borda leu 2 das 21 fotos e o fluxo em nuvem, nenhuma, com 27% a 31% dos dígitos corretos. As duas leituras corretas são de fotos em que a janela de dígitos ocupa a maior parte do quadro, a mesma condição em que a caixa prevista se aproximou da janela no firmware de depuração.

Com a janela e a inclinação marcadas à mão, porém, o resultado mudou pouco: 2 das 21 leituras e 36% a 37% dos dígitos, nos dois fluxos. Ao contrário do conjunto comum, portanto, nos medidores reais a localização da janela não é a única causa dos erros. A Tabela **??** separa as fotos pela polaridade da janela e mostra de onde eles vêm.

Tabela 25 – Leitura do conjunto físico com a janela marcada à mão, por polaridade da janela

		Tons originais		**Tons invertidos**	
Fotos	**Qtd.**	**Leituras**	**Dígitos**	**Leituras**	**Dígitos**
Dígitos escuros em fundo claro, fotos frontais	7	2	43/48	—	—
Dígitos escuros em fundo claro, perspectiva extrema	2	0	2/10	—	—
Dígitos claros em fundo escuro	12	0	11/94	1	40/94

Fonte: Elaborado pelos autores (2026). Fluxo em nuvem; leituras: número de leituras exatas. A inversão só se aplica às fotos de fundo escuro.

Nas sete fotos frontais de medidores com dígitos escuros em fundo claro, o classificador acertou 43 dos 48 dígitos (90%), resultado próximo ao do conjunto comum, e duas fotos foram lidas por inteiro; dos cinco dígitos errados, três estão no último rolete, parado entre dois números. Nas duas fotos com perspectiva extrema, em que os dígitos aparecem inclinados em relação às fatias, a leitura falhou.

Nas 12 fotos de medidores com dígitos claros em fundo escuro, o classificador respondeu 0 na maior parte das fatias e acertou apenas 11 dos 94 dígitos (12%). A causa está no treino: considerando de fundo escuro os recortes cuja mediana dos pixels fica abaixo de 100, apenas 421 dos 49.038 recortes de treino (0,9%) têm essa polaridade, e o modelo responde com a classe mais frequente. Invertendo os tons da janela nessas fotos, o que equivale a informar a polaridade do medidor na instalação, assim como o número de roletes, o acerto subiu para 40 dos 94 dígitos (43%), e uma foto passou a ser lida por inteiro. As fotos que continuaram com erros têm perspectiva forte, reflexos no vidro, baixo contraste, dígitos com cerca de 10 pixels de largura ou a janela cortada pela borda do quadro. A votação entre caixas deslocadas não melhorou o resultado.

Com a janela, a inclinação e a polaridade informadas, o conjunto físico chegou a 3 das 21 leituras (14%) e 56% dos dígitos. Os erros que restam, porém, não têm todos a mesma origem, e a maior parte não está no fluxo nem na janela. A Tabela **??** classifica as fotos pela causa dos erros.

Tabela 26 – Causas dos erros de leitura no conjunto físico

Origem	**Causa**	**Fotos**
Captura	Perspectiva extrema, com os dígitos inclinados em relação às fatias	01, 06, 11, 21, 26
Captura	Janela cortada pela borda do quadro	21, 23, 25
Captura	Reflexo no vidro, baixo contraste ou dígitos com cerca de 10 pixels de largura	02, 05, 15, 16
Treino	Dígitos claros em fundo escuro	02, 03, 05, 06, 07, 11, 13, 15, 16, 21, 23, 24
Treino	Último rolete parado entre dois números	04, 20, 25
Classificador	Dígito visível lido como 0	08, 22

Fonte: Elaborado pelos autores (2026). Uma foto pode ter mais de uma causa. As fotos 12 e 19 foram lidas sem erro com a janela marcada à mão.

As causas de captura são resolvidas por uma instalação adequada, com a câmera de frente para o medidor e a janela inteira no quadro, e não indicam falha do OCR. As causas de treino, ao contrário, são limitações do classificador, e não das fotos. A foto 07, por exemplo, é frontal e nítida, mas foi lida como `00000000` com os tons originais e por inteiro com os tons invertidos. O rolete parado entre dois números, por sua vez, ocorre em qualquer medidor em uso, mas as imagens com essa situação foram descartadas do principal conjunto de treino por terem rótulo ambíguo (Seção 3.2.6.4). Restam, por fim, erros do próprio classificador em dígitos visíveis, raros nas fotos de boa qualidade.

Para estimar o desempenho em uma instalação adequada, foram separadas as fotos que atendem a três condições: câmera frontal, sem perspectiva extrema; dígitos escuros em fundo claro, a polaridade do treino; e todos os dígitos visíveis no quadro. Restaram seis fotos (04, 08, 12, 19, 20 e 22), com 41 dígitos. Com a janela marcada à mão, o classificador acertou 37 dos 41 dígitos (90,2%, com intervalo de confiança de 95% de 77,5% a 96,1%) e leu por inteiro duas das seis fotos, nos dois fluxos; com a janela prevista pelo modelo, o acerto foi de 61% a 66% dos dígitos. Dos quatro dígitos errados, dois estão no último rolete, parado entre dois números, e dois são dígitos visíveis lidos como 0. Esse subconjunto foi definido depois da avaliação do conjunto completo e é pequeno, de modo que deve ser lido como uma indicação do que uma instalação adequada permite, e não como substituto do resultado das 21 fotos.

Os resultados do conjunto físico indicam, portanto, o que uma instalação real precisa: câmera frontal com a janela inteira no quadro; ROI, inclinação e polaridade calibradas; e um classificador treinado também com dígitos claros em fundo escuro, por exemplo pela inversão dos recortes no aumento de dados, e com roletes em transição.

Em síntese, os resultados próximos de zero obtidos com a janela prevista pelo modelo são causados pela localização automática, e não pelo classificador nem pelo fluxo: com a janela informada, a leitura chega a 82% das leituras e 92% dos dígitos no conjunto comum

e a 90% dos dígitos nas fotos de medidores reais que atendem às condições de instalação, e o classificador reconhece 97,7% dos dígitos de um conjunto público independente. Em medidores reais, porém, a janela correta não basta: a geometria da captura, que a instalação resolve, e as limitações do treino quanto à polaridade e aos roletes em transição também limitam a leitura. Como os dois fluxos executam o mesmo pipeline e a compressão não altera a leitura, essas conclusões valem igualmente para a borda e para a nuvem.

3.4.3 Latência (H2)

A Tabela **??** apresenta a latência ponta a ponta dos dois fluxos, medida com o nó no modo autônomo.

Tabela 27 – Latência ponta a ponta dos dois fluxos

Marco, desde o disparo	**Fluxo em nuvem**	**Fluxo em borda**
Captura concluída	0,70 s	—
Leitura pronta no nó	—	18,20 s
Primeiro pacote no computador	1,97 s	18,84 s
Dado disponível no servidor (mediana)	**189,1 s**	**18,84 s**
Mínimo e máximo	172,7 a 213,5 s	18,836 a 18,840 s
Desvio-padrão	11,8 s	0,9 ms
Leituras consideradas	20 imagens completas	27 leituras recebidas

Fonte: Elaborado pelos autores (2026).

A latência do fluxo em borda foi de 18,84 s, cerca de dez vezes menor que os 189,1 s do fluxo em nuvem. Na corrida controlada, com o nó sempre ligado, a latência do fluxo em nuvem foi de 197,1 s (de 148,1 a 204,9 s, com 21 imagens completas), compatível com a corrida em modo autônomo, já que as imagens dessa corrida foram maiores (mediana de 4.379 contra 4.068 bytes).

Os dois fluxos têm gargalos opostos. No fluxo em nuvem, a imagem fica pronta em 0,70 s, e quase toda a latência é a transmissão. Cada bloco leva cerca de 8,1 s: os 7 s do intervalo entre pacotes e cerca de 1,1 s de comando AT, transmissão e confirmação do módulo. Por isso, a latência cresce com o tamanho do JPEG: as três imagens mais rápidas tinham 22 blocos e chegaram em cerca de 173 s, e a mais lenta, com 27 blocos, em 213,5 s. O tempo no ar propriamente dito, de cerca de 3 s por imagem (Tabela 15), representa menos de 2% da latência. Em 3 das 30 leituras, o primeiro bloco só chegou entre 10 e 12 s após o disparo, o que indica que o bloco inicial se perdeu. A execução do OCR no servidor, que não está incluída na medição, leva cerca de 6 ms (Tabela 21) e não altera esses números.

No fluxo em borda, ao contrário, quase toda a latência é processamento. A leitura fica pronta no nó em 18,20 s, dos quais cerca de 15,5 s são a localização da janela e 2,0 s a

classificação dos dígitos, segundo o log do próprio firmware; o envio e a recepção levam mais 0,64 s. Como nenhum desses passos depende da rede, a latência é praticamente determinística, com desvio-padrão de 0,9 ms.

Dois cenários ajudam a avaliar se a conclusão depende das escolhas de implementação. Ambos são estimativas, e não medições. No fluxo em borda, retirar a localização da janela, como prevê a solução com ROI calibrada, reduziria a latência para cerca de 3,3 s. No fluxo em nuvem, mesmo eliminando o intervalo de 7 s entre pacotes, que é uma escolha conservadora e não uma exigência regulatória (Seção 3.2.5.2), restaria cerca de 1,1 s por bloco, ou aproximadamente 26 s para uma imagem de 24 blocos, acima da latência medida no fluxo em borda. Assim, a vantagem do processamento local em latência não depende do intervalo escolhido.

Quanto à latência, a hipótese H2 foi confirmada com folga, em concordância com a menor latência atribuída à computação em borda na Tabela 4 (ANDRIULO et al., 2024). Nesta implementação, a vantagem vem da quantidade de dados transmitida, e não da velocidade do processamento: o microcontrolador é cerca de 2.800 vezes mais lento que o servidor na inferência, mas o fluxo em borda transmite 9 bytes por leitura, contra cerca de 4 KB do fluxo em nuvem.

3.4.4 Entrega dos dados (H2)

A Tabela **??** apresenta a entrega dos dados nas três corridas, por pacote e por leitura.

Tabela 28 – Entrega por pacote e por leitura

Corrida	**PDR por pacote**	**Entrega por leitura**
Fluxo em nuvem, modo autônomo	699/713 (98,0%)	20/30 (66,7%)
Fluxo em nuvem, modo controlado	749/759 (98,7%)	21/30 (70,0%)
Fluxo em borda, modo autônomo	27/34 (79,4%)	27/34 (79,4%)

Fonte: Elaborado pelos autores (2026).

As duas métricas apontam em sentidos opostos. Por pacote, o fluxo em nuvem entregou mais: 98,0% contra 79,4%, diferença estatisticamente significativa (teste exato de Fisher, $p < 0{,}001$). Por leitura, o fluxo em borda entregou mais: 79,4% contra 66,7%; com 30 e 34 leituras, porém, essa diferença não é estatisticamente significativa ($p = 0{,}27$), nem em relação à corrida controlada ($p = 0{,}40$).

No fluxo em nuvem, a diferença entre as duas métricas decorre da fragmentação. Como nenhum dos firmwares retransmite blocos no modo autônomo, basta um bloco perdido para que a imagem não seja remontada: as dez imagens incompletas da corrida em modo autônomo perderam apenas um ou dois blocos cada. Se cada bloco fosse perdido de forma independente, com a taxa observada, seriam esperadas 18,7 imagens completas em 30, e foram obtidas 20; na corrida controlada, 21,5 esperadas e 21 obtidas. Ou seja, a

probabilidade de uma imagem chegar inteira é aproximadamente o PDR elevado ao número de blocos, e uma taxa de 98% por pacote resulta em pouco mais de 60% das imagens de 24 blocos. A retransmissão seletiva dos blocos perdidos recuperaria essas leituras, mas com tempo no ar e energia adicionais, e não foi implementada no modo autônomo.

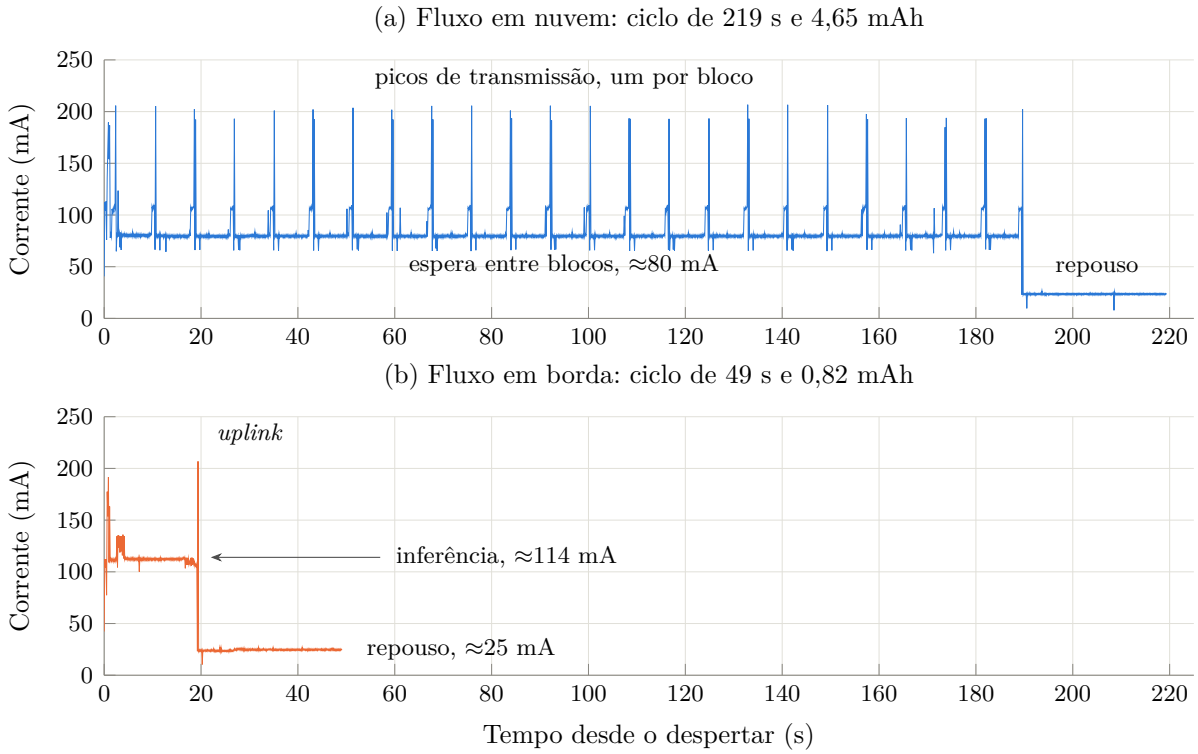
No fluxo em borda, as perdas não se explicam pelo sinal: o RSSI ficou entre -31 e -27 dBm, e, nos dois ciclos perdidos registrados no log do nó, o módulo aceitou o envio sem falhas. As perdas também não se distribuíram de maneira uniforme no fluxo em nuvem. Na corrida controlada, 6 dos 10 blocos perdidos foram o primeiro ou o último de uma imagem, embora esses blocos sejam apenas 60 dos 759 enviados: uma taxa de perda de 10% nas extremidades de cada sequência, contra 0,57% nos blocos do meio (teste exato de Fisher, $p < 0{,}001$). O *uplink* do fluxo em borda é, ao mesmo tempo, o primeiro e o último da sua sequência, e a sua taxa de perda, de 20,6%, não difere significativamente da taxa das extremidades ($p = 0{,}21$). A causa não foi investigada; a coincidência sugere que as perdas estão associadas ao início e ao fim de uma sequência de transmissões, no módulo ou no receptor de canal único, e não ao enlace de rádio.

Quanto à entrega, portanto, a hipótese H2 não foi confirmada: o fluxo em borda entregou mais leituras, mas sem significância estatística, e entregou uma fração menor de pacotes. O resultado mostra ainda que a métrica de interesse da aplicação é a entrega por leitura, e não o PDR: um PDR de 98% por pacote resultou em um terço das imagens perdidas. Em um cenário com retransmissão, a diferença de custo seria grande: o fluxo em borda reenviaria um pacote de 9 bytes, enquanto o fluxo em nuvem precisaria reenviar os blocos faltantes e manter o nó acordado durante a espera.

3.4.5 Consumo de energia e autonomia (H3)

A Figura **??** mostra a corrente do nó durante um ciclo típico de cada fluxo, e a Tabela **??** resume as medições.

Figura 17 – Corrente do nó sensor em um ciclo de leitura de cada fluxo



Fonte: Elaborado pelos autores (2026). Ciclos com carga próxima da mediana de cada corrida; envelope dos valores mínimo e máximo a cada 0,25 s das amostras de 100 Hz.

Tabela 29 – Consumo do nó sensor por ciclo de leitura, com repouso de 30 s

Grandeza	**Fluxo em nuvem**	**Fluxo em borda**
Ciclos medidos	30	32
Carga por leitura (média)	**4,64 mAh**	**0,824 mAh**
Desvio-padrão da carga	0,24 mAh	0,014 mAh
Carga apenas do tempo acordado	4,46 mAh	0,62 mAh
Tempo acordado	189,9 s (173 a 219 s)	19,45 s
Corrente média acordado	84,5 mA	114 mA
Picos de transmissão	207 a 220 mA	206 a 227 mA
Corrente em repouso	23,7 mA	25,2 mA
Período do ciclo	218,5 s	49,1 s

Fonte: Elaborado pelos autores (2026). No fluxo em borda, tempo acordado e corrente acordado são medianas; três ciclos tiveram a fronteira deslocada por picos isolados de uma amostra.

Uma leitura do fluxo em nuvem consumiu 4,64 mAh, contra 0,824 mAh do fluxo em borda: 5,6 vezes mais. Considerando apenas o tempo acordado, sem o repouso, a razão é de 7,2 vezes (4,46 contra 0,62 mAh). A Figura **??** mostra a origem da diferença. No fluxo em nuvem, o nó passa cerca de 190 s acordado, a maior parte em espera entre blocos, com a CPU ligada a cerca de 80 mA; os picos de transmissão acima de 150 mA ocupam só 2,2% do tempo acordado e respondem por 4,9% da carga acordado. O consumo é determinado,

portanto, pelo tempo acordado, que depende do número de blocos e do intervalo entre eles, e não pela energia do rádio. Isso explica também a variação entre ciclos, de 4,26 a 5,24 mAh, que acompanha o tamanho de cada imagem. No fluxo em borda, a corrente é maior durante a inferência, cerca de 114 mA, mas o ciclo é dez vezes mais curto.

O consumo do fluxo em nuvem também mostra o efeito de decisões de firmware. Na versão anterior, que mantinha a câmera ligada durante o envio, a corrente acordado era de 162 mA e a leitura custava 6,62 mAh. Desligar a câmera logo após a captura reduziu a corrente acordado em 48% e a carga por leitura em 30%; a redução da carga foi menor porque as imagens da corrida final foram maiores, o que aumentou o tempo acordado. No fluxo em borda, a localização da janela responde por cerca de 0,48 mAh dos 0,62 mAh acordado, considerando 15,5 s a cerca de 112 mA. Com a ROI calibrada, a carga acordado cairia para cerca de 0,13 mAh, uma estimativa ainda não medida.

3.4.5.1 Autonomia

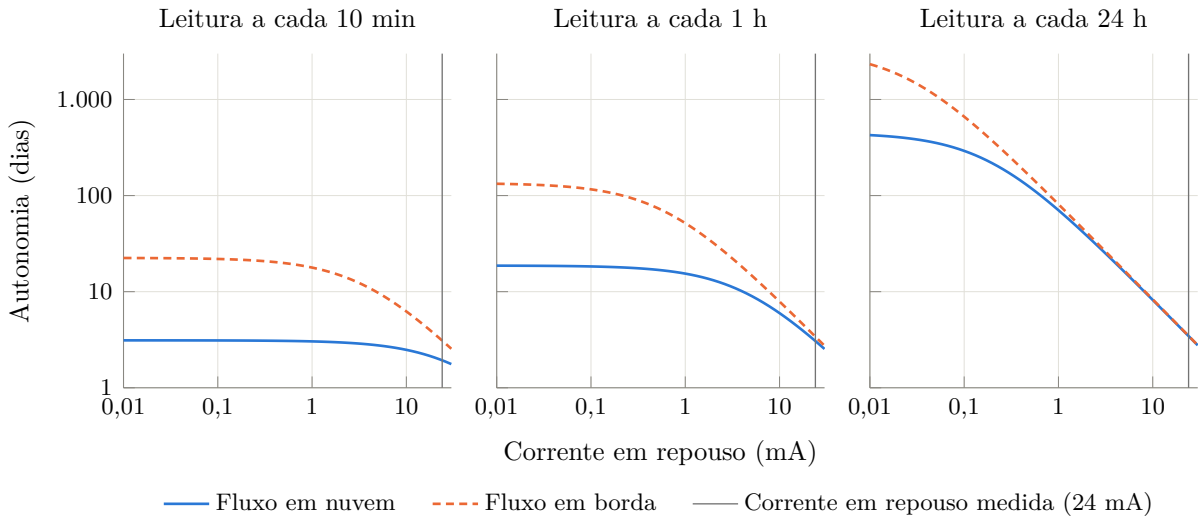
Com repouso de 30 s, a corrente em repouso responde por apenas 4% da carga de uma leitura no fluxo em nuvem, mas por 25% no fluxo em borda. Em intervalos mais longos, ela passa a dominar os dois fluxos. Os cerca de 24 mA medidos em repouso, porém, não são uma característica dos fluxos: incluem o módulo Radioenge, que permanece ligado sem receber o comando de suspensão, e os demais componentes da placa. Segundo o fabricante, o módulo consome 1,8 µA quando suspenso (Radioenge, 2022), de modo que esse piso pode ser muito reduzido. Como a medição que separaria essas parcelas não foi feita, a autonomia foi estimada com o piso medido e com pisos hipotéticos, pela Equação (3.12). O modelo reproduz as cargas medidas (4,65 e 0,825 mAh), e os resultados estão na Tabela **??** e na Figura **??**.

Tabela 30 – Autonomia estimada para uma bateria de 2.000 mAh

Intervalo	**Corrente em repouso**	**Nuvem**	**Borda**	**Borda / nuvem**
10 min	24 mA (medida)	1,9 dia	3,1 dias	1,6
	5 mA	2,8 dias	9,8 dias	3,5
	1 mA	3,0 dias	17,8 dias	5,9
	0,1 mA	3,1 dias	21,9 dias	7,0
1 h	24 mA (medida)	3,1 dias	3,4 dias	1,1
	5 mA	9,1 dias	14,9 dias	1,6
	1 mA	15,4 dias	51,7 dias	3,4
	0,1 mA	18,3 dias	116,1 dias	6,3

Fonte: Elaborado pelos autores (2026). Modelo ideal, sem perdas do regulador nem autodescarga; apenas a corrente de 24 mA foi medida.

Figura 18 – Autonomia estimada em função da corrente em repouso



Fonte: Elaborado pelos autores (2026). Bateria hipotética de 2.000 mAh, modelo da Equação (3.12) com os valores medidos de corrente e tempo acordado de cada fluxo.

Com o piso medido, a vantagem do fluxo em borda na autonomia é pequena: 1,6 vez com leituras a cada 10 minutos e 1,1 vez a cada hora, com autonomias de poucos dias nos dois casos. À medida que o piso diminui, a razão se aproxima da razão entre as cargas acordado, de 7,2 vezes: com 0,1 mA, o fluxo em borda alcança 21,9 dias com leituras a cada 10 minutos, contra 3,1 dias do fluxo em nuvem. A Figura **??** mostra também o efeito do intervalo: com leituras diárias, mesmo um piso de 0,1 mA consome mais que as próprias leituras, e a razão cai para 2,3 vezes (663 contra 292 dias).

Quanto à hipótese H3, o fluxo em nuvem apresentou, de fato, maior consumo por leitura e, portanto, autonomia inferior, de modo que a hipótese foi confirmada. A magnitude da diferença na autonomia, porém, depende menos do fluxo do que da corrente em repouso e do intervalo entre leituras: vai de praticamente nula, com o piso medido e leituras horárias ou diárias, a cerca de sete vezes, com um piso baixo e leituras frequentes. Para um projeto real, o resultado indica que reduzir a corrente em repouso, suspendendo o módulo de rádio e os demais componentes da placa, é condição para que a escolha do fluxo faça diferença na autonomia.

3.4.6 Desempenho computacional e complexidade de desenvolvimento

A pergunta de pesquisa inclui, além da energia e da acurácia, o desempenho computacional e a complexidade de desenvolvimento do firmware, aspectos avaliados a partir das medições do firmware e do servidor (Seções 3.2.7 e 3.2.8) e da experiência de implementação.

Em desempenho computacional, a diferença é de mais de três ordens de grandeza: a leitura completa levou 17.838 ms no ESP32-CAM e cerca de 6 ms no servidor (Tabela 21). A literatura relata inferência abaixo de um segundo para modelos de OCR de medidores em hardware embarcado (ADEMOLA; PETLENKOV; LEIER, 2023; JAWAS; INDRIANTO,

2018); aqui, apenas a classificação dos seis dígitos ficou próxima disso, com cerca de 330 ms por dígito, enquanto a localização da janela levou 15,6 s. A causa não foi investigada (Seção 3.2.7.2). Ao contrário do observado por Bravo, Javier e Tolentino (2025), para quem medidores mais antigos ou obstruídos podem elevar o tempo de processamento, o tempo de inferência aqui não depende do conteúdo da imagem: variou apenas 4 ms em 61 capturas, pois a rede executa sempre as mesmas operações.

A restrição de memória limitou a escolha dos modelos. O fluxo em borda reservou cerca de 2,9 MB da PSRAM para as *arenas* e usou 1,65 MB (Tabela 20), e o pipeline A foi descartado porque pedia uma *arena* de 15,3 MB e um operador ausente na versão do TensorFlow Lite Micro disponível (Seção 3.2.6.1). No servidor, nenhuma dessas restrições existe.

A complexidade de desenvolvimento também foi maior no fluxo em borda, ainda que os dois firmwares tenham tamanhos parecidos, com 525 e 624 linhas. A diferença está fora do laço principal. O fluxo em borda exigiu a quantização completa e a sua validação, a conversão dos modelos em cabeçalhos C, o registro dos operadores de cada modelo, o dimensionamento das *arenas*, que falhou na primeira tentativa e motivou um *script* de estimativa, e um firmware de depuração para observar na placa o que o modelo via. Recursos que melhoraram a leitura, como a correção de inclinação, que elevou a leitura exata de 46% para 82%, e a votação entre caixas, foram implementados apenas no servidor. Por fim, a manutenção também difere: atualizar o modelo no fluxo em nuvem significa trocar um arquivo no servidor, enquanto no fluxo em borda exige regravar o firmware de cada nó, pois a atualização remota não foi implementada. Essas observações confirmam, para este sistema, a menor complexidade de firmware e a maior facilidade de manutenção atribuídas à computação em nuvem, bem como a dificuldade de executar modelos de aprendizado de máquina na borda (Tabela 4) (ANDRIULO et al., 2024).

3.4.7 Síntese e discussão

A Tabela **??** resume o resultado de cada hipótese.

Tabela 31 – Síntese dos resultados das hipóteses

Hipótese	**Resultado medido**	**Situação**
H1	INT8 com 95,38% contra 95,46% do modelo em ponto flutuante, diferença de 0,08 ponto percentual sem significância estatística ($p = 0{,}29$)	Confirmada no sentido, mas a redução é desprezível
H2, latência	18,84 s no fluxo em borda contra 189,1 s no fluxo em nuvem	Confirmada
H2, entrega	Por leitura, 79,4% contra 66,7%, sem significância ($p = 0{,}27$); por pacote, 79,4% contra 98,0%	Não confirmada
H3	0,824 contra 4,64 mAh por leitura (5,6 vezes); autonomia de 1,0 a 7,0 vezes maior, conforme a corrente em repouso e o intervalo	Confirmada, com magnitude dependente do repouso

Fonte: Elaborado pelos autores (2026).

Em relação à pergunta de pesquisa, os resultados indicam que, em um enlace LoRaWan de baixa taxa, o volume de dados transmitido pesa mais do que a capacidade de processamento. Mesmo com uma inferência cerca de 2.800 vezes mais lenta que a do servidor, o fluxo em borda teve latência dez vezes menor e consumo por leitura de cinco a sete vezes menor, porque transmite cerca de 450 vezes menos dados. A acurácia não diferenciou os fluxos: ambos executam os mesmos modelos, a quantização teve efeito desprezível e a compressão JPEG não alterou a leitura. O limite da acurácia, nos dois casos, está na localização da janela de dígitos, que a calibração da ROI na instalação resolve quando a câmera é fixa, e, em medidores reais, também na geometria da captura e no treino do classificador, que não cobre dígitos claros em fundo escuro nem roletes em transição. O custo do fluxo em borda aparece na complexidade de desenvolvimento e de manutenção e na restrição de memória, que impede o uso de modelos maiores e de recursos como a correção de inclinação.

Em comparação com os trabalhos correlatos, Bravo, Javier e Tolentino (2025) relatam precisão de 99,71% no reconhecimento dos dígitos com OCR executado no próprio ESP32-CAM. O classificador deste trabalho reconheceu 97,7% dos dígitos em um conjunto público independente e 92% nas fotos do ESP32-CAM com a janela informada. Como os conjuntos de dados são diferentes, os números não são diretamente comparáveis, mas os resultados aqui obtidos indicam que, nesse tipo de sistema, a acurácia depende menos da arquitetura do classificador do que de localizar corretamente a janela de dígitos e de o treino cobrir a variedade dos medidores reais, como a polaridade dos dígitos. Em relação a Vilela (2021), cujo trabalho não apresentou métricas de consumo energético e de processamento, este trabalho quantifica essas métricas para os dois métodos de processamento: carga por leitura, latência, tempo de inferência e uso de memória.

3.4.8 Limitações

Os resultados devem ser interpretados considerando as seguintes limitações:

- **Imagens de teste**: o conjunto comum, as fotos de ajuste fino e as de calibração foram capturadas de um monitor que exibia imagens de hidrômetros, sem os reflexos, a iluminação e os ângulos de um medidor instalado. O conjunto de hidrômetros físicos tem apenas 21 fotos, de medidores diferentes e não de uma instalação fixa, e as suas janelas, inclinações e polaridades foram marcadas pelos autores, sem conferência independente. O subconjunto que representa uma instalação adequada tem apenas seis fotos e foi definido após a avaliação do conjunto completo.

- **Acurácia emulada**: a acurácia dos dois fluxos foi medida por uma emulação em Python com os mesmos modelos. Na placa, as leituras não tinham gabarito; na corrida de latência do fluxo em borda, com a câmera fixa, as 27 leituras recebidas tiveram 13 valores distintos, instabilidade que não foi investigada.

- **ROI não implementada no firmware**: o firmware medido ainda executa a localização da janela. Os valores de leitura em cerca de 2 s, latência de cerca de 3,3 s e carga acordado de cerca de 0,13 mAh para a versão com ROI são estimativas.

- **Cenário de enlace**: as medições foram feitas em bancada, a 2 m de distância, com sinal forte, um receptor de canal único no lugar de um *gateway* e sem retransmissão. As execuções em que o receptor ficou de 15 a 25 minutos sem repassar quadros foram excluídas por se tratar de uma falha do receptor, cuja causa não foi identificada; a sua ocorrência indica que o receptor usado é um ponto frágil do protótipo. As amostras, de 30 a 34 leituras por fluxo, só permitem detectar diferenças grandes de entrega.

- **Energia**: a corrente em repouso inclui o módulo de rádio e os componentes da placa, e a medição que separaria essas parcelas não foi feita. O nó foi alimentado pela porta USB, e não pela bateria e pelo regulador de uma instalação real, e as autonomias vêm de um modelo ideal com bateria hipotética.

- **Instante de disparo**: o disparo foi marcado de 0,3 a 0,5 s após o despertar real, nos dois fluxos, e a latência do fluxo em nuvem não inclui a execução do OCR no servidor, de cerca de 6 ms.

- **Escala**: os experimentos usaram um único nó, um único receptor e poucos medidores, de modo que não avaliam o comportamento de uma rede com vários dispositivos compartilhando o canal.

4 Conclusão

4.1 Retomada do problema e dos objetivos

Retome o problema e destaque o que foi atingido.

4.2 Contribuições para a Computação

Explique o que a pesquisa trouxe de novo.

4.3 Limitações e trabalhos futuros

Liste limites e sugira próximos passos coerentes.

Observações importantes

Para graduação, Wazlawick aceita trabalhos aplicados desde que:

- O problema seja preciso;
- A solução esteja fundamentada;
- Haja análise crítica;
- Não seja apenas entrega funcional.

Glossário

Cloud Computing Método de processamento em nuvem 2, 3, 11, 23, 24, 41

Edge Computing Método de processamento local 2, 3, 11, 12, 23, 24, 30, 37, 41

ESP32-CAM Módulo para acoplamento no microprocessador ESP32, conferindo ao dispositivo a capacidade de capturar imagens 2–4, 13, 15, 38, 41–43, 47, 50, 51, 54–59, 62, 66, 68, 69

payload Caracteres enviados no corpo do pacote de rede 17, 22

retrofitting Técnica de modernização ou atualização de dispositivos e equipamentos 2, 3, 12, 14

Smart Metering Tecnologias para Medição Inteligente 2, 3, 12

Referências

ADAM, L. *Desenvolvimento e implantação de solução para comunicação bidirecional de longo alcance com medidores de energia elétrica*. Dissertação (Dissertação (Mestrado em Engenharia Elétrica)) — Universidade Federal de Santa Maria, Santa Maria, RS, 2022. Programa de Pós-Graduação em Engenharia Elétrica, Área de Concentração em Sistemas Elétricos de Potência. 11, 25

ADEMOLA, O. A.; PETLENKOV, E.; LEIER, M. Resource-aware scene text recognition using learned features, quantization, and contour-based character extraction. *IEEE Access*, v. 11, p. 56865–56874, 2023. Revista de acesso aberto (IEEE Access). 36

AHMAD, S. et al. Comparative analysis and practical implementation of the esp32 microcontroller module for the internet of things. In: *2021 International Conference on Computing, Electronic and Electrical Engineering (ICE Cube)*. [S.l.: s.n.], 2021. p. 1–6. 20

ALVISI, S. et al. Wireless middleware solutions for smart water metering. *Sensors*, v. 19, n. 8, 2019. 12

ANATEL. *Ato nº 14.448, de 4 de dezembro de 2017. Aprova os Requisitos Técnicos para a Avaliação da Conformidade de Equipamentos de Radiocomunicação de Radiação Restrita*. 2017. Boletim de Serviço Eletrônico da Anatel, Brasília, 02 jan. 2018. Acesso em: 16/09/2026. Disponível em: <https://informacoes.anatel.gov.br/legislacao/atos-de-certificacao-de-produtos/2017/1139-ato-14448>. 24, 25, 45

ANDRIULO, F. C. et al. Edge computing and cloud computing for internet of things: A review. *Informatics*, MDPI, v. 11, n. 4, p. 71, 2024. Disponível em: <https://doi.org/10.3390/informatics11040071>. 12, 14, 23, 24

ATIKA, L. et al. Image segmentation for mitral regurgitation with convolutional neural network based on unet, resnet, vnet, fractalnet and segnet: A preliminary study. *Big Data and Cognitive Computing*, MDPI, v. 6, n. 4, p. 141, 2022. Disponível em: <https://doi.org/10.3390/bdcc6040141>. 31, 34

AYENI, J. A. Convolutional neural network (cnn): The architecture and applications. *Applied Journal of Physical Science*, Integrity Research Journals, v. 4, n. 4, p. 42–50, 2022. ISSN 2756-6684. 31, 32

BARKAT, S. *Smart Water Management System Using Deep Learning and IoT-Based Predictive Modeling*. Master's Thesis — Department of Computer Science, 2024. 40

BRAVO, J. K. G.; JAVIER, C. P.; TOLENTINO, V. T. Ants: Automated network tracking system for water consumption using lora technology. *FAITH Colleges*, 2025. Acesso em: 23 abr. 2026. 36, 38, 39, 40

CHEN, X. et al. Text recognition in the wild: A survey. *ACM Computing Surveys*, v. 54, n. 2, p. 1–35, 2021. Versão de acesso aberto: arXiv:2005.03492. 32, 33

FERREIRA, A.; CAVALCANTE, A. L. de C. Iot on an esp32: Optimization methods regarding battery life and write speed to an sd-card. *Journal of Software Engineering and Applications*, v. 13, n. 9, 2020. 20

FERREIRA, G. B. *Compressão de redes neurais artificiais com restrições de acurácia utilizando otimização multiobjetivo baseada em modelos substitutos*. 38 p. Monografia (Monografia (Bacharelado em Ciência da Computação)) — Universidade Federal de Ouro Preto, Instituto de Ciências Exatas e Biológicas, Departamento de Computação, Ouro Preto, MG, 2022. 36

Fortune Business Insights. *Smart Water Metering Market Size, Share & Industry Analysis*. [S.l.], 2025. Disponível em: <https://www.fortunebusinessinsights.com/pt/industry-reports/smart-water-metering-market-100776>. Acesso em: 24 abr. 2026. 11

GRAVES, A. et al. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In: *Proceedings of the 23rd International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2006. p. 369–376. PDF público: <https://www.cs.toronto.edu/~graves/icml_2006.pdf>. 35, 36

HANNUN, A. Sequence modeling with ctc. *Distill*, v. 2, n. 11, p. e8, 2017. 30, 33, 35, 36

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. PDF de acesso aberto disponível em: <https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.1735.pdf>. 29, 30

IQBAL, A. *Digits (0-9) Dataset*. 2019. Kaggle. Conjunto de dados com imagens dos algarismos de 0 a 9 em diferentes fontes e estilos. Disponível em: <https://www.kaggle.com/datasets/aquibiqbal/digits-09>. 51

JAWAS, N.; INDRIANTO. Image based automatic water meter reader. *Journal of Physics: Conference Series*, v. 953, p. 012027, 2018. Artigo de acesso aberto (IOP Conference Series). 32, 36

JESUS, V. A. de; COQUEIRO, T. A. S. Inteligência artificial: Uma breve revisão bibliográfica de ferramentas de machine learning e deep learning na medicina. *Faculdade de Computação, Universidade Federal do Pará (UFPA)*, Castanhal, PA, Brasil, 2023. Trabalho de Conclusão / Artigo Acadêmico. 29, 30, 31

KUCEV, R. *Water Meters Dataset*. Kaggle, 2020. Disponível em: <https://www.kaggle.com/datasets/tapakah68/yandextoloka-water-meters-dataset>. Acesso em: 02 jun. 2026. 51, 54

LoRa Alliance Regional Parameters Workgroup. *LoRaWAN Regional Parameters RP002-1.0.5*. [S.l.], 2025. Authored by the Technical Committee Regional Parameters Workgroup. Editor: D. Tholl. Disponível em: <https://lora-alliance.org/>. 17, 25, 26, 27, 28, 46

NAZRE, R. et al. A temporal convolutional network-based approach for network intrusion detection. In: *arXiv preprint arXiv:2412.17452*. [S.l.: s.n.], 2024. PDF de acesso aberto: <http://export.arxiv.org/pdf/2412.17452v1>. 30

NGUYEN, V. K. et al. Solution for direct implementation of recurrent neural networks on microcontrollers. *Journal of Science and Technology*, 2026. Artigo de acesso aberto. Disponível em: <https://jst.tnu.edu.vn/jst/article/view/15162>. 30

PASTóRIO, A. F. et al. Fundamentos de lorawan – teoria e prática. *Federal University of Technology of Paraná*, 2022. 15, 18, 19, 20, 21, 22

Radioenge. *EndDevice LoRaWAN Radioenge: manual de operação*. Revisão de junho de 2022. [S.l.], 2022. 45, 46

Radioenge. *Gateway LoRaWAN*. 2026. Radioenge Indústria de Telecomunicações. Disponível em: <https://www.radioenge.com.br/produto/gateway-lorawan/>. Acesso em: 02 nov. 2026. 21

ROBINSON, S. *StuartCAM – ESP32CAM Picture Transfers with LoRa*. 2021. Disponível em: <https://stuartsprojects.github.io/2021/11/21/StuartCAM-ESP32CAM-Picture-Transfers-with-LoRa.html>. 20

SARI, M. E. et al. Design and implementation of esp32-based iot devices. *Sinkron: Jurnal dan Penelitian Teknik Informatika*, v. 7, n. 1, 2022. 20

Semtech Corporation. *AN1200.13: SX1272/3/6/7/8 LoRa Modem Design Guide*. [S.l.], 2013. Disponível em: <https://www.semtech.com>. 48

Semtech Corporation. *AN1200.86: LoRa and LoRaWAN*. [S.l.], 2024. Application Note Version 1.0. Disponível em: <https://www.semtech.com>. 18, 19, 20, 21, 22, 26

Semtech Corporation. *AN1200.87: LoRaWAN Device Classes*. [S.l.], 2024. Application Note Version 2.0. Disponível em: <https://www.semtech.com>. 16

Semtech Corporation. *AN1200.88: Sending and Receiving Messages*. [S.l.], 2024. Application Note Version 2.0. Disponível em: <https://www.semtech.com>. 16, 17

SHI, B.; BAI, X.; YAO, C. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. 39, n. 11, p. 2298–2304, 2017. Versão de acesso aberto: arXiv:1507.05717. 30, 33, 35, 36

TEDJOJUWONO, S. M.; JAHJA, J. D. Smart water management system using IOT based sensors. In: *Proceedings of the 7th International Conference on Eco Engineering Development 2023 (ICEED 2023)*. [S.l.]: IOP Publishing, 2024. (IOP Conference Series: Earth and Environmental Science, 012106), p. 1–9. 11, 36, 37

UGGERI, D.; SIMEON, E. *Smart water metering: analytical model computation for an estimated costs and benefits analysis*. Dissertação (Mestrado) — Politecnico di Milano, 2020. Disponível em: <https://www.politesi.polimi.it/handle/10589/171225>. Acesso em: 20 abr. 2026. 11

VILELA, R. da S. Solução iot para digitalização de medidores de consumo por visão computacional e microml. *Universidade Federal de Campina Grande*, 2021. 11, 12, 28, 29, 36, 37

ZHAO, S. et al. A comprehensive dataset for word-wheel water meter reading under challenging conditions. *Scientific Data*, v. 13, p. 479, 2026. 54